

Venzini

Software Design Document

An AI-Assisted Yield Management Platform for Data-Driven Optimization in the Petroleum Industry

Prepared by: Mohamed Zouari

Follows IEEE 1016-2009

Document control

This SDD turns the Venzini SRS (final version) into a buildable design for all of version 1 and for the MVP prototype. It follows IEEE 1016-2009.

Item	Value
Document	Venzini – Software Design Document (SDD)
Author	Mohamed Zouari
Standard	IEEE 1016-2009, Software Design Descriptions
Main source	Venzini 2 – SRS FV (decisions D-01 to D-28 in its Appendix A)
Precedence	Where this SDD and the SRS differ, the SRS applies

Labels used in this document

- **SRS IDs** – FR, NFR, BR, AC, UI, UC, API, DE, CMP, IF, ERR, OBS and D keep their SRS meaning. New IDs in this SDD: **MOD** (backend module), **TBL** (database table), **SEQ** (sequence diagram), **SM** (state machine), **ALG** (algorithm), **TPL** (explanation template), **ADR** (design decision record).

Contents

1. Introduction 9

1.1 Purpose 9

1.2 Scope 9

1.3 Intended readers 9

1.4 Document overview 9

2. References 10

3. Glossary and abbreviations 11

4. Design stakeholders and concerns 13

4.1 Stakeholders and their design concerns 13

4.2 Concerns mapped to views 13

4.3 Viewpoints used 14

5. Design views 15

5.1 Context view 15

5.2 Composition view 16

5.3 Logical view 23

5.4 Dependency view 27

5.5 Information view 28

5.6 Interface view 46

5.7 Interaction view 58

5.8 State-dynamics view 67

5.9 Algorithm view 69

5.10 Resource view 89

5.11 Deployment view 91

5.12 UI design view 94

6. Cross-cutting concerns 99

6.1 Security 99

6.2 Error handling 101

6.3 Logging and metrics 102

6.4 Configuration 102

6.5 Time zones and units 103

7. MVP design 103

7.1 Mock data layer 104

7.2 Fixtures 104

7.3 Screens and flows included 105

7.4 Path to the real backend 106

8. Testing strategy 106
8.1 Levels and tools 106
8.2 Test data 108
8.3 Key test cases by design element 109
8.4 How AC-01 to AC-21 are verified 109

9. Design rationale 111

Appendix A. Traceability matrix 113
A.1 Functional requirements 113
A.2 Non-functional requirements and AI capabilities 116

Appendix B. Repository and folder structure 118

List of figures

Figure 1 · System context 15
Figure 2 · System architecture 17
Figure 3 · Data flow diagram, level 1 18
Figure 4 · Venzini tech stack 18
Figure 5 · Container diagram 19
Figure 6 · Backend modules 20
Figure 7 · Frontend structure 21
Figure 8 · Class diagram, planning modules 23
Figure 9 · Class diagram, platform and foundation modules 23
Figure 10 · Domain model 24
Figure 11 · Module dependency graph 27
Figure 12 · ERD part 1: users, stations and forecasts 29
Figure 13 · ERD part 2: tankers, drivers and daily resources 30
Figure 14 · ERD part 3: plans, routes and decisions 31
Figure 15 · Full entity-relationship diagram, table names only 31
Figure 16 · SEQ-01 Login with lockout 59
Figure 17 · SEQ-02 Generate plan 60
Figure 18 · SEQ-03 Reorder or move with live recalculation 61
Figure 19 · SEQ-04 Quantity edit 62
Figure 20 · SEQ-05 Approve or reject one action 63
Figure 21 · SEQ-06 Approve all or reject all 64
Figure 22 · SEQ-07 Dispatcher uploads resources 65
Figure 23 · SEQ-08 Dispatcher records trip times 66
Figure 24 · SM-01 Plan 67
Figure 25 · SM-02 Recommended action 67
Figure 26 · SM-03 Trip 68
Figure 27 · SM-04 User account 68
Figure 28 · ALG-01 forecast for ST014, Petrol 95 71
Figure 29 · ALG-02 confidence and 80% band 72
Figure 30 · ALG-03 stockout, risk and stock allocation 74
Figure 31 · ALG-04 Routing optimizer 75
Figure 32 · ALG-04 insertion, 2-opt and local search 77
Figure 33 · ALG-05 route cost and plan comparison 78
Figure 34 · ALG-06 risk severity scales 80
Figure 35 · ALG-07 maximum quantity and binding limit 81
Figure 36 · ALG-09 road network and distance matrix 85
Figure 37 · Algorithm complexity 87

Figure 38 · Asymptotic bounds per algorithm 88
Figure 39 · Asymptotic bounds of Venzini 89
Figure 40 · Local deployment with Docker Compose 91
Figure 41 · Screen navigation map per role 96
Figure 42 · Venzini design system 97
Figure 43 · Preliminary designs, visual reference only. 99
Figure 44 · MVP versus v1 104

List of equations

Equation 1 · Multiplicative seasonal forecast model (ALG-01) 69
Equation 2 · Seasonal moving average (weekly baseline) (ALG-01) 69
Equation 3 · Clamped seasonal index ratio (ALG-01) 70
Equation 4 · Additive event uplift factor (ALG-01) 70
Equation 5 · Clamped delivery-to-sales ratio (ALG-01) 70
Equation 6 · Gaussian noise term (seeded normal distribution) (ALG-01) 70
Equation 7 · Mean relative factor impact (ALG-01) 70
Equation 8 · Coefficient of variation (ALG-02) 71
Equation 9 · Linear confidence penalty score (ALG-02) 72
Equation 10 · Forecast error standard deviation (ALG-02) 72
Equation 11 · 80% prediction interval (normal quantile z = 1.28) (ALG-02) 72
Equation 12 · Minimum aggregation and threshold rule (ALG-02) 72
Equation 13 · Cumulative stock depletion (ALG-03) 73
Equation 14 · Linear interpolation of time to stockout (ALG-03) 73
Equation 15 · Piecewise threshold classification (ALG-03) 73
Equation 16 · Lexicographic sort key (ALG-03) 73
Equation 17 · Floor-rounded capped refill quantity (ALG-03) 74
Equation 18 · Nearest-neighbour assignment (argmin) (ALG-03) 74
Equation 19 · Greedy sequential allocation recurrence (ALG-03) 74
Equation 20 · Arrival-time recurrence (time-window propagation) (ALG-04) 75
Equation 21 · Linear service-time model (ALG-04) 75
Equation 22 · Second-trip feasibility constraint (ALG-04) 76
Equation 23 · Lexicographic multi-objective function (ALG-04) 76
Equation 24 · Cheapest-insertion cost (detour) (ALG-04) 76
Equation 25 · 2-opt move gain (ALG-04) 76
Equation 26 · Linear route cost model (ALG-05) 77
Equation 27 · Total plan cost (ALG-05) 77
Equation 28 · Absolute cost difference (ALG-05) 78
Equation 29 · Relative percentage change (ALG-05) 78
Equation 30 · Capacity violation condition (RISK-01) (ALG-06) 79
Equation 31 · Newly caused stockout condition (RISK-02) (ALG-06) 79
Equation 32 · Newly caused time-window violation (RISK-03) (ALG-06) 79
Equation 33 · Lateness step function (ALG-06) 79
Equation 34 · Percentage-increase step function (ALG-06) 79
Equation 35 · Maximum compartment capacity (ALG-07) 81
Equation 36 · Free tank capacity (ALG-07) 81
Equation 37 · Remaining warehouse stock (ALG-07) 81
Equation 38 · Minimum of constraints (binding limit) (ALG-07) 81
Equation 39 · Travel time from distance and speed (ALG-09) 84
Equation 40 · Distance matrix initialisation (ALG-09) 84
Equation 41 · Floyd–Warshall recurrence (edge relaxation) (ALG-09) 84
Equation 42 · Shortest-path travel time (ALG-09) 84
Equation 43 · Big O definition (upper bound) (5.9.11) 87
Equation 44 · Big Omega definition (lower bound) (5.9.11) 87
Equation 45 · Big Theta definition (tight bound) (5.9.11) 87

1. Introduction

1.1 Purpose

This document describes how Venzini version 1 is built: its parts, its data, its interfaces, its algorithms and its screens. It is precise enough for one developer, helped by an AI coding assistant, to implement and test v1 and the MVP without guessing. Every design element traces back to an SRS ID. Design choices that go beyond the SRS were reviewed and approved.

1.2 Scope

- **Covered:** all of version 1 – the Must core and the Should items – plus the MVP, a clickable frontend-only prototype with fake data (AC-21, D-26).
- **Built on:** the decided stack (Section 5.2): React 19 frontend, NestJS modular monolith, PostgreSQL 17 with Prisma, forecasting and routing engines written as TypeScript modules inside the backend, local Docker Compose deployment.
- **Not covered:** future versions (station-manager interface, data-provider role, real data integrations, and in-app, push or email notifications planned for a future mobile version). Where a v1 design choice prepares for them, the section says so. In v1, users are told about new events only by page banners and tab badges.
- **Not built:** other operational resources (FR-10). Stock allocation follows the ALG-03 priority rule (FR-26).

1.3 Intended readers

Reader	Reads mainly	To
Developer (Mohamed Zouari)	All sections	Build the MVP, then v1
AI coding assistant	Sections 5.3 to 5.11, 7, Appendix B	Generate modules, schema, contracts and tests that match the design
Academic supervisors and reviewers	Sections 4, 5.1, 5.2, 9, Appendix A	Check that the design is complete, consistent and traceable
Tester	Sections 5.6, 5.7, 8, Appendix A	Derive and run tests

1.4 Document overview

- **Sections 2–4:** references, glossary, and who cares about which part of the design.
- **Section 5:** the twelve IEEE 1016 design views, from the system boundary down to the screens.
- **Sections 6–8:** concerns that cut across all modules, the MVP design, and the testing strategy.
- **Section 9:** the main design decisions, with the options considered (ADR style).
- **Appendices:** traceability and the repository layout.

Figures are numbered in reading order and drawn in one style: white boxes, navy (#021C45) borders, headers and arrows, and light navy group areas. Red, orange and green appear only for risk levels, as in the product itself.

2. References

Ref	Document	Use in this SDD
R1	Venzini 2 – SRS FV, M. Zouari, 25 Oct 2026	Main source of truth: FR-01–85, NFR-01–31, BR-01–31, AC-01–21, UI-01–08, UC-01–11, API-01–19, DE-01–23, CMP-, *IF-*, ERR-, *OBS-*, D-01–28
R2	Venzini – Project Scope FV, M. Zouari	Scope, MVP and v1 split, out of scope
R3	Venzini 1 – Requirement Elicitation FV, M. Zouari	Stakeholders, user needs, use cases
R4	IEEE 1016-2009, Standard for Information Technology – Systems Design – Software Design Descriptions	Structure: stakeholders, concerns, viewpoints and views
R5	Preliminary designs “Venzini core screens” (Google Stitch)	Visual reference for the UI design view only
R6	W3C, Web Content Accessibility Guidelines (WCAG) 2.1	Contrast level AA (NFR-11)
R7	OWASP Top 10 (2021)	Security baseline for Section 6.1

3. Glossary and abbreviations

Business terms (station, tanker, trip, route, stop, plan, safety stock, station risk, confidence and so on) keep the meaning given in SRS Section 1.4. The table below adds the design and technical terms used in this SDD.

Term	Meaning, with an example
ADR	Architecture Decision Record: a short note of one design decision, the options and why one was chosen (Section 9)
Adapter	A class that hides an outside data source behind a fixed interface. Example: SimulatedWeatherAdapter implements IF-WEATHER; a real weather adapter can replace it later
Advisory lock	A named lock held by PostgreSQL for the length of a transaction. Example: saving resources and approving a plan for 3 Nov take the same lock, so they never run at once
API contract	The exact shape of a request and response for one endpoint, written as a Zod schema and published as OpenAPI
Append-only table	A table that accepts new rows but refuses changes and deletions. Example: the decision log (BR-13)
Argon2id	A modern password-hashing method; it stores a slow, salted fingerprint of the password, never the password itself
Container (Docker)	A packaged, isolated process with everything it needs to run. Venzini runs three: web, api, db
Correlation ID	A unique ID given to each request and written on every log line it causes, so one request can be followed end to end
CSRF	Cross-site request forgery: another website tricking the browser into sending a request with the user's cookie
CVRPTW	Capacitated Vehicle Routing Problem with Time Windows: choosing which stops each tanker visits and in what order, without exceeding tanker capacity and while arriving inside each delivery window
Cheapest insertion	A routing method that adds stations one by one, each at the position in a route where it adds the least cost
2-opt	An improvement step that reverses a section of a route when that makes it shorter. Example: A-C-B-D becomes A-B-C-D
Relocate	An improvement step that moves one stop to another position or another route of the same warehouse

Term	Meaning, with an example
Deterministic	Same inputs, same outputs, every time (NFR-13). Achieved with fixed seeds and fixed ordering
Fixture	A fixed set of fake data used by the MVP and by tests. Example: stations.json with 20 stations
JWT	JSON Web Token: a signed token that says who the user is. Venzini keeps it in an HttpOnly cookie and checks it against a session row
HttpOnly cookie	A cookie the browser sends to the server but page scripts cannot read, which protects it from script injection
Migration	A versioned script that changes the database schema, applied in order by Prisma
Mock layer / MSW	Mock Service Worker: a browser library that answers the frontend's real /api/v1 requests with fake data. It powers the MVP
Modular monolith	One deployable backend split into modules with clear boundaries. Example: the routing module cannot reach into the decision module's tables
Monorepo	One Git repository holding the web app, the API and shared packages
ORM	Object-Relational Mapper: a library (Prisma) that maps database tables to typed code
PRNG / seed	A pseudo-random number generator; the seed is its starting value. The same seed gives the same "random" numbers
RBAC	Role-Based Access Control: what each role may see and do (SRS Section 4.3)
Session row	A database row that records an active login; deleting or revoking it ends the session at once
SPA	Single-Page Application: the React app that runs in the browser and calls the API
Snapshot	A frozen JSON copy of data at one moment. Example: the plan as it stood when a decision was taken
Zod	A TypeScript library to describe and validate data shapes; the same schema checks forms in the browser and requests on the server

Abbreviations: API – application programming interface · CI – continuous integration · DB – database · ERD – entity–relationship diagram · E2E – end to end · FK – foreign key · PK – primary key · REST – resource-based HTTP API style · UI – user interface · UTC – Coordinated Universal Time.

4. Design stakeholders and concerns

Each design concern below is owned by at least one stakeholder and answered by at least one view. Stakeholders come from the Requirement Elicitation (Section 2); only those with a design concern are listed.

4.1 Stakeholders and their design concerns

ID	Stakeholder	Design concerns
SH-1	Operations manager	Trustworthy, explained forecasts; edits that update in ≤ 1 s; clear risks and costs; nothing approved behind their back
SH-2	Dispatcher	Fast resource entry; certainty the plan uses the latest resources; clear lock after the plan is decided
SH-3	System admin	Correct role enforcement; simple user and station management
SH-4	Distributor management	Every decision traceable; no action without approval; no price or equipment control
SH-5	Developer (solo)	Small, buildable design; modules that can be built and tested one at a time; MVP reusable for v1
SH-6	Academic reviewers	IEEE 1016 completeness; traceability to the SRS; justified decisions
SH-7	Tester	Testable interfaces, deterministic results, scenario data
SH-8	Regulators (indirect)	Plans never exceed tank or tanker capacity

4.2 Concerns mapped to views

Concern	Stakeholders	Main SRS refs	Views that answer it
C-01 What is inside and outside the system	SH-4, SH-6	Section 2, Section 5	5.1 Context
C-02 How the system is split into parts and with which technology	SH-5, SH-6	CMP-*, D-12	5.2 Composition, 5.11 Deployment
C-03 Responsibilities of each module and class	SH-5, SH-7	CMP-*, AIC-01–04	5.3 Logical
C-04 Modules can be replaced and	SH-5	NFR-12, NFR-15	5.4 Dependency,

Concern	Stakeholders	Main SRS refs	Views that answer it
built separately			5.6 Interface
C-05 Data correctness, retention and audit	SH-4, SH-8	DE-01–23, BR-13, NFR-14	5.5 Information
C-06 Exact API behavior and errors	SH-5, SH-7	API-01–19, ERR-*	5.6 Interface
C-07 Flows behave as the use cases say	SH-1, SH-2, SH-7	UC-01–11	5.7 Interaction
C-08 Lifecycle of plans, actions, trips, accounts	SH-1, SH-4	Section 8.5, BR-30	5.8 State dynamics
C-09 Forecasts, risks and routes are correct and explained	SH-1, SH-8	AIC-01–04, BR-26–28, D-18	5.9 Algorithm
C-10 Speed and safe concurrent use	SH-1, SH-2	NFR-01, -02, -19, -20, -24	5.10 Resource
C-11 One-command local start	SH-5, SH-6	NFR-30	5.11 Deployment
C-12 Usable, accessible screens per role	SH-1, SH-2, SH-3	UI-01–08, NFR-08–11, NFR-27	5.12 UI design
C-13 Security and privacy	SH-3, SH-4	Section 12, NFR-05–07, -21–23, -25	6.1 Security
C-14 Errors, logs and metrics	SH-5, SH-7	Sections 14–15, NFR-29	6.2, 6.3
C-15 A clickable MVP that leads to v1 without rework	SH-5, SH-6	AC-21, D-26	7 MVP design
C-16 Every requirement is verified	SH-6, SH-7	Sections 16–17	8 Testing, Appendix A

4.3 Viewpoints used

Each view in Section 5 follows the IEEE 1016 viewpoint of the same name: context, composition, logical, dependency, information, interface, interaction, state dynamics, algorithm, resource, and deployment (physical). The UI design view is an extra viewpoint added for this product, because three of the four main concerns of the operations manager are about screens.

5. Design views

5.1 Context view

Venzini v1 is one web application used by three roles. It reads three kinds of external data, all simulated in v1, through one interface each, and it hands nothing to anyone outside the platform (Figure 1).

Figure 1 below shows the system context. Solid lines are data connections of the platform; the map tile API is an outside service called by the browser only, and no Venzini data is sent to it. The dashed line is not a system interface: the approved plan leaves the platform offline.

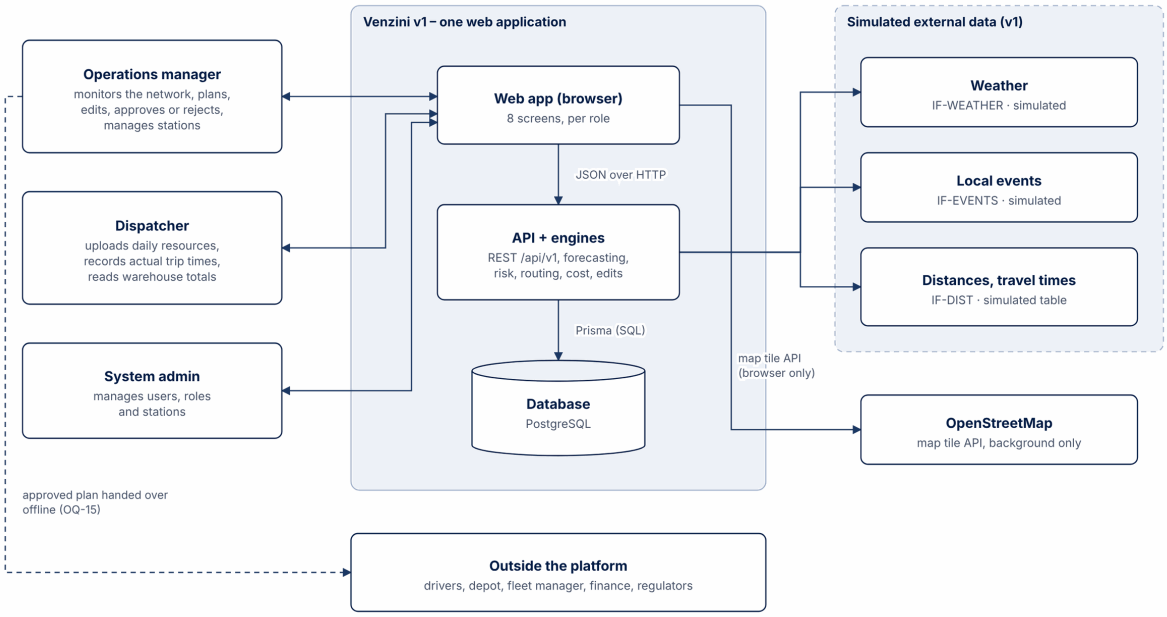


Figure 1 · System context

Element	Kind	What crosses the boundary	SRS refs
Operations manager	Role (user)	Network, forecasts, plans, edits, decisions, station records	Section 4.3, D-15
Dispatcher	Role (user)	Daily resources, trip actual times; reads per-warehouse totals and history	D-05, D-09, D-10, D-17
System admin	Role (user)	Users, roles, stations; reads history	FR-05, FR-06
Weather source	External data, simulated	Daily temperature and condition per area	IF-WEATHER, NFR-15
Local events source	External data, simulated	Events with date, area, expected impact (%)	IF-EVENTS
Distance service	External data, simulated	km and minutes between every pair of points	IF-DIST, AS-08

Element	Kind	What crosses the boundary	SRS refs
	table		
OpenStreetMap tile API	Map background	Outside API called by the browser only; it returns map images and no Venzini data is sent to it	Section 5.3 (SRS)
Drivers, depot, fleet, finance, regulators	Outside	Nothing in v1; the approved plan is handed over offline	BR-15

Not at the boundary: no LLM, no AI service, no public API, no price or equipment interface (D-11, NFR-17).

5.2 Composition view

Venzini is built as three containers – web, api and db – and the api is a modular monolith with one NestJS module per SRS component. Figure 2 shows the whole architecture on one page and Figure 3 how data moves through it; Figure 4 shows the decided stack; Figures 5 to 7 zoom into the containers, the backend modules and the frontend.

Figure 2 below shows the whole architecture, read top to bottom: the three roles use the React app in the browser; nginx serves it and forwards API calls; inside the api container a request passes the pipeline, reaches a platform or planning module, which uses the engines and DataModule, then CoreModule writes to PostgreSQL. Dashed boxes are simulated outside data and the MVP-only mock.

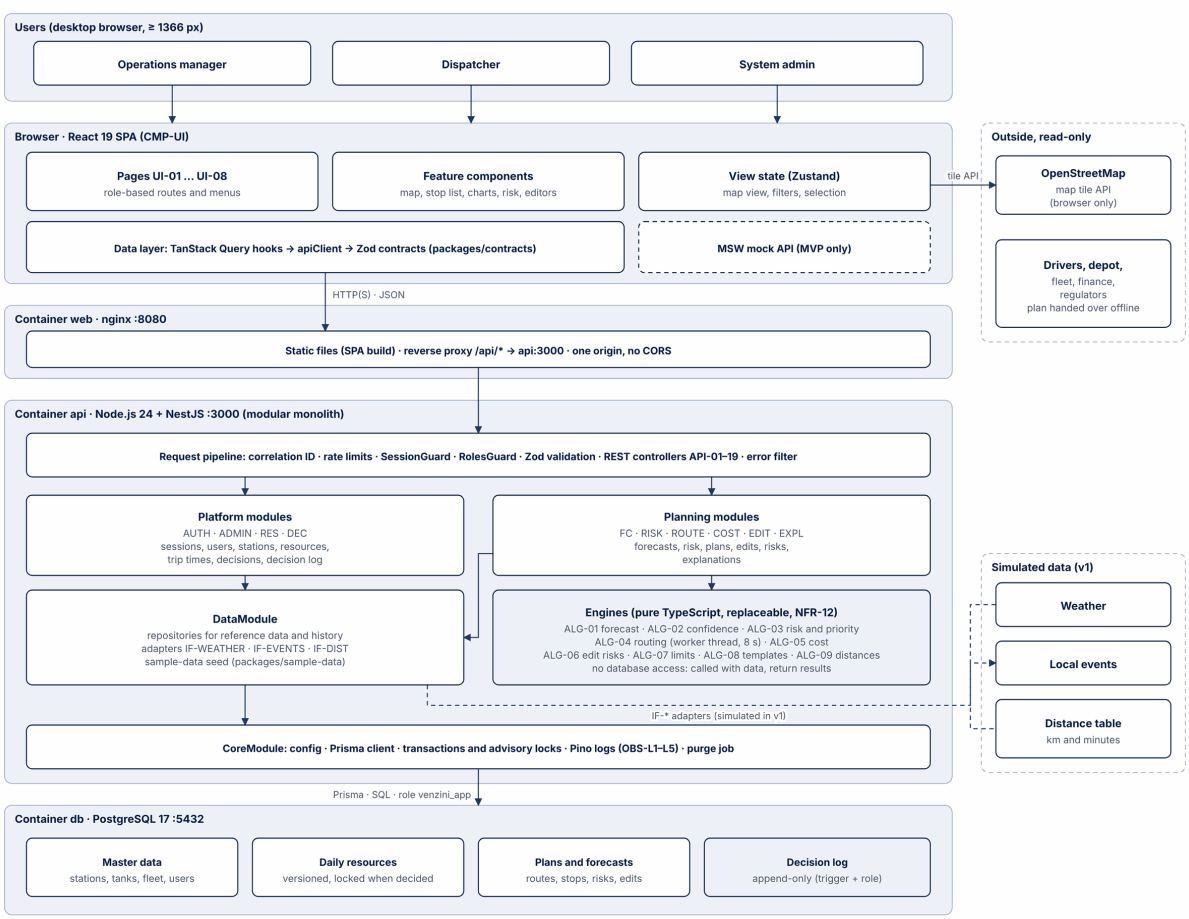


Figure 2 · System architecture

Figure 2 shows how Venzini is built; Figure 3 below shows how data moves through it. It is a level-1 data flow diagram: the six main processes, the data stores they read and write, and the outside parties that send or receive data. The admin maintains users and the station network. The dispatcher records each day's resources. Sales history, weather and events feed the forecast. Forecasts and tank levels feed the risk assessment. Risks, resources and distances feed the delivery plan. The manager reviews, edits and decides, every decision is logged, and the approved plan is handed to the drivers. Login and role checks apply to every user and are shown in Figure 16.

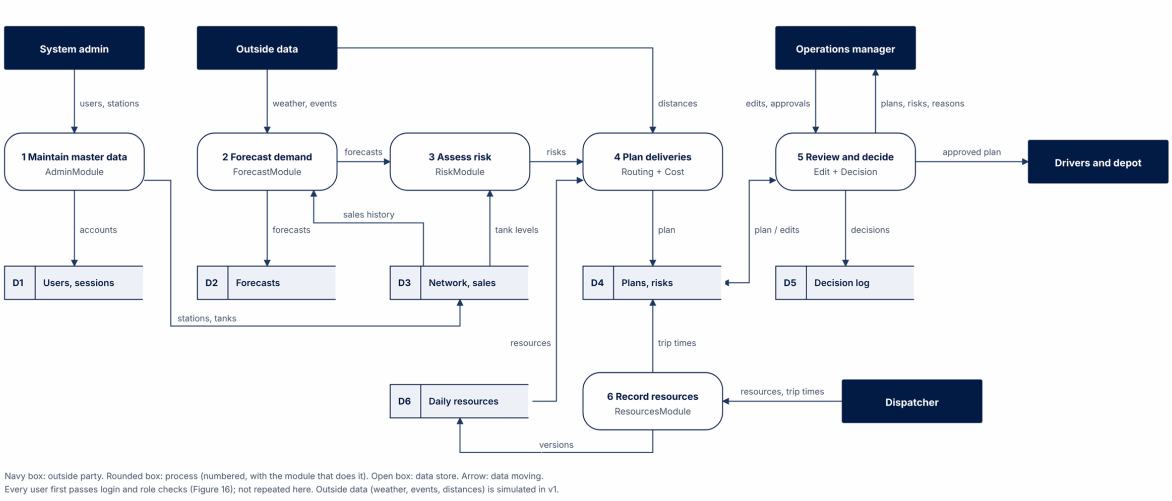


Figure 3 · Data flow diagram, level 1

Engines never read or write the database: modules load the data, call the engine and save its result. This keeps the engines pure functions that can be tested alone and replaced (NFR-12).

Figure 4 below shows the decided technology stack, grouped by layer. Solid boxes are the decided tools; dashed boxes are supporting libraries.

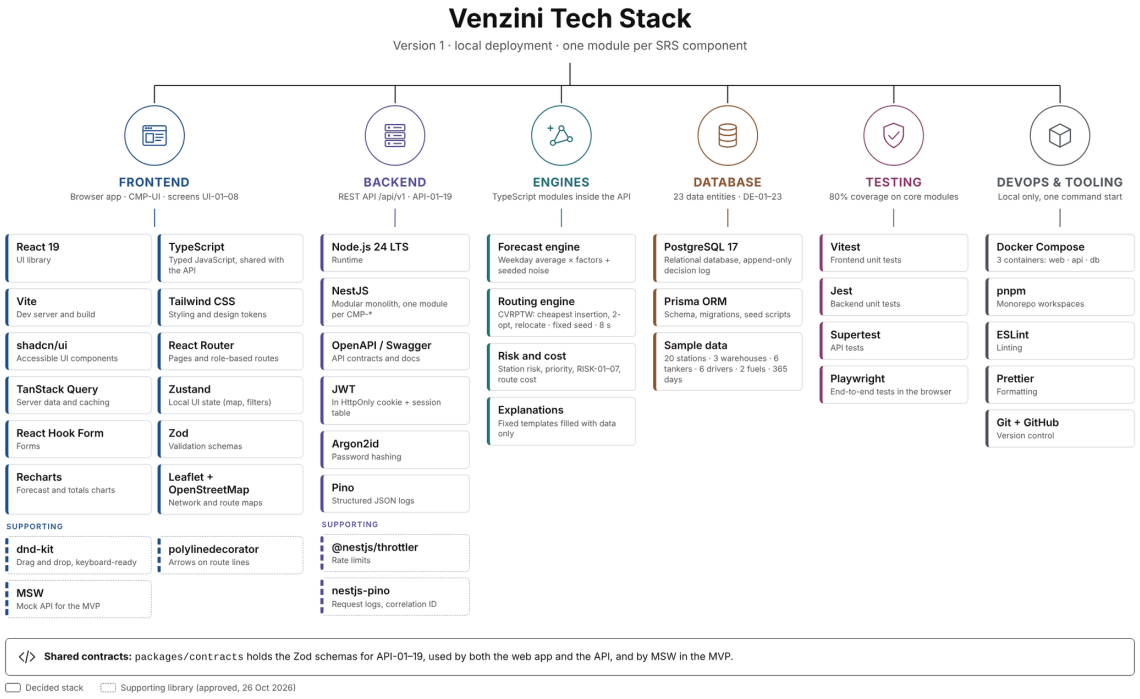


Figure 4 · Venzini tech stack

5.2.1 Containers

Figure 5 below shows the three containers. The browser talks only to nginx on port 8080; nginx serves the single-page application (SPA) and forwards /api/ to the api container, so the web app and the API share one origin and no Cross-Origin Resource Sharing (CORS) setup is needed.

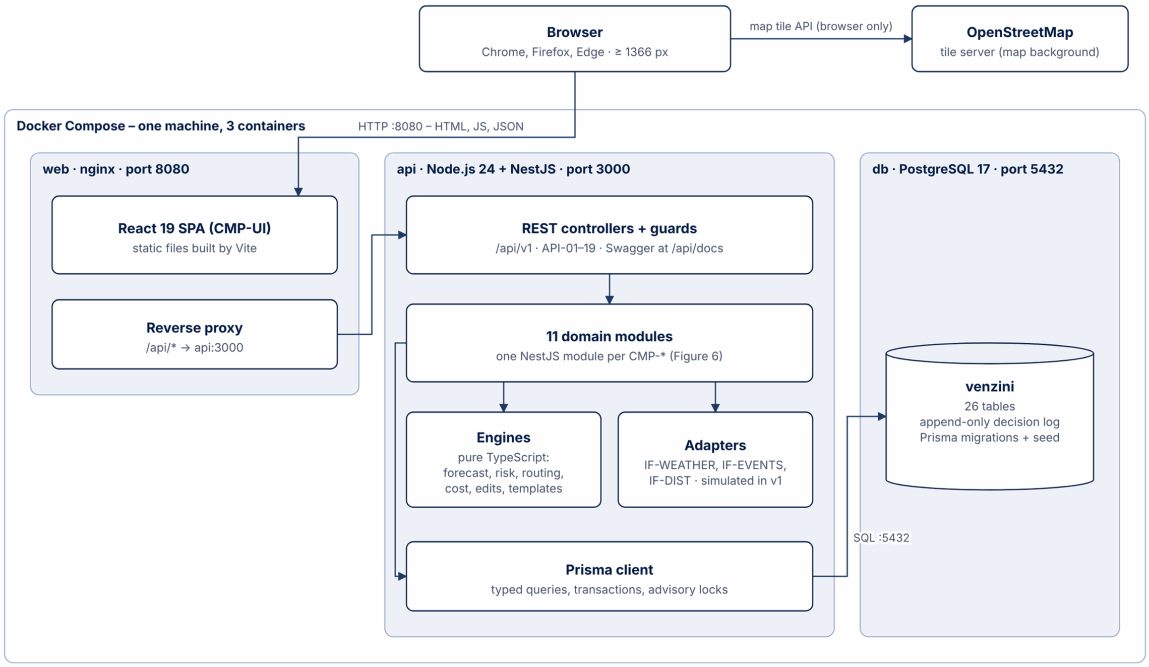


Figure 5 · Container diagram

Container	Image	Responsibility	Exposed port
web	nginx:1.27-alpine with the Vite build	Serves the SPA; reverse proxy for /api	8080 on the host
api	node:24-alpine running the NestJS build	All business logic, engines, adapters, OpenAPI docs	3000, internal only
db	postgres:17-alpine	All data; decision log protected at database level	5432, internal only (host 5433 in dev profile)

5.2.2 Backend modules

Each SRS component becomes exactly one NestJS module (Figure 6). Engines are plain TypeScript functions and classes inside their module, with no NestJS or database import, so they can be unit-tested alone and replaced behind an interface (NFR-12).

Figure 6 below shows the backend modules grouped by role. The colored bar marks the group: platform (navy #021C45), planning (burgundy #7B2D43) and foundation (dark green #2F5D50).

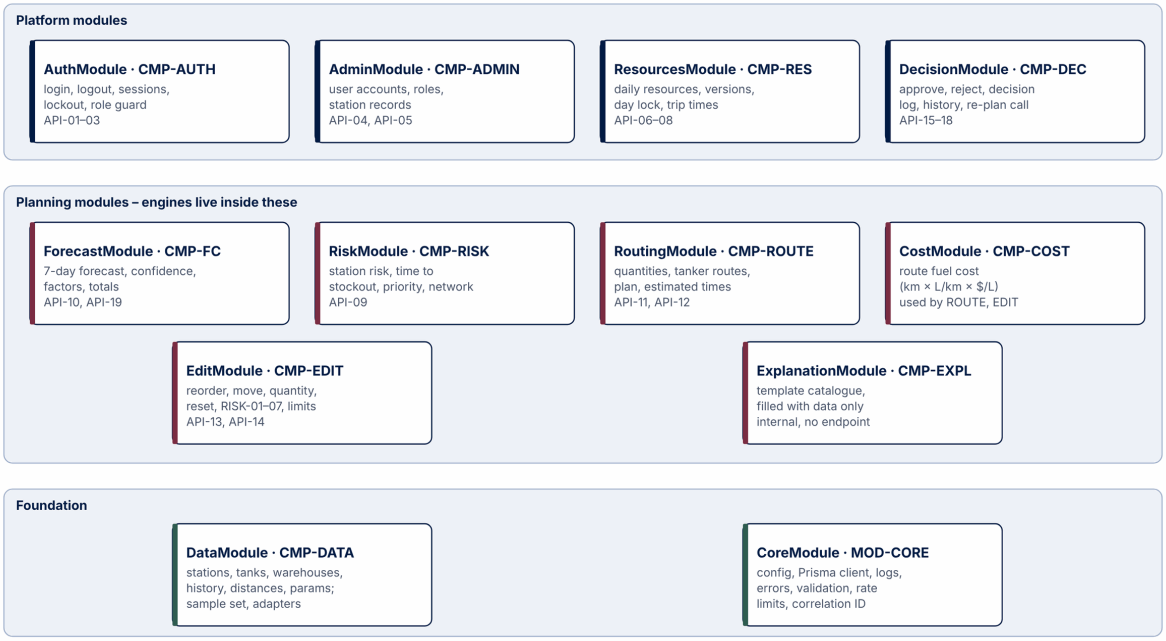


Figure 6 · Backend modules

Module	SRS component	Owns tables (Section 5.5)	Endpoints
MOD-AUTH AuthModule	CMP-AUTH	app_user, session, login_attempt	API-01, API-02, API-03
MOD-ADMIN AdminModule	CMP-ADMIN	none (writes through AUTH and DATA services)	API-04, API-05
MOD-RES ResourcesModule	CMP-RES	resource_version, tanker_availability, driver_availability, warehouse_stock; trip columns of route	API-06, API-07, API-08
MOD-FC ForecastModule	CMP-FC	forecast	API-10, API-19
MOD-RISK RiskModule	CMP-RISK	none (computes on read)	API-09
MOD-ROUTE RoutingModule	CMP-ROUTE	plan, route (plan columns), stop, stop_quantity	API-11, API-12
MOD-COST CostModule	CMP-COST	none	– (internal)
MOD-EDIT EditModule	CMP-EDIT	plan_edit, risk	API-13, API-14

Module	SRS component	Owns tables (Section 5.5)	Endpoints
MOD-EXPL ExplanationModule	CMP-EXPL	none (templates in code)	– (internal)
MOD-DEC DecisionModule	CMP-DEC	decision_log_entry	API-15 to API-18
MOD-DATA DataModule	CMP-DATA	station, tank, fuel_type, warehouse, tanker, tanker_compartment, driver, sales_history, external_factor, distance_entry, parameter	– (internal)
MOD-CORE CoreModule	–	none	Health check GET /api/v1/health

Rule: a module writes only the tables it owns. It reads other modules' data through their exported services, with one documented exception: MOD-RES reads route rows directly to find open trips (WARN-RES-01), to avoid a dependency cycle (Section 5.4).

5.2.3 Frontend structure

Figure 7 below shows the frontend structure: one page per screen (UI-01 to UI-08), shared feature components, and a data layer that talks to MSW in the MVP or to the real API in v1, with no change to the pages.

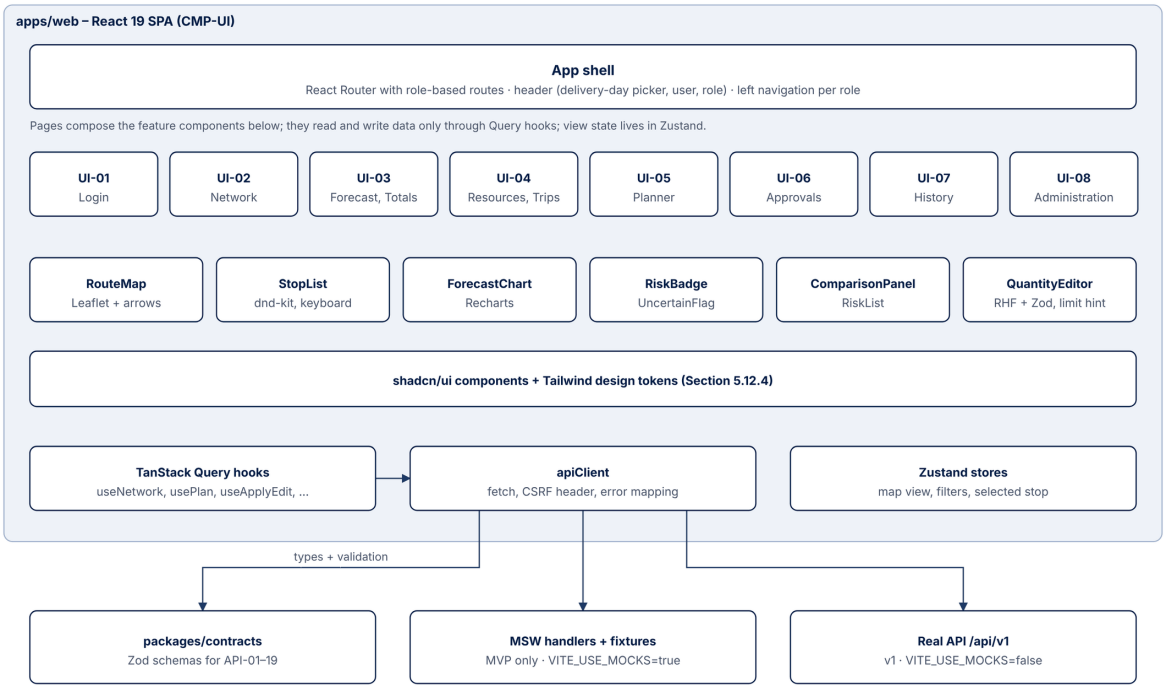


Figure 7 · Frontend structure

- **Server data** (network, forecast, plan, decisions) lives only in TanStack Query's cache; the browser never computes a business value (SRS Section 9.1).
- **View state** (map zoom, tanker filter, selected stop, open panels) lives in small Zustand stores.
- **Forms** use React Hook Form with the same Zod schemas the API uses, so browser and server reject the same values.

5.3 Logical view

The logical view lists the main classes of each module, with the attributes and methods that matter for the design. Engine interfaces are the replaceable seams required by NFR-12; their v1 implementations are simulated or heuristic.

Figure 8 below shows the classes of the planning modules (FC, RISK, ROUTE, COST, EDIT, EXPL). «interface» marks a replaceable engine; a dashed arrow with an open triangle means "implements".

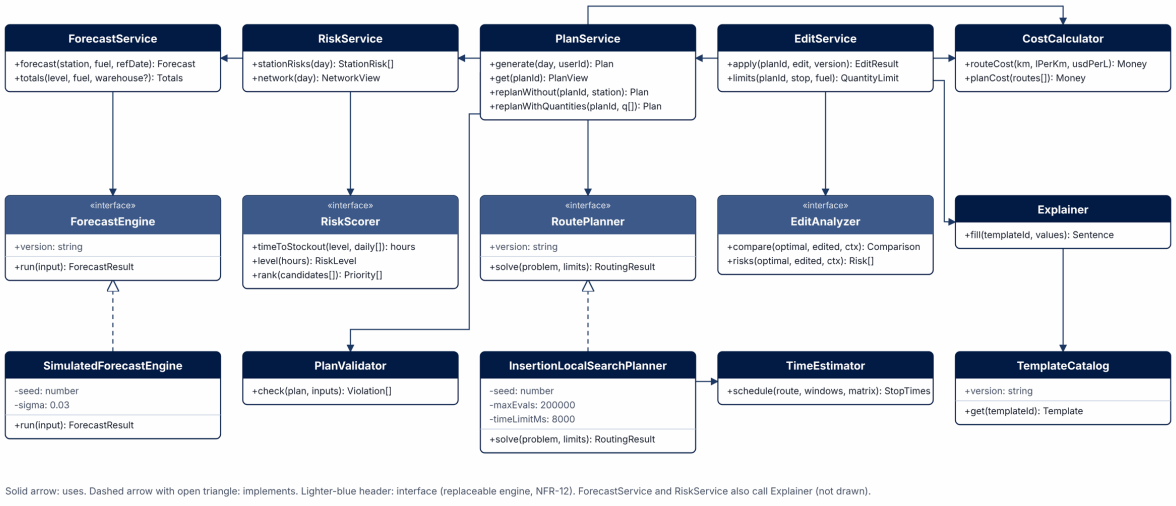


Figure 8 · Class diagram, planning modules

Figure 9 below shows the classes of the platform and foundation modules (AUTH, ADMIN, RES, DEC, DATA), including the three provider interfaces IF-WEATHER, IF-EVENTS and IF-DIST.

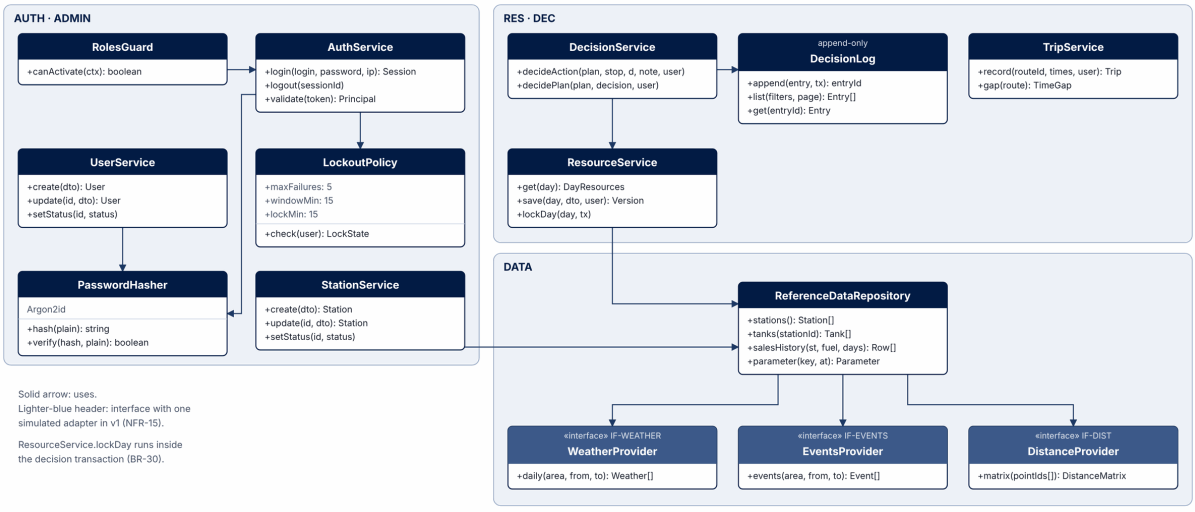


Figure 9 · Class diagram, platform and foundation modules

Domain model. Figures 8 and 9 show the services; Figure 10 shows the domain classes they create and change, with their main attributes, key methods and relationships. It is the object view of the same data as the ERDs in Section 5.5.

Figure 10 below shows the domain model. A filled diamond is a composition: a Plan owns its Routes, Risks and PlanEdits; a Route owns its Stops; a Stop owns its quantities; a Tanker owns its Compartments; a Station owns its Tanks. Plain lines are associations, with how many objects on each side.

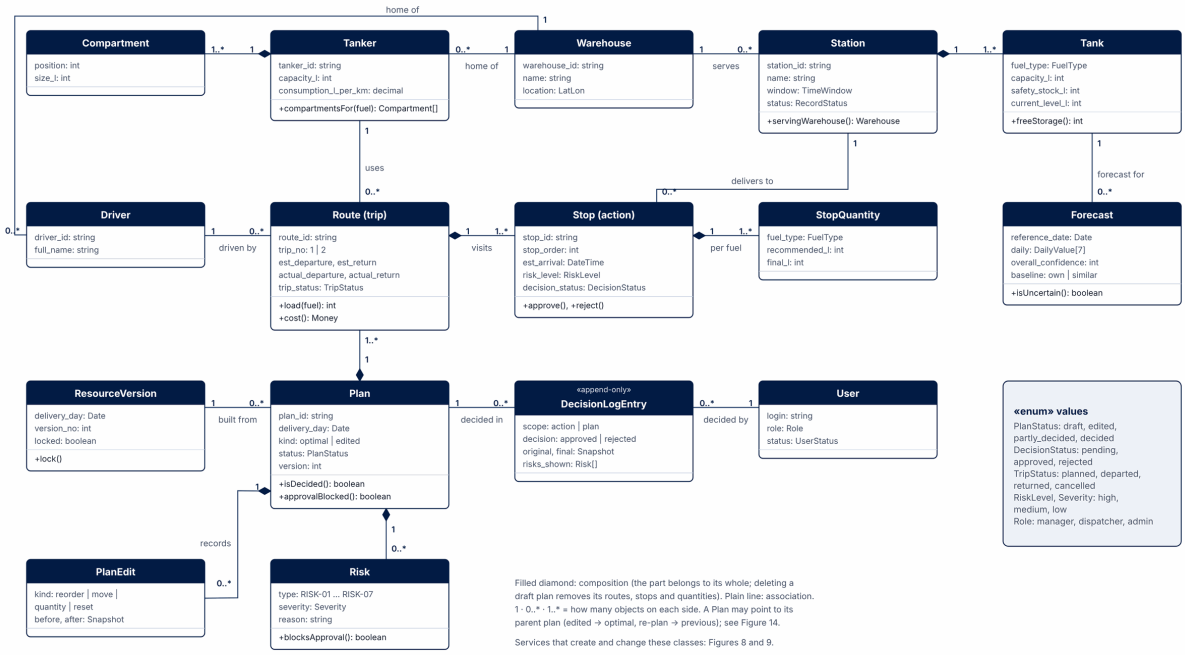


Figure 10 · Domain model

- **Plan** is the aggregate root of planning: edits, approvals and re-plans always load and save a Plan with its Routes, Stops, quantities and Risks together, in one transaction.
- **DecisionLogEntry** only refers to a Plan by ID and stores snapshots, so it never changes when the plan is re-planned (BR-13).
- **Forecast** belongs to a Tank (a station and fuel pair); a Stop reads it to set its risk level and uncertain flag.

5.3.1 Main classes per module

Module	Class	Kind	Key attributes	Key methods	SRS refs
AUTH	AuthService	Service	–	login(), logout(), validate()	FR-01, NFR-21, NFR-22
AUTH	PasswordHasher	Service	Argon2id: memory 19 MiB, 2 iterations,	hash(), verify()	NFR-06

Module	Class	Kind	Key attributes	Key methods	SRS refs
			parallelism 1		
AUTH	LockoutPolicy	Policy	maxFailures 5, windowMin 15, lockMin 15	check()	NFR-21
AUTH	SessionGuard, RolesGuard	NestJS guards	–	canActivate()	FR-02, NFR-05
ADMIN	UserService, StationService	Services	–	create(), update(), setStatus()	FR-05, FR-06
RES	ResourceService	Service	–	get(day), save(day, dto), lockDay(day, tx)	FR-07–13
RES	TripService	Service	–	record(routeId, times), gap()	FR-79, FR-84
FC	ForecastService	Service	–	forecast(), totals()	FR-17–23, FR-83, FR-85
FC	ForecastEngine	Interface	version	run(ForecastInput): ForecastResult	AIC-01, NFR-12
FC	SimulatedForecastEngine	Engine (v1)	seed, sigma 0.03	run()	AIC-01
RISK	RiskService	Service	–	stationRisks(), network()	FR-14–16, FR-24
RISK	RiskScorer	Interface	thresholds 24 h, 72 h	timeToStockout(), level(), rank()	BR-26, FR-25
ROUTE	PlanService	Service	–	generate(), get(), replanWithout(), replanWithQuantities()	FR-28–39, FR-59, FR-82
ROUTE	RoutePlanner	Interface	version	solve(RoutingProblem, limits)	AIC-03, NFR-12
ROUTE	InsertionLocalSearchPlanner	Engine (v1)	seed, timeLimitMs 8000	solve()	AIC-03, NFR-19
ROUTE	TimeEstimator	Engine helper	unloading parameters	schedule()	FR-78

Module	Class	Kind	Key attributes	Key methods	SRS refs
ROUTE	PlanValidator	Safety check	–	check(): Violation[]	FR-29, FR-31, FR-33, ERR-SYS-02
COST	CostCalculator	Engine	3 decimals stored	routeCost(), planCost()	FR-40, FR-42
EDIT	EditService	Service	–	apply(), limits()	FR-55–61, FR-75–77
EDIT	EditAnalyzer	Interface	severity thresholds (D-18)	compare(), risks()	FR-61–66, RISK-01–07
EXPL	Explainer, TemplateCatalog	Engine	catalog version	fill(templateId, values)	AIC-04, D-11
DEC	DecisionService	Service	–	decideAction(), decidePlan()	FR-69–72, FR-80, FR-82
DEC	DecisionLog	Repository	append-only	append(), list(), get()	FR-72, FR-73, NFR-14
DATA	ReferenceDataRepository	Repository	–	stations(), tanks(), salesHistory(), parameter()	FR-74
DATA	WeatherProvider, EventsProvider, DistanceProvider	Interfaces	source = simulated	daily(), events(), matrix()	IF-WEATHER, IF-EVENTS, IF-DIST

5.3.2 Shared domain types (packages/contracts)

- **Litres** are integers; **money** is a decimal string with 3 decimals in the database and a number with 2 decimals in API responses (D-20, NFR-31).
- **Date-times** are ISO 8601 in UTC; a **delivery day** is a local date (YYYY-MM-DD) in Africa/Tunis (Section 6.5).
- **IDs** keep the SRS formats: ST014, WH-RIV, T-01, D-04, P95, DIESEL; generated IDs use a prefix plus a sequence (PL-2211, R-02, S-07, RV-0192).

5.4 Dependency view

Module dependencies form a directed graph with no cycles (Figure 11). A module may only call modules below it; the build fails if a cycle appears.

Figure 11 below shows the module dependency graph. An arrow means "imports and calls". DataModule and CoreModule are used by many modules, so their arrows are listed in the table instead.

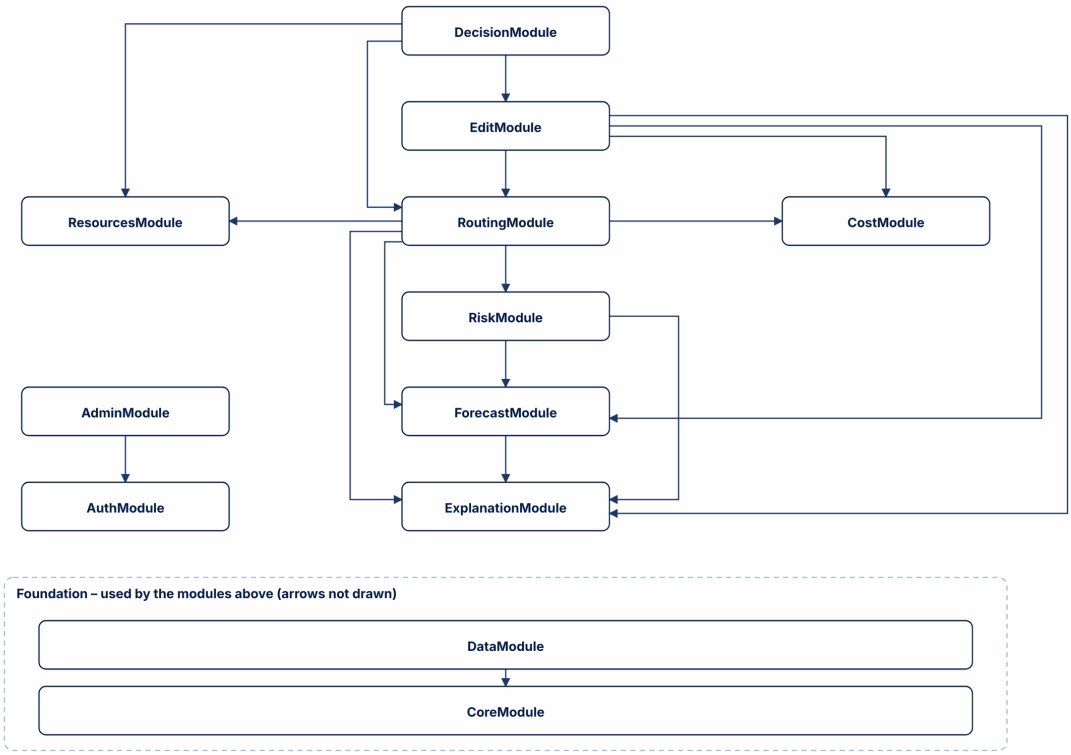


Figure 11 · Module dependency graph

Module	Depends on	Why
DecisionModule	Edit, Routing, Resources, Core	RISK-01 check before approval; re-plan after rejection (FR-82); lock the day's resources (BR-30)
EditModule	Routing, Cost, Forecast, Explanation, Data, Core	Recalculate routes; re-plan after a quantity edit; stockout-before-arrival needs forecasts; risk reasons
RoutingModule	Resources, Risk, Forecast, Cost, Explanation, Data, Core	Latest resources (FR-12); priority; quantities; costs; reasons
RiskModule	Forecast, Explanation, Data, Core	Time to stockout from forecasts; urgency reasons

Module	Depends on	Why
ForecastModule	Explanation, Data, Core	History and external factors; factor sentences
CostModule	Data, Core	Fuel price and consumption parameters
ResourcesModule	Data, Core	Tanker and driver master data
AdminModule	Auth, Data, Core	Password hashing for new users; station records
AuthModule	Core	–
ExplanationModule	Core	–
DataModule	Core	–

- **Enforcement:** eslint-plugin-boundaries with the allowed edges above, and madge --circular in CI. A new edge needs an update of this table.
- **Cycle avoided:** trip times (FR-79) belong to ResourcesModule but live on route rows owned by RoutingModule. Resources writes only the four trip columns and reads routes with a plain query, so Routing → Resources stays the only module edge between them.

5.5 Information view

The database has 26 tables: one or more for each of the 23 SRS data entities (DE-01 to DE-23), plus two tables for sessions and login attempts. Figures 12 to 14 show the relationships; Section 5.5.3 is the full data dictionary.

5.5.1 Conventions

- **Names:** snake_case tables and columns; `app_user` instead of `user`, which is a reserved word in PostgreSQL.
- **Keys:** business entities keep readable text keys from the SRS (ST014, WH-RIV, T-01, P95); users, sessions and log entries use UUIDs. Generated plan IDs use a prefix and a database sequence (PL-2211, R-02, S-07).
- **Types:** litres `integer`; money `numeric(12,3)` in USD (D-20); km `numeric(9,2)`; coordinates `numeric(9,6)`; all date-times `timestamptz` stored in UTC; delivery days `date` in Africa/Tunis.
- **Enums:** PostgreSQL enum types generated by Prisma: user_role, user_status, record_status (active, inactive), data_source (simulated, real), plan_kind, plan_status, trip_status, risk_level, severity, decision_status, decision, decision_scope, edit_kind, factor_kind, forecast_status.
- **Audit columns:** every table has `created_at`; mutable tables also have `updated_at`. Reference data has a `source` column, so simulated rows can never be mistaken for real ones (SRS 8.7).
- **Deletes:** master data is never deleted, only set to `inactive` (SRS 8.5). Foreign keys use ON DELETE RESTRICT, except the plan → route → stop → stop_quantity chain, which cascades only for the draft-plan purge (Section 5.5.5).

5.5.2 Entity–relationship diagrams

Figure 12 below shows the first part: users and sessions, stations, tanks, warehouses, sales history and forecasts. Three tables have no foreign keys and are not drawn: external_factor, distance_entry and parameter.

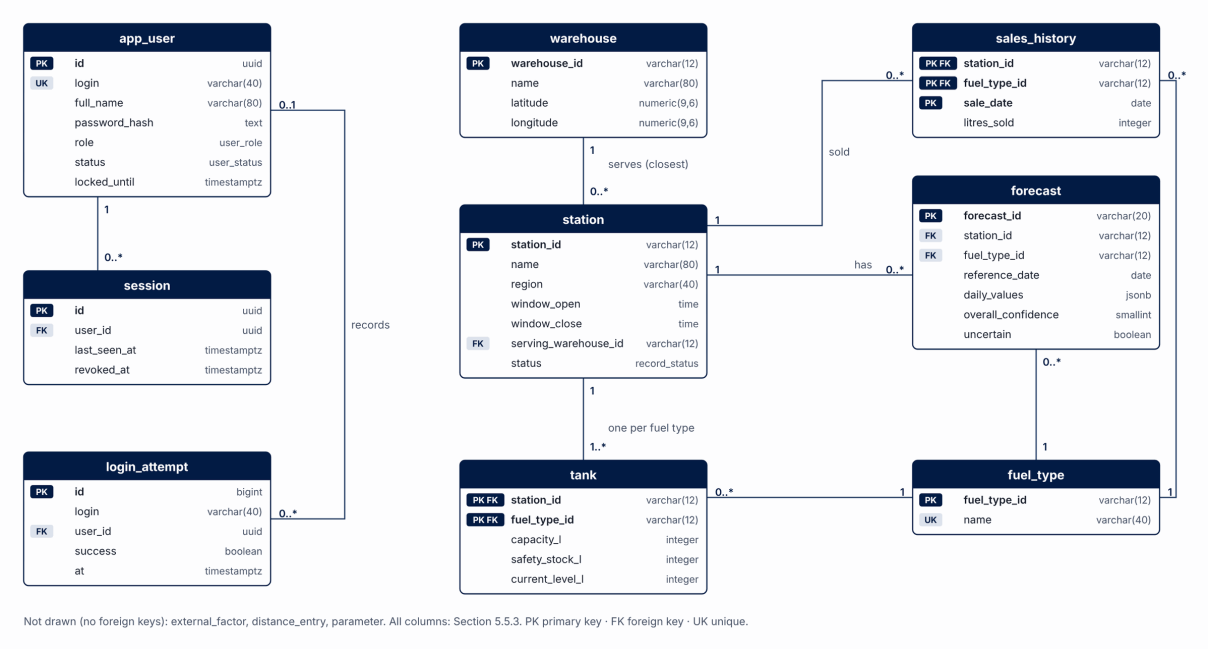


Figure 12 · ERD part 1: users, stations and forecasts

Figure 13 below shows the second part: tankers with their compartments, drivers, and the versioned daily resources uploaded by the dispatcher.

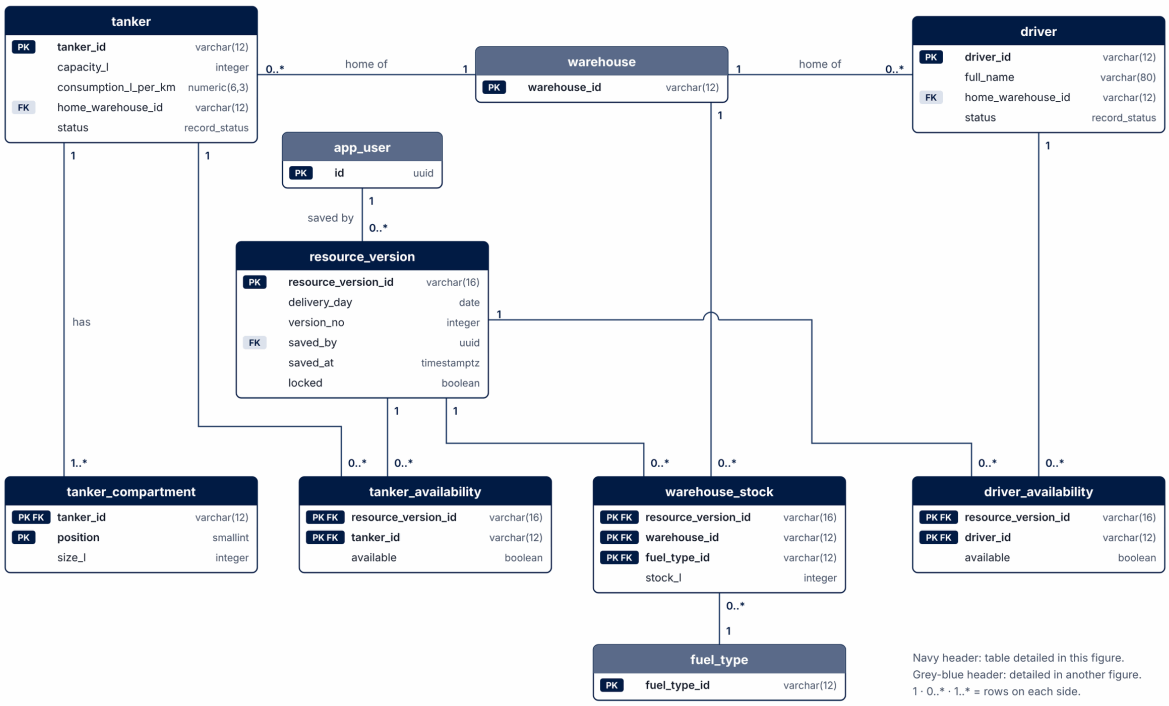


Figure 13 · ERD part 2: tankers, drivers and daily resources

Figure 14 below shows the third part: plans, routes (trips), stops, quantities, risks, edits and the decision log. A plan may point to its parent plan: an edited plan to its optimal plan, a re-plan to the plan it replaced. A route carries its trip number (1 or 2) and, when the dispatcher cancels it, the reason.

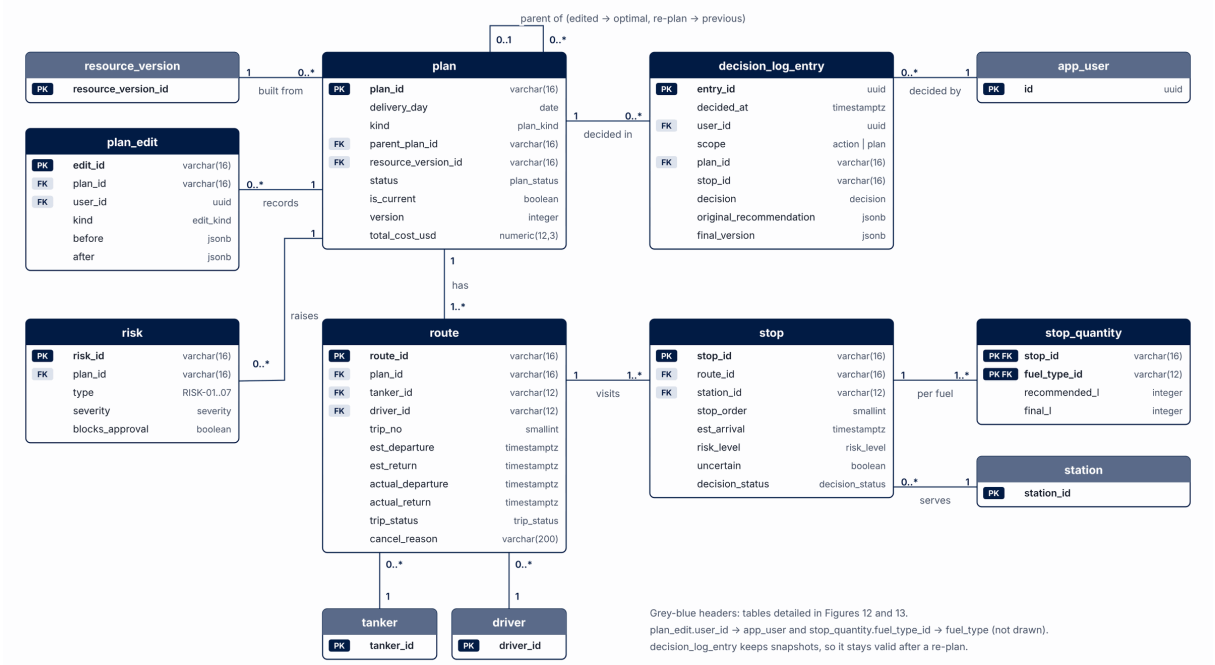


Figure 14 · ERD part 3: plans, routes and decisions

Figures 12 to 14 show each part of the database in detail. Figure 15 below puts them together: all 23 tables and every link between them, without columns. It shows how the three parts connect end to end. The station network and the users feed the daily resources, the resources and forecasts feed the plan, and every decision on the plan ends up in the decision log, linked back to the user who made it. Links that the three parts only show as grey stub boxes, such as stop → station, route → tanker and driver, and plan → resource_version, appear here in full.

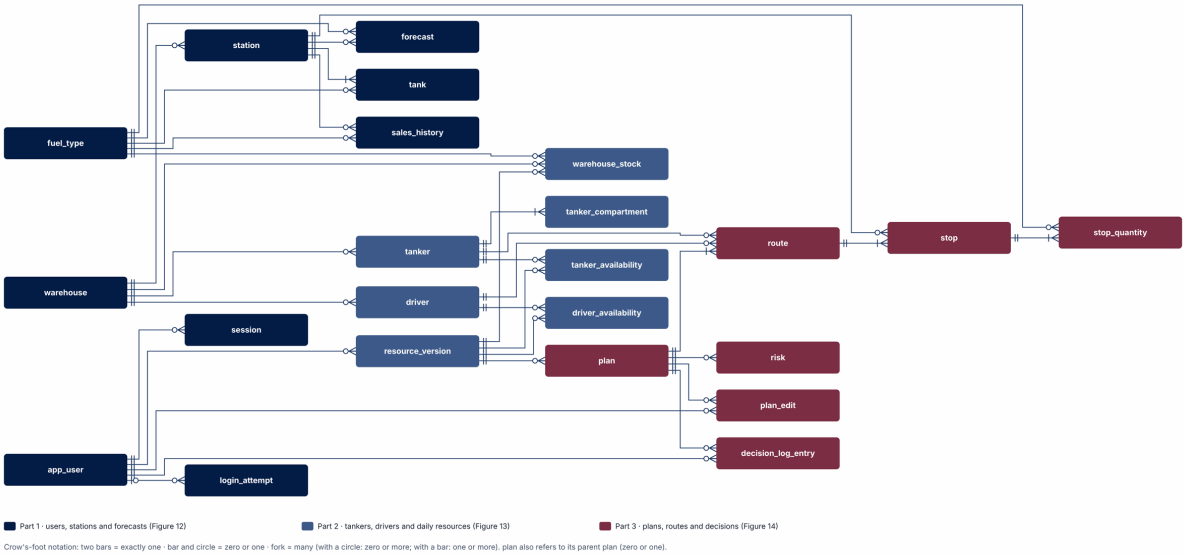


Figure 15 · Full entity-relationship diagram, table names only

SRS entity → table

SRS entity	Table(s)	Notes
DE-01 User	TBL-01 app_user	+ locked_until (NFR-21)
DE-02 Station	TBL-06 station	serving_warehouse_id is derived and stored
DE-03 Tank	TBL-07 tank	–
DE-04 Fuel type	TBL-04 fuel_type	–
DE-05 Warehouse	TBL-05 warehouse	+ status, source
DE-06 Warehouse stock	TBL-14 warehouse_stock	Day comes from its resource version
DE-07 Tanker	TBL-08 tanker, TBL-09 tanker_compartment	Compartments get their own table (D-19)
DE-08 Tanker availability	TBL-12 tanker_availability	Day comes from its resource version
DE-09 Driver	TBL-10 driver	No contact data (NFR-07)
DE-10 Driver availability	TBL-13 driver_availability	–
DE-11 Resource version	TBL-11 resource_version	–
DE-12 Sales history	TBL-15 sales_history	–
DE-13 External factor	TBL-16 external_factor	–
DE-14 Forecast	TBL-17 forecast	Daily values and factors as validated JSON
DE-15 Plan	TBL-18 plan	+ is_current, version
DE-16 Route (trip)	TBL-19 route	–
DE-17 Decision log entry	TBL-24 decision_log_entry	Append-only at database level
DE-18 Stop	TBL-20 stop	+ plan_id copy for the one-stop-per-station rule
DE-19 Stop quantity	TBL-21 stop_quantity	–
DE-20 Risk	TBL-22 risk	–
DE-21 Distance entry	TBL-25 distance_entry	–
DE-22 Parameter	TBL-26 parameter	Keeps history by valid_from

SRS entity	Table(s)	Notes
DE-23 Plan edit	TBL-23 plan_edit	Append-only within a plan
–	TBL-02 session, TBL-03 login_attempt	NFR-21, NFR-22

5.5.3 Data dictionary

NN = not null. Every table also has `created_at timestamptz NN default now()`; tables marked (mutable) have `updated_at`. Owner = the only module allowed to write the table (Section 5.2.2).

TBL-01 app_user – DE-01 · owner AUTH · (mutable)

Column	Type	Constraints	Description
id	uuid	PK, default gen_random_uuid()	User ID
full_name	varchar(80)	NN	Shown in the header and the decision log
login	varchar(40)	NN, unique on lower(login), CHECK length 3–40	Login name
password_hash	text	NN	Argon2id hash with salt; never returned by any API
role	user_role	NN (manager, dispatcher, admin)	Exactly one role (FR-02)
status	user_status	NN, default active	Inactive users cannot log in (FR-05)
locked_until	timestamptz	null	Lockout end time (NFR-21)

TBL-02 session · owner AUTH · (mutable)

Column	Type	Constraints	Description
id	uuid	PK	Session ID; carried as `sid` inside the JWT
user_id	uuid	NN, FK → app_user	Owner of the session
last_seen_at	timestamptz	NN	Updated at most once a minute; idle limit 30 min (NFR-22)
revoked_at	timestamptz	null	Set on logout or deactivation; ends the session at once
ip	inet	null	Client address, for the security log

Column	Type	Constraints	Description
user_agent	varchar(200)	null	Browser, for the security log

Indexes: (user_id), (last_seen_at).

TBL-03 login_attempt · owner AUTH · append-only

Column	Type	Constraints	Description
id	bigint	PK, identity	–
login	varchar(40)	NN	Login typed, even if it does not exist
user_id	uuid	null, FK → app_user	Set when the login exists
success	boolean	NN	–
ip	inet	null	Client address
at	timestamptz	NN	Attempt time

Index: (login, at desc), used to count failures in the last 15 minutes.

TBL-04 fuel_type – DE-04 · owner DATA

Column	Type	Constraints	Description
fuel_type_id	varchar(12)	PK	P95, DIESEL
name	varchar(40)	NN, unique	Petrol 95, Diesel
display_order	smallint	NN	Order in selectors and tables

TBL-05 warehouse – DE-05 · owner DATA · (mutable)

Column	Type	Constraints	Description
warehouse_id	varchar(12)	PK	e.g. WH-RIV
name	varchar(80)	NN	–
latitude	numeric(9,6)	NN, CHECK −90..90	–
longitude	numeric(9,6)	NN, CHECK −180..180	–
status	record_status	NN, default active	–
source	data_source	NN	simulated in v1

TBL-06 station – DE-02 · owner DATA (written by ADMIN through StationService) · (mutable)

Column	Type	Constraints	Description
station_id	varchar(12)	PK	e.g. ST014
name	varchar(80)	NN	–
latitude, longitude	numeric(9,6)	NN, same checks as warehouse	Map position
region	varchar(40)	NN	Area used to join weather and events
window_open, window_close	time	NN, CHECK window_open < window_close	Daily delivery window, local time
serving_warehouse_id	varchar(12)	null, FK → warehouse	Closest warehouse (BR-17), recomputed on every station or distance change
status	record_status	NN, default active	Inactive stations are left out of new plans
source	data_source	NN	–

Indexes: (status), (serving_warehouse_id).

TBL-07 tank – DE-03 · owner DATA · (mutable)

Column	Type	Constraints	Description
station_id	varchar(12)	PK, FK → station	–
fuel_type_id	varchar(12)	PK, FK → fuel_type	One tank per station and fuel type
capacity_l	integer	NN, CHECK > 0	–
safety_stock_l	integer	NN, CHECK 0 ≤ safety_stock_l < capacity_l	–
current_level_l	integer	NN, CHECK 0 ≤ current_level_l ≤ capacity_l	–
level_updated_at	timestamptz	NN	When the level was last read (sample data in v1)

TBL-08 tanker – DE-07 · owner DATA · (mutable)

Column	Type	Constraints	Description
tanker_id	varchar(12)	PK	e.g. T-01
capacity_l	integer	NN, CHECK > 0	Must equal the sum of its compartments; checked by a deferred constraint trigger
consumption_l_per_km	numeric(6,3)	NN, CHECK > 0	Tanker fuel use, one fixed value per tanker (sample value)
home_warehouse_id	varchar(12)	NN, FK → warehouse	One home warehouse (BR-18)
status	record_status	NN, default active	–
source	data_source	NN	–

TBL-09 tanker_compartment – DE-07 (D-19) · owner DATA

Column	Type	Constraints	Description
tanker_id	varchar(12)	PK, FK → tanker	–
position	smallint	PK, CHECK ≥ 1	Compartment number, front to back
size_l	integer	NN, CHECK > 0	Maximum litres (BR-31)

TBL-10 driver – DE-09 · owner DATA · (mutable)

Column	Type	Constraints	Description
driver_id	varchar(12)	PK	e.g. D-04
full_name	varchar(80)	NN	Only personal data kept for drivers (NFR-07, D-22)
home_warehouse_id	varchar(12)	NN, FK → warehouse	Used to pair drivers with tankers (FR-35)
status	record_status	NN, default active	–

TBL-11 resource_version – DE-11 · owner RES

Column	Type	Constraints	Description
resource_version_id	varchar(16)	PK	e.g. RV-0192
delivery_day	date	NN	Local delivery day
version_no	integer	NN; unique (delivery_day, version_no)	1, 2, 3 … per day; latest = current (FR-12)
saved_by	uuid	NN, FK → app_user	Dispatcher who saved (FR-13)
saved_at	timestamptz	NN	–
locked	boolean	NN, default false	Set to true, on the latest version, when the day's plan is decided (BR-30)

A new version is a full copy of the day's tanker, driver and stock rows, so a plan always points to an unchanging set of resources (AI-18). Index: (delivery_day, version_no desc).

TBL-12 tanker_availability – DE-08 · owner RES

Column	Type	Constraints	Description
resource_version_id	varchar(16)	PK, FK → resource_version	–
tanker_id	varchar(12)	PK, FK → tanker	–
available	boolean	NN	Available on that delivery day

TBL-13 driver_availability – DE-10 · owner RES

Column	Type	Constraints	Description
resource_version_id	varchar(16)	PK, FK → resource_version	–
driver_id	varchar(12)	PK, FK → driver	–
available	boolean	NN	–

TBL-14 warehouse_stock – DE-06 · owner RES

Column	Type	Constraints	Description
resource_version_id	varchar(16)	PK, FK → resource_version	–
warehouse_id	varchar(12)	PK, FK → warehouse	–
fuel_type_id	varchar(12)	PK, FK → fuel_type	–
stock_l	integer	NN, CHECK ≥ 0	Caps deliveries from that warehouse (BR-08)

TBL-15 sales_history – DE-12 · owner DATA

Column	Type	Constraints	Description
station_id, fuel_type_id	varchar(12)	PK, FK → tank	–
sale_date	date	PK	One row per day; missing days allowed and counted (SRS 8.7)
litres_sold	integer	NN, CHECK ≥ 0	–
source	data_source	NN	–

Size: 20 stations × 2 fuels × 365 days = 14,600 rows in the sample set.

TBL-16 external_factor – DE-13 · owner DATA

Column	Type	Constraints	Description
factor_date	date	PK	–
area	varchar(40)	PK	Same values as station.region
kind	factor_kind	PK (weather, event, calendar)	–
value	jsonb	NN, checked by Zod on write	weather {temp_c, condition}; event {name, impact_pct}; calendar {holiday, name}
source	data_source	NN	simulated in v1

TBL-17 forecast – DE-14 · owner FC

Column	Type	Constraints	Description
forecast_id	varchar(20)	PK	e.g. FC-ST014-P95-20261103
station_id, fuel_type_id	varchar(12)	NN, FK → tank	–
reference_date	date	NN	Day 1 = reference_date + 1 (FR-18)
model_version	varchar(20)	NN	e.g. fc-sim-1.0.0 (AI-18)
input_version	varchar(40)	NN	Hash of the inputs used
daily_values	jsonb	NN; exactly 7 items	{date, litres ≥ 0, confidence 0–100, band_low, band_high}
factors	jsonb	NN	{factor, impact_pct, explanation, available}
overall_confidence	smallint	NN, CHECK 0–100	Lowest daily confidence (FR-21)
uncertain	boolean	generated always as (overall_confidence < 70) stored	BR-27
baseline	forecast_baseline	NN (own, similar_stations)	FR-83
baseline_note	varchar(200)	null; NN when baseline = similar_stations	Reason shown with TPL-FC-09

Unique: (station_id, fuel_type_id, reference_date, model_version, input_version), so the same inputs reuse the same forecast row (NFR-13).

TBL-18 plan – DE-15 · owner ROUTE (status changes by DEC through PlanService) · (mutable until decided)

Column	Type	Constraints	Description
plan_id	varchar(16)	PK	e.g. PL-2211
delivery_day	date	NN	–
kind	plan_kind	NN (optimal, edited)	–
parent_plan_id	varchar(16)	null, FK → plan	Edited plan → its optimal plan; re-plan → the plan it replaced

Column	Type	Constraints	Description
resource_version_id	varchar(16)	NN, FK → resource_version	Resources used (FR-12, AI-18)
data_version	varchar(40)	NN	Hash of reference data and history used
model_versions	jsonb	NN	{forecast, risk, routing, cost, templates} versions
status	plan_status	NN (draft, edited, partly_decided, decided)	Section 5.8
is_current	boolean	NN	The one plan shown for the day
version	integer	NN, default 1	Optimistic lock; +1 on every edit
time_limited	boolean	NN, default false	True when the 8 s optimizer limit was reached (SRS 14.5)
unserved	jsonb	NN, default []	{station_id, fuel_type_id, risk_level, hours_to_stockout, reason} (FR-38)
total_km, total_minutes, total_litres	numeric(9,2), integer, integer	NN	Plan totals (FR-52)
total_cost_usd	numeric(12,3)	null	Null when cost is not available (ERR-DATA-03)
created_by	uuid	NN, FK → app_user	Manager who generated or edited

Indexes: partial unique (delivery_day) WHERE is_current (one current plan per day, DE-15); (delivery_day, created_at desc).

TBL-19 route – DE-16 · owner ROUTE (trip columns written by RES) · (mutable)

Column	Type	Constraints	Description
route_id	varchar(16)	PK	e.g. R-02
plan_id	varchar(16)	NN, FK → plan, ON DELETE CASCADE	–
tanker_id	varchar(12)	NN, FK → tanker; unique (plan_id, tanker_id, trip_no)	At most two trips per tanker per plan (BR-23)
trip_no	smallint	NN, CHECK trip_no	Second trip only when the first

Column	Type	Constraints	Description
		IN (1, 2), default 1	returns early enough (ALG-04)
driver_id	varchar(12)	NN, FK → driver	–
color	varchar(16)	NN	Route line color (FR-53)
compartment_plan	jsonb	NN	{position, fuel_type_id, litres} per compartment (BR-31)
est_departure, est_return	timestamptz	NN	FR-78
km, minutes, litres	numeric(9,2), integer, integer	NN	Includes the return leg (BR-25)
cost_usd	numeric(12,3)	null	FR-40
actual_departure, actual_return	timestamptz	null; CHECK actual_return > actual_departure	Written by the dispatcher (FR-79)
time_gap_min	integer	null	Actual minus estimated duration (FR-84)
trip_status	trip_status	NN, default planned	planned, departed, returned, cancelled
cancel_reason	varchar(200)	null; NN when trip_status = cancelled	Reason typed by the dispatcher when cancelling a planned trip (SM-03)
trip_updated_by, trip_updated_at	uuid, timestamptz	null	Who recorded times, and when

Index: (trip_status) WHERE trip_status = 'departed', for open-trip checks (WARN-RES-01).

TBL-20 stop – DE-18 · owner ROUTE (decision columns written by DEC) · (mutable until decided)

Column	Type	Constraints	Description
stop_id	varchar(16)	PK	e.g. S-07; one stop = one recommended action
route_id	varchar(16)	NN, FK → route, ON DELETE CASCADE	–
plan_id	varchar(16)	NN, FK → plan; unique (plan_id, station_id)	One stop per station per plan (BR-24)

Column	Type	Constraints	Description
station_id	varchar(12)	NN, FK → station	–
stop_order	smallint	NN, CHECK ≥ 1; unique (route_id, stop_order) deferrable	1..n without gaps (checked by the app)
est_arrival	timestamptz	NN	FR-78
priority_rank	smallint	NN	FR-25
risk_level	risk_level	NN	Station's highest fuel risk (FR-47)
hours_to_stockout	numeric(6,1)	null	Null when > 168 h
reason	text	NN	Template sentence (FR-27)
uncertain	boolean	NN	Any forecast used is uncertain (FR-23)
decision_status	decision_status	NN, default pending	pending, approved, rejected
decided_at	timestamptz	null	–

TBL-21 stop_quantity – DE-19 · owner ROUTE

Column	Type	Constraints	Description
stop_id	varchar(16)	PK, FK → stop, ON DELETE CASCADE	–
fuel_type_id	varchar(12)	PK, FK → fuel_type	–
recommended_l	integer	NN, CHECK ≥ 0 and recommended_l % 100 = 0	System value (FR-28)
final_l	integer	NN, CHECK ≥ 0	Value after manager edits; ≤ BR-28 maximum, checked by EditService

TBL-22 risk – DE-20 · owner EDIT

Column	Type	Constraints	Description
risk_id	varchar(16)	PK	–
plan_id	varchar(16)	NN, FK → plan, ON DELETE CASCADE	–
type	varchar(8)	NN, CHECK in RISK-01..RISK-	–

Column	Type	Constraints	Description
		07	
severity	severity	NN	low, medium, high (D-18)
measured_value	numeric(9,2)	null	Minutes late or % extra, used by the template
affected	jsonb	NN	{stops, segments, routes} IDs to highlight (FR-65)
reason	text	NN	Template sentence (FR-64)
blocks_approval	boolean	generated always as (type = 'RISK-01') stored	BR-21

TBL-23 plan_edit – DE-23 · owner EDIT · append-only

Column	Type	Constraints	Description
edit_id	varchar(16)	PK	–
plan_id	varchar(16)	NN, FK → plan	–
user_id	uuid	NN, FK → app_user	–
kind	edit_kind	NN (reorder, move, quantity, reset)	–
before, after	jsonb	NN	Changed stops only, as snapshots

TBL-24 decision_log_entry – DE-17 · owner DEC · append-only, protected in the database

Column	Type	Constraints	Description
entry_id	uuid	PK	–
decided_at	timestamptz	NN	–
user_id	uuid	NN, FK → app_user (RESTRICT)	Manager who decided
scope	decision_scope	NN (action, plan)	–
plan_id	varchar(16)	NN, FK → plan (RESTRICT)	–
stop_id	varchar(16)	null; NN when scope = action	Kept as text: the snapshot is the record
delivery_day,	date, varchar(12)	NN, null	Copies for fast filters (FR-73)

Column	Type	Constraints	Description
station_id			
decision	decision	NN (approved, rejected)	–
original_recommendation	jsonb	NN	Optimal plan or action as recommended
final_version	jsonb	NN	What was approved or rejected
was_edited	boolean	NN	Suggested plan or edited version (FR-72)
edits	jsonb	NN, default []	plan_edit snapshots behind the final version
risks_shown	jsonb	NN, default []	Risks visible at decision time
cost_optimal, cost_final	numeric(12,3)	null	–
cost_diff_pct	numeric(6,1)	null	–
note	varchar(500)	null	Optional rejection note, stored as plain text
input_versions	jsonb	NN	Resource, data and model versions (AI-18)
correlation_id	uuid	NN	Links to the technical logs

Indexes: (decided_at desc), (station_id, decided_at desc), (user_id, decided_at desc), (delivery_day).

TBL-25 distance_entry – DE-21 · owner DATA

Column	Type	Constraints	Description
from_point_id, to_point_id	varchar(12)	PK	Station or warehouse IDs
km	numeric(8,2)	NN, CHECK > 0	–
minutes	integer	NN, CHECK > 0	–
source	data_source	NN	simulated table in v1 (AS-08)

Size: 23 points × 22 = 506 rows; loaded into memory once per request as the distance matrix.

TBL-26 parameter – DE-22 · owner DATA

Column	Type	Constraints	Description

Column	Type	Constraints	Description
key	varchar(60)	PK	e.g. fuel_price_usd_per_l, unloading_base_min, unloading_min_per_1000l
valid_from	timestamptz	PK	History: the value in force is the latest valid_from ≤ now
value	numeric(12,4)	NN	–
unit	varchar(20)	NN	USD/L, min, min/1000 L
changed_by	uuid	null, FK → app_user	Null for seed values

5.5.4 Integrity rules enforced in the database

Rule	How	SRS refs
One current plan per day	Partial unique index on plan(delivery_day) WHERE is_current	DE-15
At most two trips per tanker per plan	Unique (plan_id, tanker_id, trip_no); CHECK trip_no IN (1, 2)	BR-23
One stop per station per plan	Unique (plan_id, station_id) on stop	BR-24
Tank values in range	CHECK constraints on tank	DE-03
Tanker capacity = sum of compartments	Deferred constraint trigger on tanker and tanker_compartment	DE-07, BR-31
Resources of a decided day cannot change	Trigger refuses new resource_version rows for a day whose latest version is locked	BR-30
Decision log is append-only	Trigger raises on UPDATE, DELETE and TRUNCATE; the app's database role has INSERT and SELECT only	BR-13, NFR-14
Uncertain flag cannot disagree with confidence	Generated column on forecast	BR-27
Only RISK-01 blocks approval	Generated column on risk	BR-21

Rules that need several tables at once (BR-04 to BR-08, BR-28) are checked by PlanValidator and EditService in the same transaction as the save, and again by the API on every request (Section 6.1).

5.5.5 Retention and audit

Data	Kept	Removed by	SRS refs
decision_log_entry, decided plans	Life of the v1	Nothing	BR-13

Data	Kept	Removed by	SRS refs
with their routes, stops, quantities, risks and edits	system, never deleted		
Draft plans never decided	30 days after their delivery day	Daily purge job in CoreModule	SRS 8.6
Forecast rows not used by any plan	30 days	Daily purge job	SRS 8.6
session	7 days after revocation or expiry	Daily purge job	This design (NFR-22)
login_attempt	90 days	Daily purge job	NFR-21
Application logs (files)	90 days	Log rotation	OBS-L1–L5
Inactive users, stations, tankers, drivers	Kept	Never	FR-05, SRS 8.5

Audit design

- The decision log stores snapshots, not references, so an entry stays complete even after a re-plan replaces the plan it described (FR-72, AS-11).
- Database protection works in two layers: the trigger stops any UPDATE or DELETE, and the runtime role `venzini_app` has no right to run them. Migrations run under a separate owner role `venzini_owner` that the API never uses at runtime.
- Every log entry carries the correlation ID of the request that wrote it, so a reviewer can join the business record with the technical logs (OBS-L5).
- The status change and its log entry are written in one transaction: both succeed or neither does (FR-71, ERR-SYS-04).

5.6 Interface view

The browser and the API share one REST/JSON contract: 19 endpoints from the SRS (API-01 to API-19) plus four small read endpoints that the screens need. Every request and response body is a Zod schema in `packages/contracts`; NestJS turns them into the OpenAPI document served at `/api/docs`.

5.6.1 Common rules (all endpoints)

Topic	Rule	SRS refs
Base path and format	`/api/v1`, JSON, UTF-8. Litres are integers, money is a number with 2 decimals in USD, date-times are ISO 8601 UTC, delivery days are `YYYY-MM-DD`	SRS 9.1, NFR-31
Session	Cookie `vz_session` (HttpOnly, Secure, SameSite=Strict, Path=/api) holding the JWT; missing or expired → 401 ERR-AUTH-05	NFR-22

Topic	Rule	SRS refs
Role check	Every endpoint declares its roles with `@Roles(...)`; default deny → 403 ERR-AUTH-04, logged as a security event	NFR-05, OBS-L3
CSRF	Every POST, PUT, PATCH and DELETE must carry the header `X-Requested-With: venzini`; missing → 403 ERR-AUTH-06	SRS 12
Validation	The Zod schema runs on the server for every write, even if the browser already checked; the error names the field	SRS 8.4
Error body	`{"error": {"code", "message", "field"?, "details"?}}`; never a stack trace	SRS 9.1, 14
Correlation	Every response has `X-Correlation-Id`; the same ID is on every log line of that request	OBS, 15.3
Plan versions	Every write to a plan sends `expected_version`; a mismatch → 409 ERR-API-03 "This plan changed since you opened it. Reload to see the latest version."	NFR-26
Lists	`page` (default 1) and `page_size` (default 50, max 200); response `{items, page, page_size, total}`	–
Rate limits	Login 10/min per IP; plan generation 6/min per user; others 120/min per user → 429 ERR-API-02	SRS 9.1
Plan writes	Every write that changes a plan returns the full `PlanView`, so the UI never computes a business value	SRS 9.1

5.6.2 Endpoint index

ID	Method and path	Roles	Module	FRs
API-01	POST /auth/login	Public	AUTH	FR-01
API-02	POST /auth/logout	All	AUTH	FR-01
API-03	GET /me	All	AUTH	FR-02
API-04	GET, POST /users · PATCH /users/{id}	Admin	ADMIN	FR-05
API-05	GET, POST /stations · PATCH	Manager,	ADMIN	FR-06, FR-14

ID	Method and path	Roles	Module	FRs
	/stations/{id}	admin		
API-06	GET /resources/{day}	Dispatcher, manager	RES	FR-07–09, FR-13
API-07	PUT /resources/{day}	Dispatcher	RES	FR-07–12
API-08	PATCH /trips/{routeId}	Dispatcher	RES	FR-79, FR-84
API-09	GET /network	Manager	RISK	FR-14–16, FR-24
API-10	GET /forecasts	Manager	FC	FR-17–23, FR-83
API-11	POST /plans	Manager	ROUTE	FR-25–42, FR-78
API-12	GET /plans/{id}	Manager	ROUTE	FR-42–54, FR-61
API-13	POST /plans/{id}/edits	Manager	EDIT	FR-55–61, FR-75–77
API-14	GET /plans/{id}/limits	Manager	EDIT	FR-75
API-15	POST /plans/{id}/stops/{stopId}/decision	Manager	DEC	FR-69, FR-71, FR-72, FR-82
API-16	POST /plans/{id}/decision	Manager	DEC	FR-70, FR-80
API-17	GET /decisions	All	DEC	FR-73
API-18	GET /decisions/{id}	All	DEC	FR-73
API-19	GET /forecasts/totals	Manager; dispatcher (warehouse level)	FC	FR-85
API-P1	GET /reference	All	DATA	Fuel types, warehouses, regions for selectors
API-P2	GET /plans?day={day}	Manager	ROUTE	Finds the current plan of a day (FR-68, UI-05, UI-06)
API-P3	GET /trips?day={day}	Dispatcher, manager	RES	Trips list on UI-04 (FR-79, FR-84)
API-P4	GET /health	Public	CORE	Container health check (Section 5.11)

No endpoint updates or deletes a decision log entry, sets a price, or controls equipment (NFR-14, NFR-17).

5.6.3 Contracts: access, administration and resources

API-01 · POST /auth/login · Public · rate limit 10/min per IP

```
// request
{ "login": "sara.m", "password": "••••••••••" }
// 200 + Set-Cookie: vz_session=<jwt>; HttpOnly; Secure; SameSite=Strict; Path=/api
{ "user": { "id": "6f1c…", "full_name": "Sara M.", "role": "manager" },
  "home": "/network", "idle_timeout_min": 30 }
```

- Validation: login 3–40 characters; password 1–200 characters.
- Errors: 401 ERR-AUTH-01 (wrong login or password, same message for both), 401 ERR-AUTH-02 (inactive), 401 ERR-AUTH-03 with `details.retry_after_min`, 429 ERR-API-02.
- Homes: manager `/network`, dispatcher `/resources`, admin `/admin` (FR-01).

API-02 · POST /auth/logout · All roles → 204, revokes the session row and clears the cookie at once (NFR-22).

API-03 · GET /me · All roles

```
{ "user": { "id": "6f1c…", "full_name": "Sara M.", "role": "manager" },
  "pages": ["network", "forecast", "resources", "planner", "approvals", "stations", "history"],
  "can": { "edit_plan": true, "decide": true, "save_resources": false, "record_trips": false,
           "manage_users": false, "manage_stations": true } }
```

The frontend builds its menu from `pages` and hides controls from `can`; the server still checks every request (UI hiding is not a security control).

API-04 · Users · Admin

```
// POST /users
{ "full_name": "Karim B.", "login": "karim.b", "password": "init-Passw0rd", "role": "dispatcher"
}
// PATCH /users/{id}   (any subset)
{ "full_name": "Karim Ben Ali", "role": "dispatcher", "status": "inactive", "password": "new-
Passw0rd!" }
// 200/201
{ "id": "0b9e…", "full_name": "Karim B.", "login": "karim.b", "role": "dispatcher", "status":
"active", "created_at": "2026-10-26T09:12:00Z" }
```

- Validation: full_name 1–80; login 3–40, unique ignoring case; password ≥ 10 characters; role in the three roles.
- Errors: ERR-VAL-01, ERR-VAL-02, ERR-VAL-03 (duplicate login), ERR-VAL-07 (admin deactivating self).
- Side effects: deactivation revokes the user's sessions; every change is written to OBS-L4 with before and after values (no password).

API-05 · Stations · Manager, admin (D-15)

```
// POST /stations
{ "station_id": "ST021", "name": "Lac 2 Centre", "latitude": 36.8421, "longitude": 10.2731,
  "region": "Tunis North", "window_open": "06:00", "window_close": "18:00",
  "tanks": [ { "fuel_type_id": "P95", "capacity_l": 30000, "safety_stock_l": 6000,
"current_level_l": 14200 },
             { "fuel_type_id": "DIESEL", "capacity_l": 40000, "safety_stock_l": 8000,
"current_level_l": 22500 } ] }
// 201
{ "station_id": "ST021", "serving_warehouse_id": "WH-RIV", "status": "active", "...": "same
fields as the request" }
```

- GET /stations accepts `status`; PATCH accepts any subset plus `status`.
- Validation: SRS 8.1 rules; safety stock or level above capacity → ERR-VAL-02 naming the field; duplicate ID → ERR-VAL-03.
- Side effect: the serving warehouse is recomputed from IF-DIST (BR-17); a station without distances is saved but flagged ERR-DATA-02 in plans.

API-06 · GET /resources/{day} · Dispatcher, manager

```
{ "delivery_day": "2026-11-03", "resource_version_id": "RV-0192", "version_no": 3, "locked":
false,
  "updated_by": { "id": "0b9e…", "full_name": "Karim B." }, "updated_at": "2026-11-02T16:40:00Z",
  "tankers": [ { "tanker_id": "T-01", "capacity_l": 36000, "compartments": [12000, 12000, 12000],
                 "home_warehouse_id": "WH-RIV", "available": true, "open_trip": null } ],
  "drivers": [ { "driver_id": "D-04", "full_name": "Ali H.", "home_warehouse_id": "WH-RIV",
"available": true } ],
  "stock": [ { "warehouse_id": "WH-RIV", "fuel_type_id": "DIESEL", "stock_l": 480000 } ] }
```

No resources for the day → 404 ERR-API-01; the UI shows "No resources uploaded for this day" (FR-13).

API-07 · PUT /resources/{day} · Dispatcher

Request as in SRS 9.3 (`tankers[]`, `drivers[]`, `stock[]`). Response:

```
{ "resource_version_id": "RV-0193", "version_no": 4,
  "warnings": [ { "code": "WARN-RES-01", "tanker_id": "T-03", "route_id": "R-11",
                  "message": "Tanker T-03 has not returned from trip R-11. Check before marking
it available." } ] }
```

- Validation: day is a valid date; every ID exists and is active; stock integer ≥ 0; at least one value changed. A full new version is written in one transaction (NFR-26).
- Errors: ERR-VAL-01, ERR-VAL-02 with `field` such as `stock[2].stock_l`; 409 ERR-VAL-04 when the day is locked.
- Side effect: a current draft plan built on an older version is shown with `resources_changed: true` (FR-11 alternative flow).

API-08 · PATCH /trips/{routeId} · Dispatcher

```
// request (one or both)
{ "actual_departure": "2026-11-03T05:10:00Z", "actual_return": "2026-11-03T13:55:00Z" }
// 200
{ "route_id": "R-02", "trip_status": "returned", "actual_departure": "2026-11-03T05:10:00Z",
  "actual_return": "2026-11-03T13:55:00Z", "time_gap_min": 45, "time_gap_pct": 9.4 }
```

- Validation: departure not in the future; return after departure; the route belongs to a decided plan and has at least one approved stop; a time may be corrected until the return is recorded (FR-79).
- Cancel: the request `{"cancel": true, "reason": "Tanker breakdown"}` is accepted only while `trip_status = planned`; it sets `trip_status = cancelled` and stores `cancel_reason` (up to 200 characters). A trip that has departed cannot be cancelled: 409 ERR-VAL-04.
- Errors: ERR-VAL-02 (return before departure), 409 ERR-VAL-04 (trip not approved or already returned).

API-P3 · GET /trips?day={day} · Dispatcher, manager → list of trips with route, tanker, driver, estimated and actual times, status and gap (UI-04 Trips tab).

5.6.4 Contracts: network, forecasts and plans

API-09 · GET /network?day={day} · Manager

```
{ "as_of": "2026-11-02T07:00:00Z",
  "kpis": { "high_risk_stations": 3, "tankers_available": 5, "tankers_total": 6,
            "warehouse_stock": [ { "warehouse_id": "WH-RIV", "fuel_type_id": "DIESEL", "stock_l":
480000 } ],
            "plan_status": "draft" },
  "stations": [ { "station_id": "ST014", "name": "Harbor Road", "latitude": 36.80, "longitude":
10.18,
                  "risk_level": "high", "data_available": true,
                  "tanks": [ { "fuel_type_id": "DIESEL", "current_level_l": 4200, "capacity_l":
40000, "level_pct": 11,
                               "safety_stock_l": 8000, "risk_level": "high", "hours_to_stockout":
14.0 } ] } ],
  "warehouses": [ { "warehouse_id": "WH-RIV", "name": "Rives Depot", "latitude": 36.79,
"longitude": 10.25 } ],
  "alerts": [ { "station_id": "ST014", "fuel_type_id": "DIESEL", "hours_to_stockout": 14.0,
                "reason": "ST014 is high risk: current Diesel stock covers 14 hours of forecast
demand." } ],
  "data_unavailable": [ { "station_id": "ST019", "code": "ERR-DATA-01", "reason": "No tank level"
} ] }
```

Alerts list high-risk pairs only, shortest time to stockout first (FR-15).

API-10 · GET /forecasts?station={id}&fuel={id}&ref_date={date?} · Manager


```
{ "forecast_id": "FC-ST014-DIESEL-20261102", "station_id": "ST014", "fuel_type_id": "DIESEL",
  "reference_date": "2026-11-02", "status": "ok", "model_version": "fc-sim-1.0.0",
  "daily": [ { "date": "2026-11-03", "litres": 7200, "confidence": 82, "band_low": 6400,
"band_high": 8000 } ],
  "overall_confidence": 66, "uncertain": true,
  "uncertain_reason": "Confidence is 66%: weather data is missing for 2 of 7 days.",
  "factors": [ { "factor": "local_events", "impact_pct": 8.0, "available": true,
                 "explanation": "A football match on 5 Nov raises expected demand by 8%." } ],
  "recent_actuals": [ { "date": "2026-10-27", "litres": 6950 } ] }
```


- `daily` always has 7 items starting the day after `reference_date` (FR-18); `ref_date` defaults to today.
- Short history: 200 with `"baseline": "similar_stations"`, daily values taken from similar stations, confidence capped at 60% and the note TPL-FC-09 (FR-83).

API-19 · GET /forecasts/totals?level={network|warehouse}&fuel={id}&warehouse={id?} · Manager (both levels); dispatcher (warehouse level only)


```
{ "level": "warehouse", "fuel_type_id": "DIESEL", "reference_date": "2026-11-02",
  "groups": [ { "warehouse_id": "WH-RIV",
                "days": [ { "date": "2026-11-03", "total_l": 61400, "band_low": 55200,
"band_high": 67900,
                            "uncertain_share_pct": 12.5 } ] } ],
  "excluded": [] }
```


A dispatcher asking for `level=network` gets 403 ERR-AUTH-04 (D-17). The dispatcher response never contains station-level values.

API-P1 · GET /reference · All roles → fuel types, warehouses (ID, name, position) and regions, for selectors and legends; cached for 10 minutes in the browser.

API-11 · POST /plans · Manager · rate limit 6/min


```
// request
{ "delivery_day": "2026-11-03" }
// 201 → PlanView (shape below)
```


- Builds the optimal plan from the latest resource version; the previous current plan of the day, if any, stays stored with `is_current = false`.
- Errors: 422 ERR-PLAN-02 (no resources), 422 ERR-PLAN-01 naming the missing resource, 409 ERR-VAL-04 (day already decided), 500 ERR-SYS-02 (validator found a violation; plan not stored or shown).

PlanView – returned by API-11, API-12, API-13 and API-P2


```
{ "plan": { "plan_id": "PL-2212", "delivery_day": "2026-11-03", "kind": "edited",
"parent_plan_id": "PL-2211",
            "status": "edited", "version": 4, "resource_version_id": "RV-0192",
"resources_changed": false,
            "time_limited": false,
            "totals": { "km": 612.4, "minutes": 1490, "litres": 148600, "cost_usd": 412.35 } },
  "routes": [ { "route_id": "R-02", "tanker_id": "T-02", "driver": { "driver_id": "D-05",
"full_name": "Marcus K." },
                "home_warehouse_id": "WH-RIV", "color": "route-2",
                "est_departure": "2026-11-03T05:00:00Z", "est_return": "2026-11-03T13:10:00Z",
                "totals": { "km": 142.0, "minutes": 490, "litres": 34000, "cost_usd": 98.12 },
                "load": { "capacity_l": 32000, "assigned_l": 34000,
                          "compartments": [ { "position": 1, "size_l": 16000, "fuel_type_id":
"P95", "litres": 16000 } ] },
                "segments": [ { "from": "WH-RIV", "to": "ST032", "km": 24.0, "minutes": 32 } ],
                "stops": [ { "stop_id": "S-07", "station_id": "ST032", "station_name": "Westside
Terminal",
                             "stop_order": 1, "est_arrival": "2026-11-03T05:32:00Z",
"priority_rank": 4,
                             "risk_level": "medium", "hours_to_stockout": 40.5, "uncertain":
false,
                             "reason": "ST032 is medium risk: Petrol 95 stock covers 40 hours of
forecast demand.",
                             "quantities": [ { "fuel_type_id": "P95", "recommended_l": 7500,
"final_l": 7500 } ],
                             "decision_status": "pending" } ] } ],
  "unserved": [ { "station_id": "ST019", "fuel_type_id": "DIESEL", "risk_level": "high",
                  "hours_to_stockout": 20.0, "reason": "No tanker capacity left at WH-RIV." } ],
  "fallback_forecast": [ { "station_id": "ST020", "fuel_type_id": "P95", "confidence": 60 } ],
  "comparison": { "cost_optimal": 388.10, "cost_new": 412.35, "cost_diff": 24.25,
"cost_diff_pct": 6.2,
                  "km_diff_pct": 6.8, "minutes_diff_pct": 4.1 },
  "risks": [ { "risk_id": "RK-31", "type": "RISK-01", "severity": "high", "blocks_approval":
true,
               "affected": { "routes": ["R-02"], "stops": ["S-07"], "segments": [] },
               "reason": "Tanker T-02 is overloaded: 34,000 L on a 32,000 L tanker. Remove 2,000
L before approving." } ],
  "approval_blocked": true }
```


`comparison` is null for an unedited optimal plan, which the UI shows as "Optimal plan, no changes" (FR-43). Route colors are token names, mapped to the palette in Section 5.12.4.

API-12 · GET /plans/{id} · Manager → PlanView. Unknown ID → 404 ERR-API-01.

API-P2 · GET /plans?day={day} · Manager → the current PlanView of the day, or 404 when no plan exists (the planner then offers "Generate plan").

API-13 · POST /plans/{id}/edits · Manager

```
// reorder a stop inside its route (FR-55)
{ "kind": "reorder", "expected_version": 4, "stop_id": "S-07", "position": 3 }
// move a station to another tanker of the same warehouse (FR-56)
{ "kind": "move", "expected_version": 4, "stop_id": "S-07", "to_route_id": "R-02", "position": 2
}
// change a quantity (FR-58); route edits are replaced, so the UI asks first (FR-59)
{ "kind": "quantity", "expected_version": 4, "stop_id": "S-03", "fuel_type_id": "P95", "litres":
7000,
  "confirm_replace_route_edits": true }
// reset to the optimal plan (FR-60)
{ "kind": "reset", "expected_version": 4 }
// 200 → PlanView with version + 1
```

- Validation: `kind` decides the schema (Zod discriminated union); `position` 1..n+1; `litres` integer ≥ 0 and ≤ the BR-28 maximum; `confirm_replace_route_edits` required when the plan has reorder or move edits.
- Errors: 409 ERR-VAL-04 (plan decided), 400 ERR-VAL-05a/b/c (quantity above the limit), 422 WARN-EDIT-01 (move to another warehouse's tanker; nothing changed), 422 ERR-PLAN-03 (no feasible plan with the fixed quantities; edit undone), 409 ERR-API-03 (stale version), 500 ERR-SYS-01 (recalculation failed; edit undone).
- An overloading move is accepted and returns RISK-01 with `approval_blocked: true` (D-02).

API-14 · GET /plans/{id}/limits?stop={id}&fuel={id} · Manager

```
{ "max_l": 8000, "binding_limit": "tanker",
  "limits": { "tanker": { "tanker_id": "T-01", "capacity_l": 36000, "assigned_other_l": 28000,
"free_l": 8000 },
              "tank": { "free_storage_l": 11800 },
              "warehouse": { "warehouse_id": "WH-RIV", "remaining_l": 52000 } },
  "hint": "Max for this stop: 8,000 L. Tanker T-01 has 8,000 L left (36,000 L capacity, 28,000 L
already assigned)." }
```

A limit that cannot be computed is returned as null with `binding_limit: "unknown"` and a hint naming the missing data; the UI then disables Save (FR-75).

5.6.5 Contracts: decisions and history

API-15 · POST /plans/{id}/stops/{stopId}/decision · Manager

```
// request
{ "decision": "reject", "expected_version": 4, "note": "Station closed for works on 3 Nov." }
// 200 – approve, or reject with re-plan (FR-82)
{ "stop": { "stop_id": "S-07", "decision_status": "rejected" },
  "plan": { "plan_id": "PL-2215", "status": "partly_decided", "version": 1 },
  "replan": { "previous_plan_id": "PL-2212", "not_dispatched": { "station_id": "ST032",
"risk_level": "medium" },
              "kept_approved": ["S-02", "S-04"], "returned_to_pending": ["S-09"], "newly_served":
["ST019"] },
  "log_entry_ids": ["5c1d…"] }
```

- `replan` is null for an approval. The UI asks for confirmation before a rejection and shows the station's risk (FR-82).

- Validation: decision in approve, reject; note ≤ 500 characters, stored and shown as plain text.
- Errors: 409 ERR-PLAN-04 (RISK-01 on this stop's tanker), 409 ERR-VAL-04 (already decided), 422 ERR-PLAN-03 (no feasible re-plan; rejection not applied, previous plan kept), 409 ERR-API-03, 500 ERR-SYS-04 (log write failed; nothing changed).

API-16 · POST /plans/{id}/decision · Manager

```
// request
{ "decision": "approve_all", "expected_version": 4 }
// 200
{ "plan": { "plan_id": "PL-2212", "status": "decided", "version": 5 },
  "decided_actions": 5, "resources_locked": true,
  "log_entry_ids": ["…5 action entries…", "…1 plan entry…"] }
```

- Applies to pending actions only; approved or rejected actions keep their status. One log entry per action plus one plan-level entry (FR-80, AC-14).
- `reject_all` rejects every pending action without re-planning: the plan is then decided with those stations not dispatched (FR-82 alternative flow).
- Errors: 409 ERR-PLAN-04 for approve_all while RISK-01 exists; 409 ERR-VAL-04 when no action is pending; 409 ERR-API-03; 500 ERR-SYS-04.

API-17 · GET /decisions · All roles

Query: `from`, `to` (dates, from ≤ to), `type` (approved, rejected, modified, not_dispatched), `station`, `user`, `q` (free search on station and manager name), `page`, `page_size`.

```
{ "items": [ { "entry_id": "5c1d…", "decided_at": "2026-10-24T14:32:00Z",
               "manager": { "id": "6f1c…", "full_name": "Sara M." }, "scope": "action",
"decision": "approved",
               "station_id": "ST004", "plan_label": "modified", "cost_diff_usd": 17.0,
"cost_diff_pct": 1.4,
               "risk_count": 1, "max_severity": "medium" } ],
  "page": 1, "page_size": 50, "total": 118 }
```

Sorted newest first (FR-73). `plan_label` is one of: as recommended, modified, not dispatched (UI-07).

API-18 · GET /decisions/{id} · All roles → the full entry (every TBL-24 column, snapshots included). Unknown ID → 404 ERR-API-01. There is no PUT, PATCH or DELETE on this path (NFR-14).

5.6.6 Provider interfaces (IF-*)

Each external data source is reached through exactly one TypeScript interface in DataModule (NFR-15). v1 binds each to a simulated adapter; a real adapter can be bound later with the environment variable shown, with no change to other modules or the UI (NFR-12).

```
// packages/contracts/src/providers.ts
export interface WeatherProvider {          // IF-WEATHER
  daily(area: string, from: LocalDate, to: LocalDate): Promise<WeatherDay[]>;   // {date, temp_c,
condition}
}
export interface EventsProvider {           // IF-EVENTS
  events(area: string, from: LocalDate, to: LocalDate): Promise<LocalEvent[]>;  // {date, name,
```

```
impact_pct}
}
export interface DistanceProvider {          // IF-DIST
  matrix(pointIds: string[]): Promise<DistanceMatrix>;  // km and minutes for every ordered pair
}
```

Interface	v1 adapter	Data behind it	Swap setting	Missing data behavior
IF-WEATHER	SimulatedWeatherAdapter	external_factor rows (kind = weather), generated by the seed script with a fixed seed	PROVIDER_WEATHER=simulated	Day without weather → factor "not available", confidence lowered, input named (ERR-AI-02)
IF-EVENTS	SimulatedEventsAdapter	external_factor rows (kind = event)	PROVIDER_EVENTS=simulated	No event → factor impact 0%, not an error
IF-DIST	SimulatedDistanceAdapter	distance_entry table (506 pairs) built by ALG-09 (Floyd–Warshall) from the simulated road network; cached in memory and rebuilt when a station changes	PROVIDER_DIST=simulated	Missing pair → station left out with ERR-DATA-02
Map tiles	Leaflet tile layer	tile.openstreetmap.org, in the browser only	VITE_MAP_TILE_URL	Tiles fail → plain background, routes still drawn (ERR-SYS-03)

5.6.7 Error codes added by this design

Code	HTTP	Message	When
ERR-API-03	409	This plan changed since you opened it. Reload to see the latest version.	expected_version does not match (two tabs or two managers)
ERR-AUTH-06	403	Request blocked for security reasons. Reload the page and try again.	CSRF header missing on a write

All other codes are used exactly as listed in SRS Section 14.2.

5.7 Interaction view

Eight sequence diagrams cover the flows where several modules work together or where the order of steps matters for safety. Every flow that changes data runs in one database transaction; flows marked "advisory lock" also hold a lock on the delivery day (Section 5.10.3).

Diagram	Flow	Use case	Main SRS refs
SEQ-01 · Figure 16	Login with lockout	UC-01	FR-01, NFR-21, NFR-22
SEQ-02 · Figure 17	Generate plan	UC-05	FR-12, FR-25–42, FR-78, ERR-SYS-02
SEQ-03 · Figure 18	Edit plan with live recalculation (reorder, move)	UC-06	FR-55–57, FR-61–66, NFR-01
SEQ-04 · Figure 19	Quantity edit with limit and re-plan	UC-06	FR-58, FR-59, FR-75, FR-76
SEQ-05 · Figure 20	Approve or reject one action, with re-plan after rejection	UC-07	FR-69, FR-71, FR-72, FR-77, FR-82
SEQ-06 · Figure 21	Approve all or reject all	UC-07	FR-70, FR-80, BR-30
SEQ-07 · Figure 22	Dispatcher uploads resources	UC-02	FR-07–13, WARN-RES-01
SEQ-08 · Figure 23	Dispatcher records trip times	UC-10	FR-79, FR-84

Figure 16 below shows SEQ-01, login with lockout. The same message is returned for an unknown login and a wrong password, so the response does not reveal which logins exist.

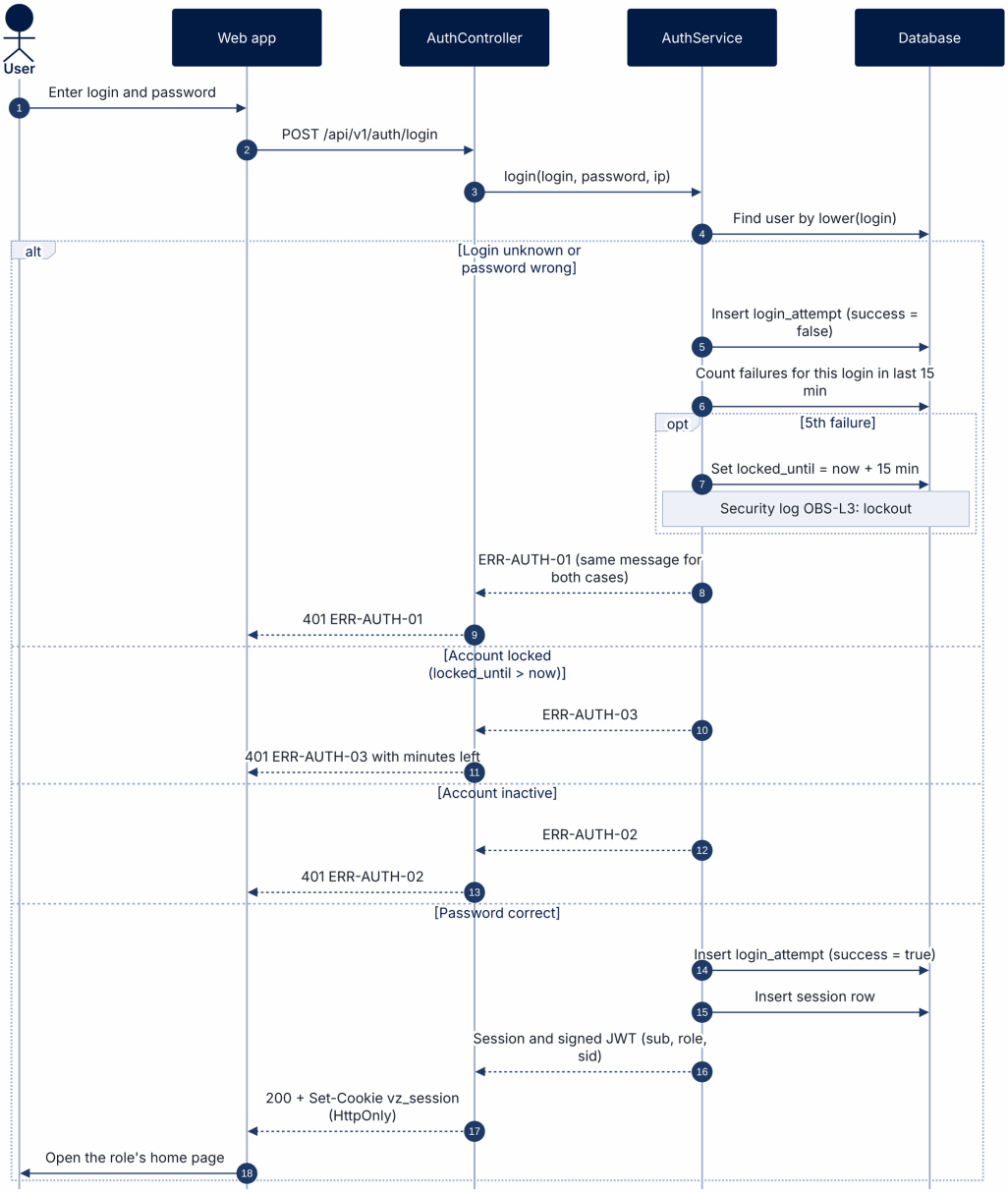


Figure 16 · SEQ-01 Login with lockout

Figure 17 below shows SEQ-02, plan generation. The validator runs after the optimizer on every plan; a plan that breaks a hard rule is never stored or shown (fail safe).

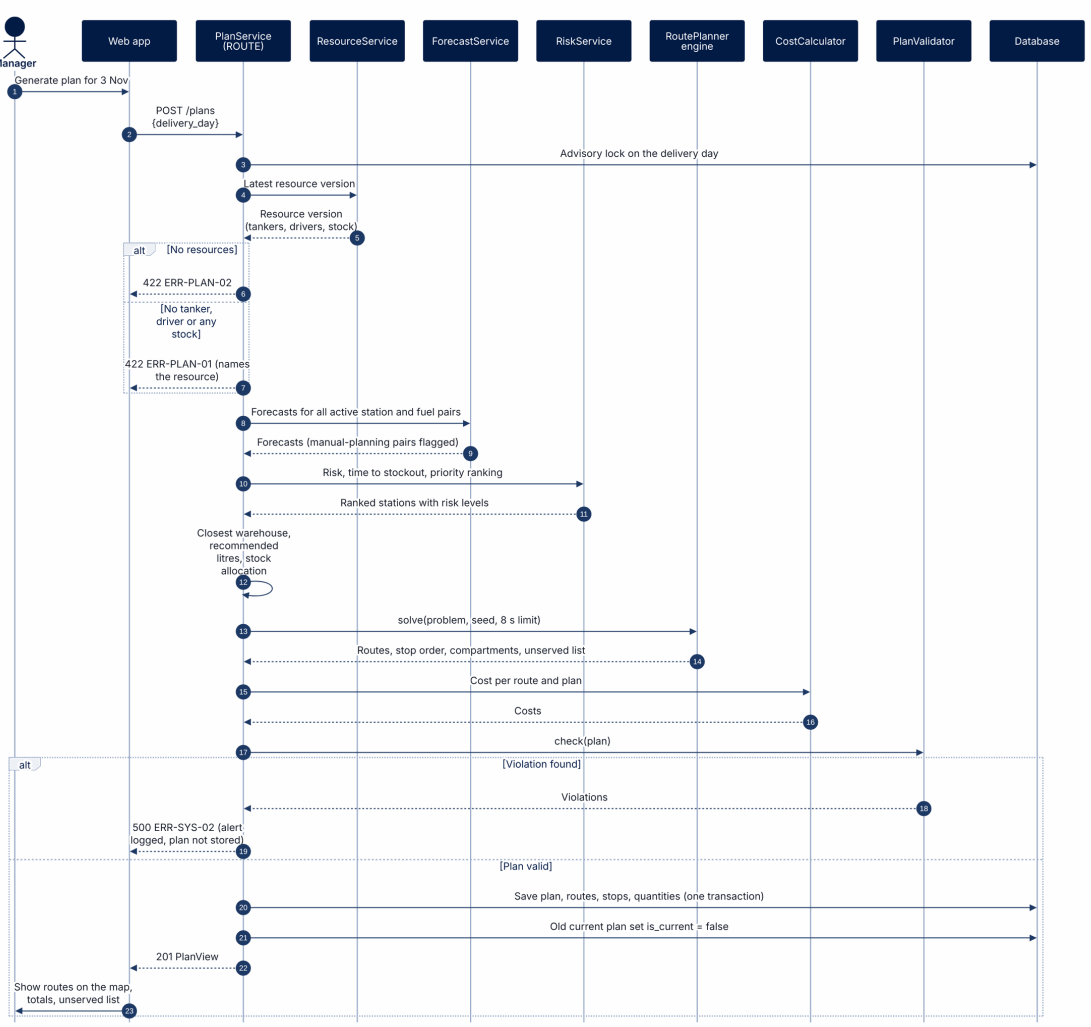


Figure 17 · SEQ-02 Generate plan

Figure 18 below shows SEQ-03, reorder or move with live recalculation. The first edit copies the optimal plan into an edited plan, so the optimal plan stays the comparison baseline and Reset can restore it exactly (FR-60).

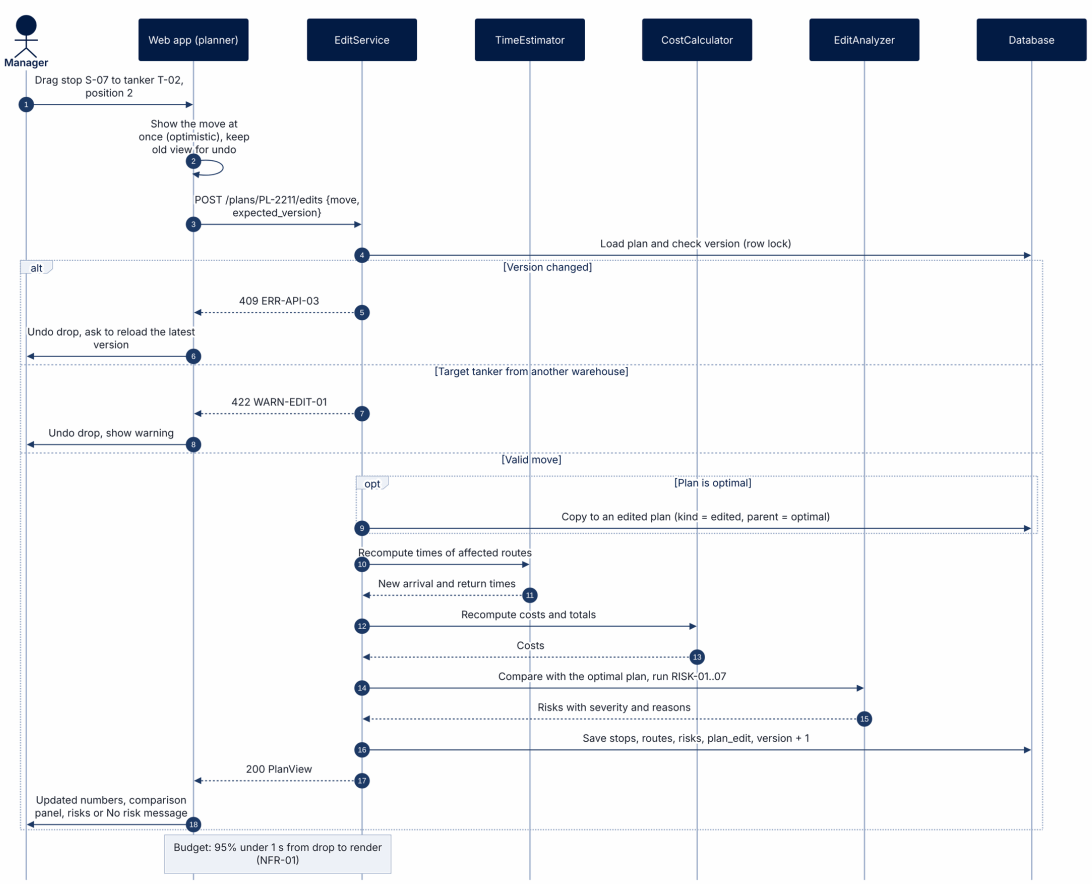


Figure 18 · SEQ-03 Reorder or move with live recalculation

Figure 19 below shows SEQ-04, quantity edit. The limit appears when the Edit panel opens, is checked again on Save in the browser, and once more on the server (BR-28). A quantity edit re-runs the full optimization with that quantity fixed.

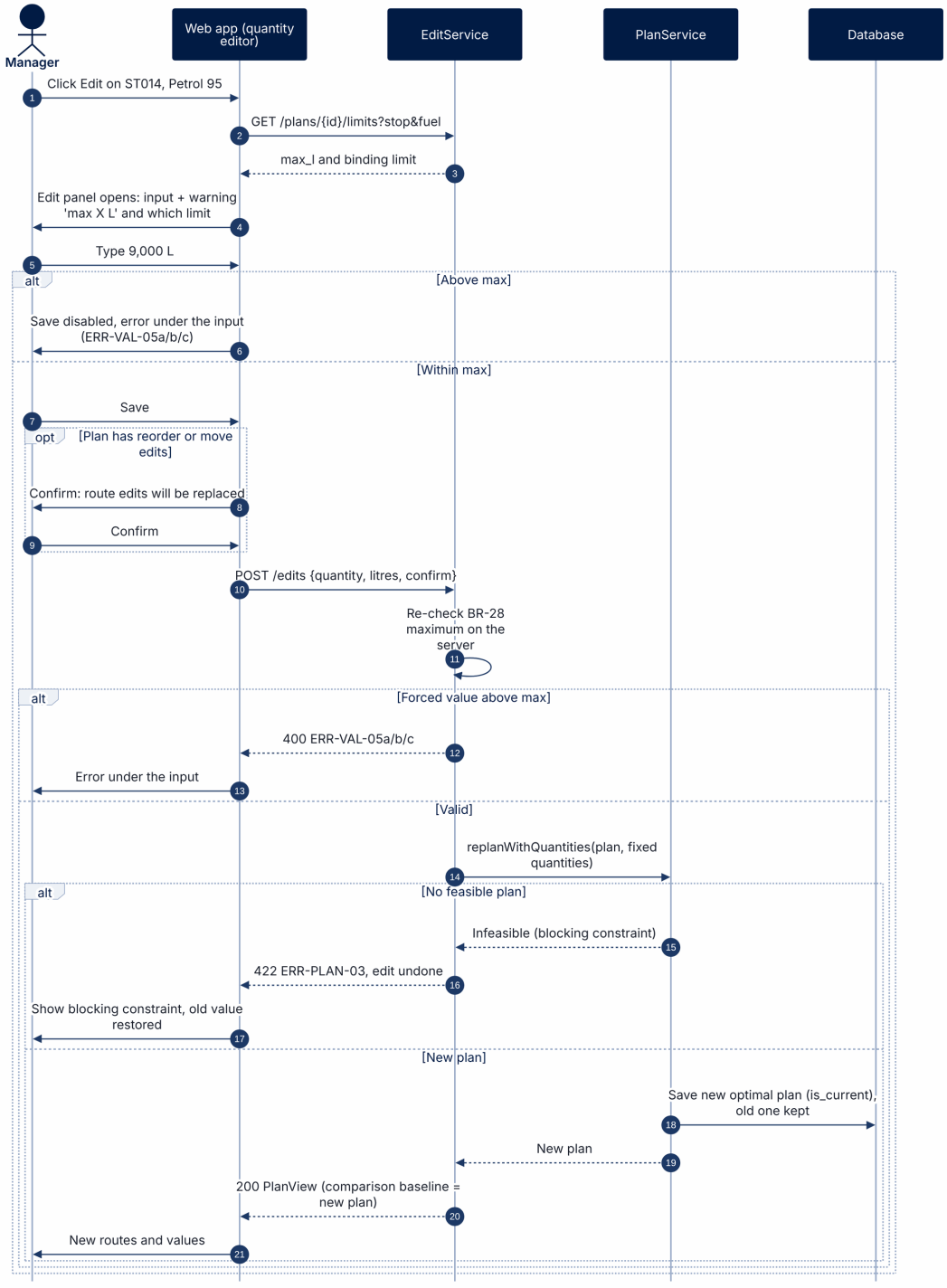


Figure 19 · SEQ-04 Quantity edit

Figure 20 below shows SEQ-05, approval or rejection of one action. Rejection, re-plan, the approvals kept and the log entry are one transaction: if the re-plan fails, nothing is applied, and a blocked approval changes nothing.

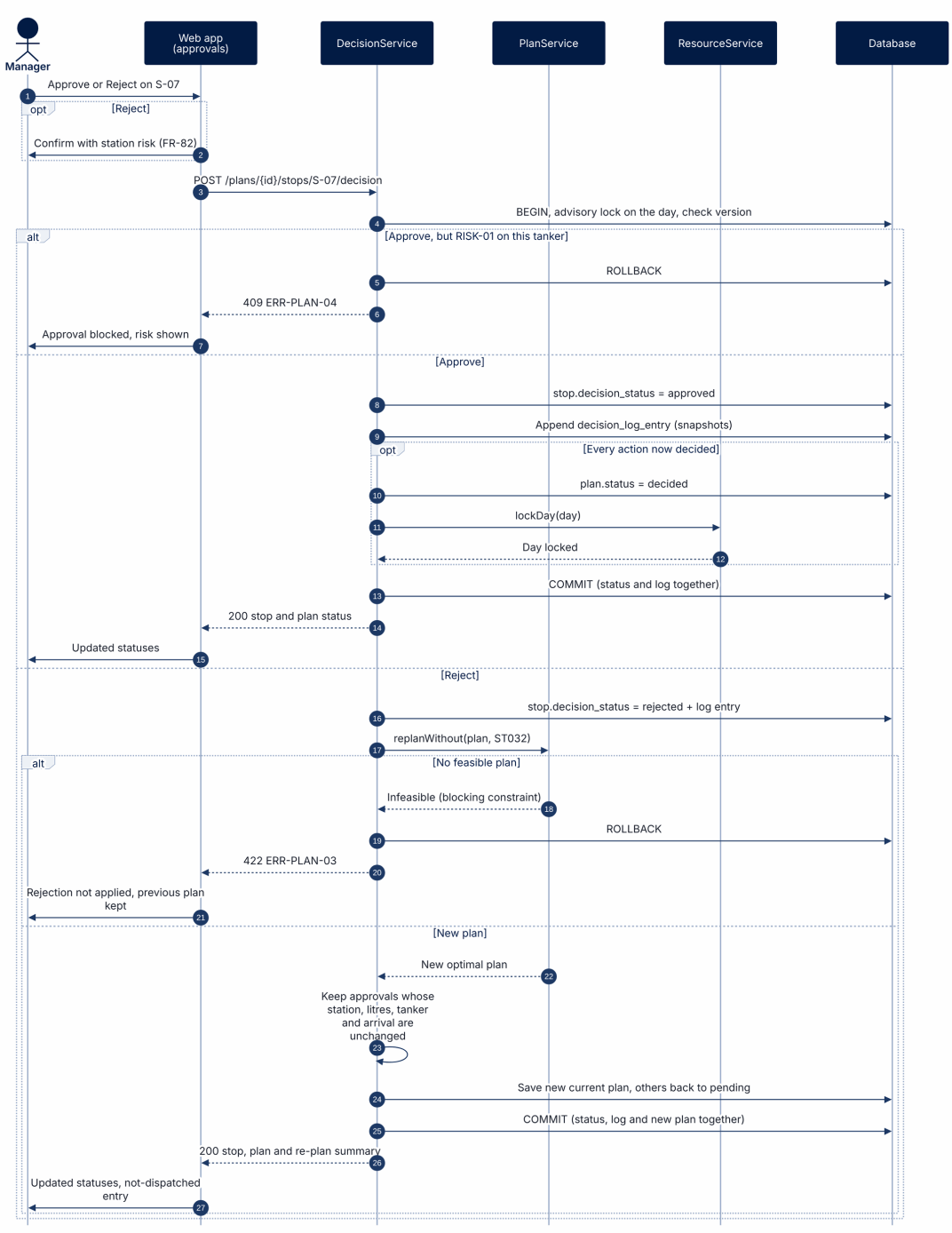


Figure 20 · SEQ-05 Approve or reject one action

Figure 21 below shows SEQ-06, approve all or reject all: one log entry per action plus one plan-level entry, then the day's resources lock.

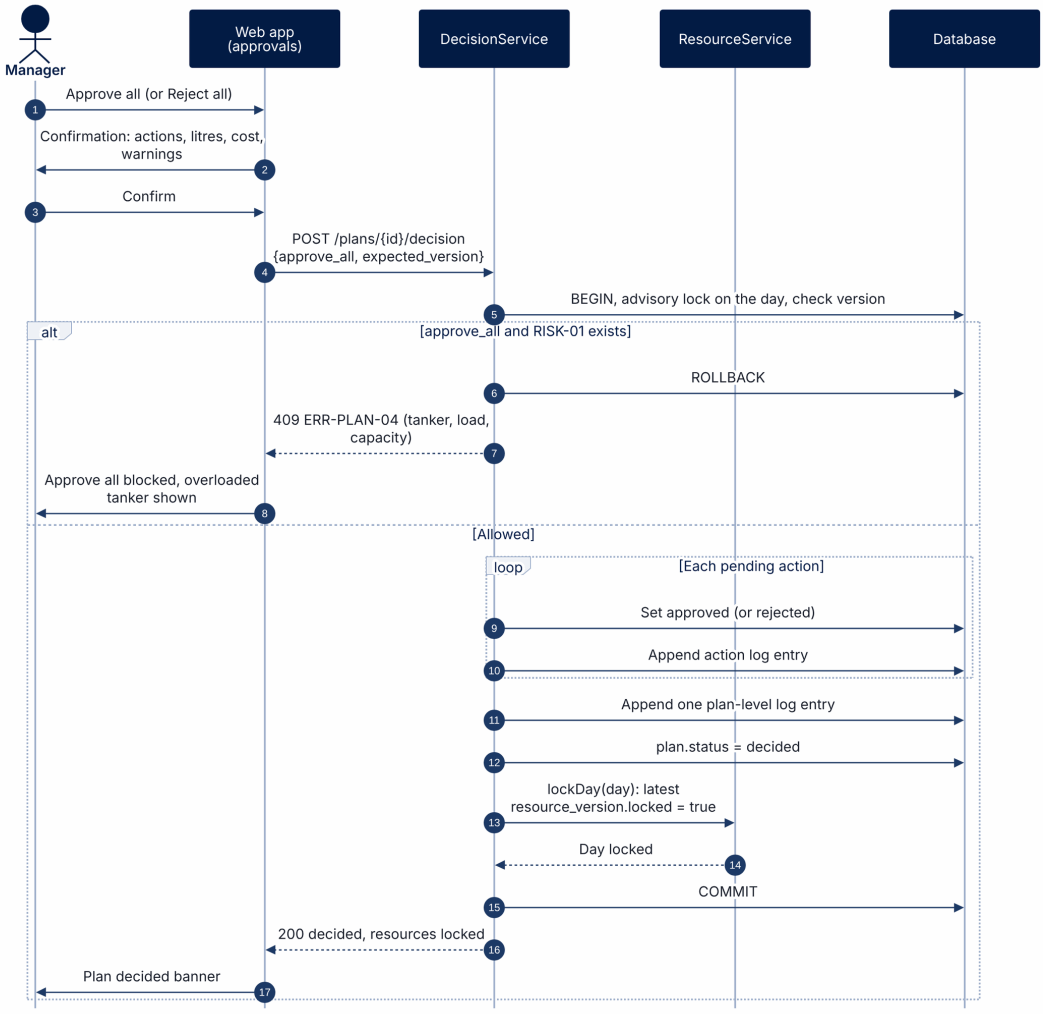


Figure 21 · SEQ-06 Approve all or reject all

Figure 22 below shows SEQ-07, the resource upload by the dispatcher. Every save creates a new full resource version; an open trip only raises a warning.

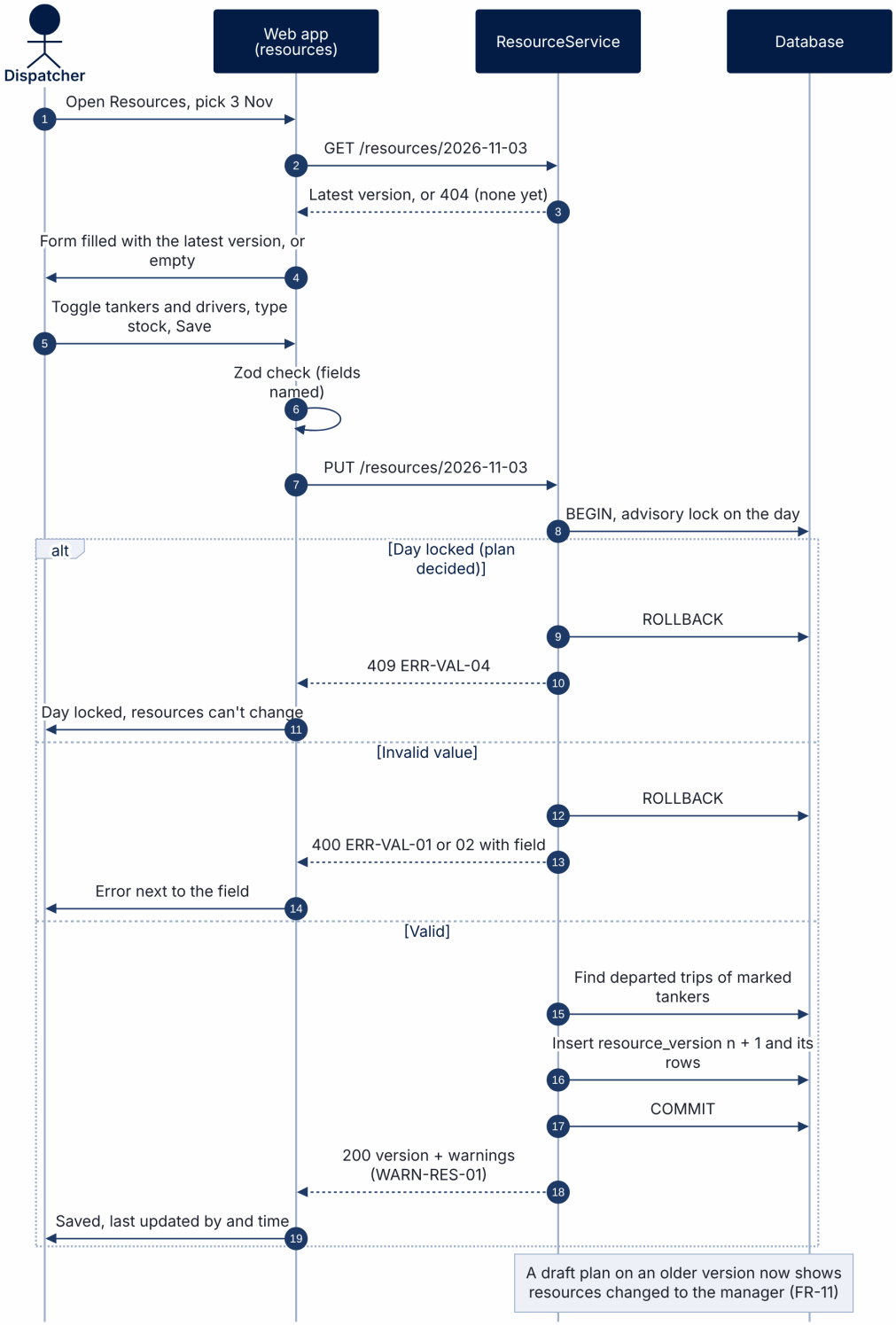


Figure 22 · SEQ-07 Dispatcher uploads resources

Figure 23 below shows SEQ-08, trip time recording by the dispatcher. The gap between the actual and estimated duration is computed when the return is recorded.

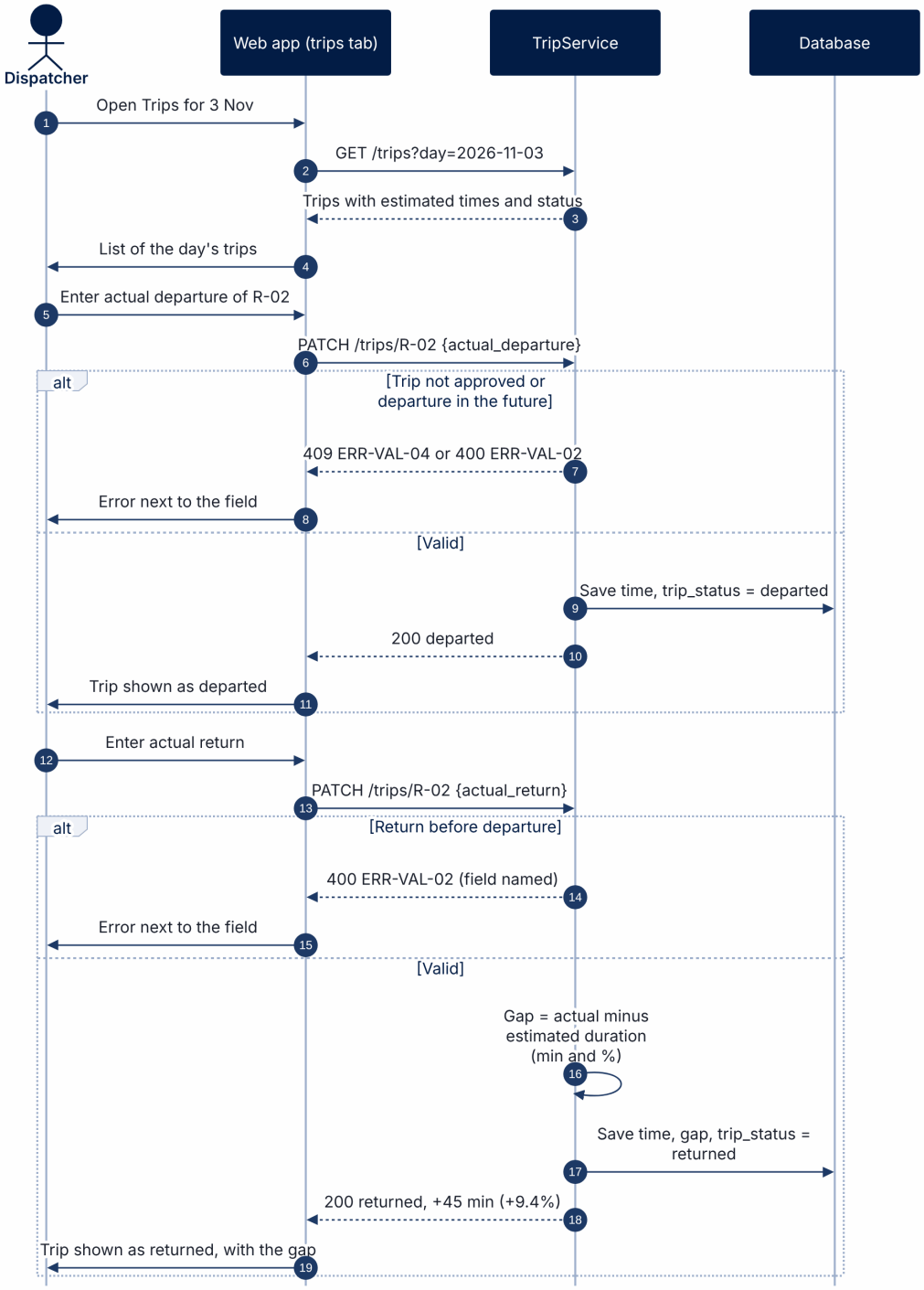


Figure 23 · SEQ-08 Dispatcher records trip times

Optimistic UI on the planner. During a drag, the planner shows the new order at once and greys the numbers it cannot know yet (time, cost, risks). The server answer replaces them; an error puts the old view back and shows the message. This keeps the screen responsive without ever showing a business value the browser computed itself.

5.8 State-dynamics view

Four objects have a lifecycle that the system enforces: the plan, the recommended action (stop), the trip (route) and the user account. A transition not drawn in Figures 24 to 27 is refused by the service that owns the object.

Figure 24 below shows the states of a plan. "Draft" and "Edited" are the SRS statuses draft and edited; a quantity edit or a rejection creates a new plan row that starts again from its own state, and the previous row keeps its history.

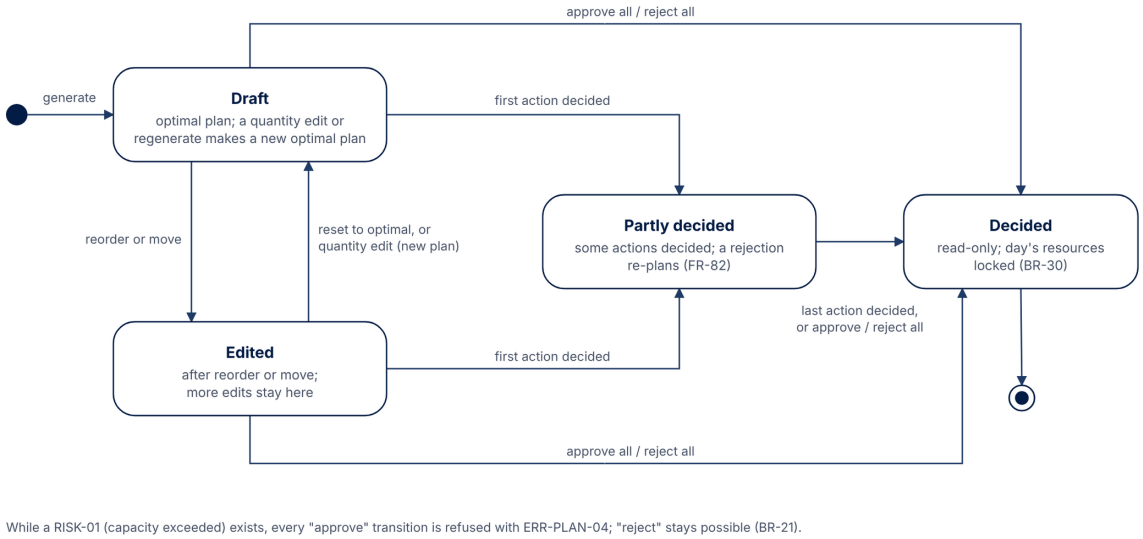


Figure 24 · SM-01 Plan

Figure 25 below shows the states of a recommended action. Only a manager request can move an action to Approved (FR-71); a re-plan that changes an approved action's station, litres, tanker or arrival sends it back to Pending.

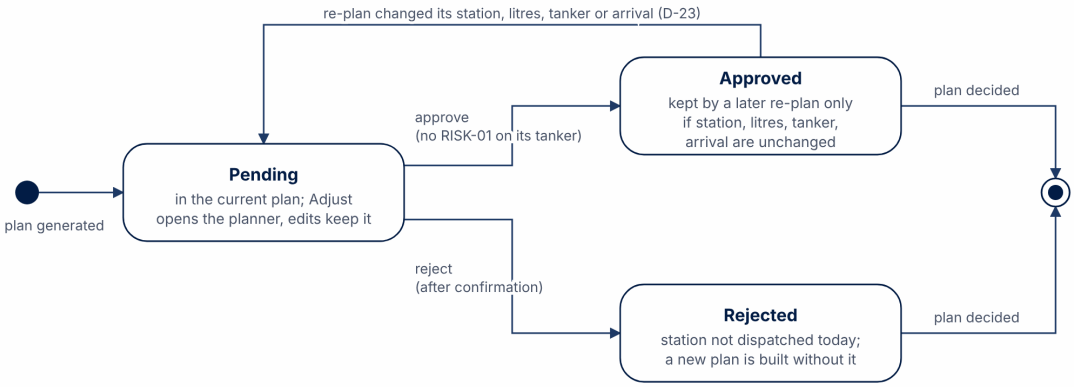


Figure 25 · SM-02 Recommended action

Figure 26 below shows the states of a trip. Only the dispatcher moves a trip forward or cancels it before departure (BR-29); returned and cancelled trips are read-only.

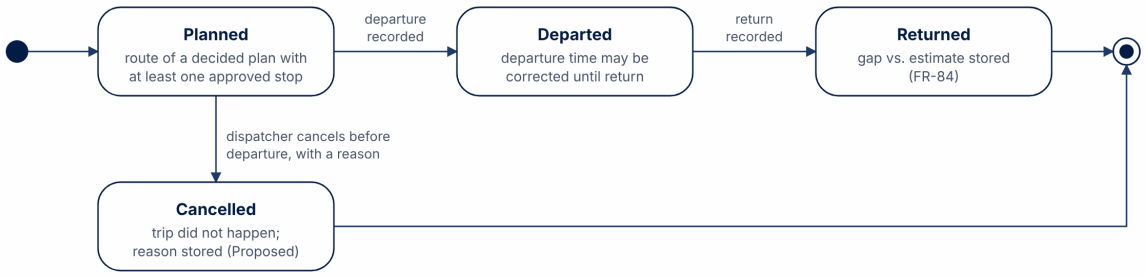


Figure 26 · SM-03 Trip

Figure 27 below shows the states of a user account. Accounts are never deleted, only deactivated (FR-05).

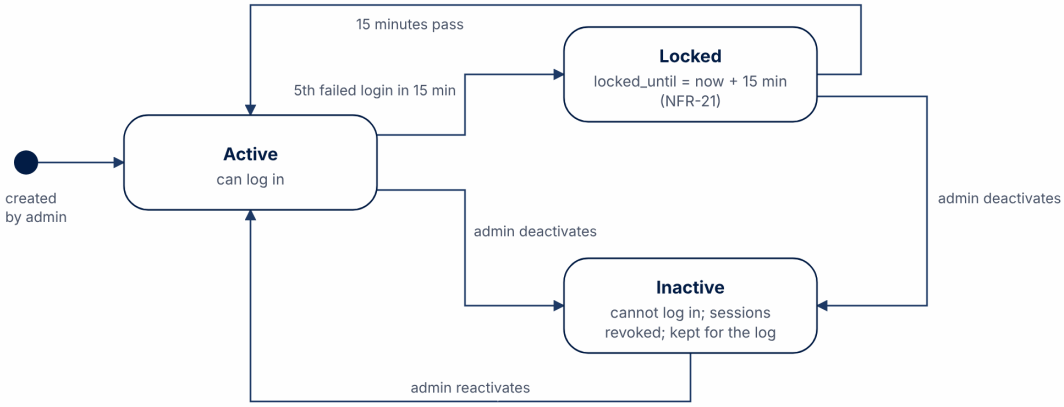


Figure 27 · SM-04 User account

Object	Stored in	Who changes it	Guard checked by
Plan	plan.status, plan.is_current	System on generate and edit; DEC on decisions	PlanService, DecisionService
Action	stop.decision_status	Manager only (approve, reject)	DecisionService, RolesGuard
Trip	route.trip_status	Dispatcher only (times and cancellation)	TripService
User account	app_user.status, locked_until	Admin (status); AuthService (lock)	UserService, LockoutPolicy

5.9 Algorithm view

All algorithms are deterministic: the same inputs and the same seeds give byte-equal outputs (NFR-13). The rules come from the SRS; the formulas and coefficients that fill the SRS's methods (approved in D-27, D-28) are tuned only against the test thresholds in TC-AI-01 to TC-AI-04. Each engine stores a version string in every plan and forecast (AI-18).

ID	Algorithm	Module	SRS refs
ALG-01	Forecasting	FC	AIC-01, FR-18–22, D-28
ALG-02	Confidence and uncertainty	FC	FR-21, FR-23, BR-27, AI-20
ALG-03	Station risk, priority and quantity	RISK, ROUTE	FR-24, FR-25, FR-28, BR-26, AI-16
ALG-04	Routing optimizer (CVRPTW with compartments)	ROUTE	AIC-03, FR-31–41, FR-78, BR-23–25, BR-31
ALG-05	Cost model	COST	FR-40–43, BR-16, D-20
ALG-06	Edit recalculation and risk analysis RISK-01 to RISK-07	EDIT	FR-57, FR-61–66, FR-77, D-18
ALG-07	Maximum quantity check	EDIT	BR-28, FR-75, FR-76
ALG-08	Template explanations	EXPL	AIC-04, D-11, AI-12
ALG-09	Distance matrix (Floyd–Warshall)	DATA	IF-DIST, AS-08, BR-17

5.9.1 ALG-01 Forecasting

ALG-01 predicts the litres sold for a station *s* and a fuel *f* on each of the seven days after a reference date *r*. It treats each station and fuel's daily sales as a **time series** and forecasts it with a **seasonal moving average**, adjusted by multiplicative factors. For a target day $t = r + d$, with $d = 1, \ldots, 7$, the forecast is given by Equation (1):

$$\hat{y}_t = \max\left(0,\ \mathrm{round}\left(B_t \cdot m_t^{season} \cdot m_t^{weather} \cdot m_t^{event} \cdot m_t^{cal} \cdot m^{orders} \cdot \varepsilon_t\right)\right) \quad (1)$$

The **baseline** B_t captures the weekly pattern: Mondays are predicted from past Mondays. It is the average of the same weekday over the last four weeks, Equation (2), where y_{t-7k} is the volume sold *k* weeks earlier and *n* is the number of those days with data (at least 2):

$$B_t = \frac{1}{n}\sum_{k=1}^{4} y_{t-7k} \quad (2)$$

If fewer than two values exist, B_t falls back to the average of the last 28 days and the forecast is marked degraded. If the station has fewer than 14 days of sales in the last 28, B_t is taken from **similar stations**: the average same-weekday sales of stations in the same region, scaled by the ratio of tank capacities. The forecast is then flagged uncertain (ALG-02) but is still planned automatically.

Each multiplier equals 1 when it has no effect. Seasonality compares the target month with the reference month through the monthly index $I(m)$, which is the average daily sales in month m divided by the yearly average (Equation 3). Weather comes from a parameter table (for example ≥ 30 °C → 1.04, rain → 0.95). Events add their impact percentages p_j (Equation 4). Holidays apply $m^{cal} = 1.10$.

$$m_t^{season} = \text{clamp}\left(\frac{I\big(\text{month}(t)\big)}{I\big(\text{month}(r)\big)},\ 0.80,\ 1.25\right) \tag{3}$$

$$m_t^{event} = 1 + \sum_j \frac{p_j}{100} \tag{4}$$

The orders factor compares the litres delivered D_{28} and sold S_{28} over the last 28 days (Equation 5). Deliveries do not cause sales, so this factor is only a weak signal of the station's recent demand trend (D-28); it is therefore capped at ±5% and can never dominate the forecast.

$$m^{orders} = \text{clamp}\left(\frac{D_{28}}{S_{28}},\ 0.95,\ 1.05\right) \tag{5}$$

Because v1 simulates forecasting (AIC-01), Equation (1) also includes a small random term ε_t, drawn from a standard normal distribution with a fixed seed. The same inputs therefore always give the same forecast (Equation 6). This noise models day-to-day variation for the simulation; a production forecasting model would not include it.

$$\varepsilon_t = 1 + \text{clamp}(0.03\,Z,\ -0.06,\ 0.06), \qquad Z \sim \mathcal{N}(0,1) \tag{6}$$

The manager sees each factor's influence as its average effect over the week (Equation 7), and the factors are listed from largest to smallest impact:

$$\text{impact}_m = \frac{1}{7}\sum_{d=1}^{7}(m_{r+d} - 1) \times 100\ \% \tag{7}$$

> **Example.** The last four Mondays at ST014 sold 4,200, 4,000, 4,400 and 4,100 L, so B = 4,175 L. For Monday 9 November, with season 1.00 (same month), rain 0.95, a local event (+8%), no holiday, orders 0.98 and noise 1.015, Equation (1) gives 4,175 × 1.00 × 0.95 × 1.08 × 1.00 × 0.98 × 1.015 ≈ **4,261 L**.

Figure 28 below shows ALG-01 on the sample data for ST014, Petrol 95. The solid blue line is the actual daily sales over six weeks; the weekly pattern is clear, with peaks on Fridays and Saturdays and a dip on Sundays. The dashed orange line is a one-day-ahead backtest over the last three weeks, meaning forecasts made each day with only the data known at the time; its Mean Absolute Percentage Error (MAPE), the average gap between forecast and actual sales, is 3.8%. After the reference date, the dashed purple line is the 7-day forecast and the shaded area its 80% confidence band from ALG-02, which widens as the forecast reaches further ahead.

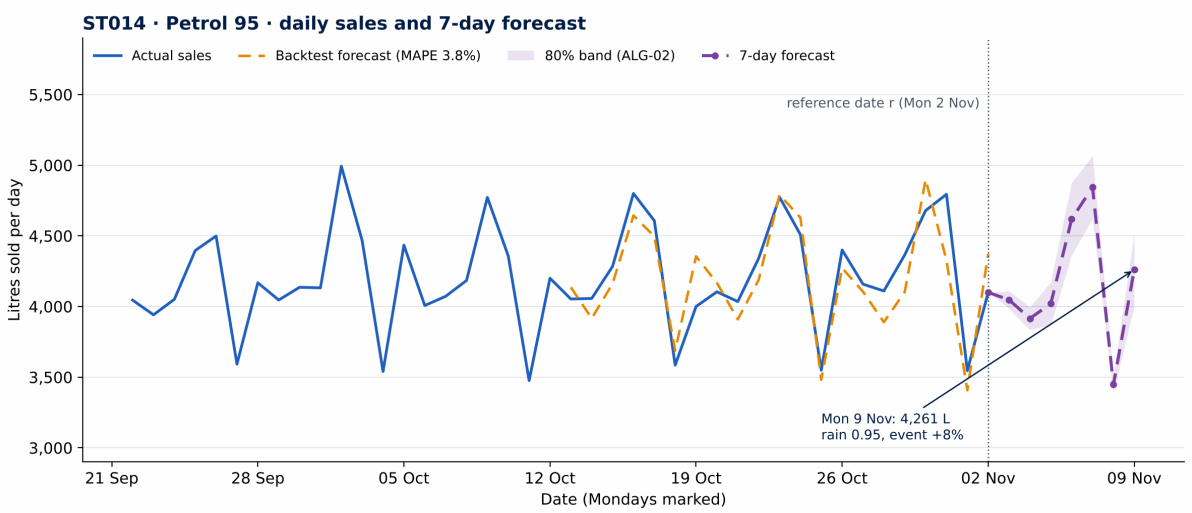


Figure 28 · ALG-01 forecast for ST014, Petrol 95

```
forecast(s, f, r):  # station, fuel, reference date
H = sales_history(s, f, last 365 days)
if days_with_data(H, last 28 days) < 14: H = similar_stations_history(s, f)  # fallback, flagged uncertain
rng = PRNG(hash(s, f, r, FORECAST_SEED))  # same inputs → same noise
for d in 1..7:
    t = r + d
    base = mean(H[t − 7k] for k in 1..4 if present)  # Equation (2)
    if fewer than 2 values: base = mean(H[last 28 days]); mark degraded
    litres[d] = max(0, round(base × m_season × m_weather × m_event × m_cal × m_orders × noise))
# Equation (1)
factors = impacts per Equation (7), sorted by |impact|
return litres, factors, model_version = "fc-sim-1.1.0"
```

5.9.2 ALG-02 Confidence and uncertainty

ALG-02 tells the manager how far each forecast from ALG-01 can be trusted. It produces three results: a confidence percentage for each forecast day, an 80% band around each forecast value, and an *uncertain* flag for the whole week. All three start from how much the station's sales vary from week to week.

That variability is measured with the **coefficient of variation** cv_t: the standard deviation of the same-weekday sales over the last eight weeks, divided by their mean (Equation 8). It has no unit, so a station selling 2,000 L a day and one selling 8,000 L can be compared directly. A cv_t of 0.08 means sales usually move about 8% around their average.

$$cv_t = \frac{s_t}{\bar{y}_t}, \qquad \bar{y}_t = \frac{1}{n}\sum_{k=1}^{8} y_{t-7k}\,, \qquad s_t = \sqrt{\frac{1}{n}\sum_{k=1}^{8}(y_{t-7k} - \bar{y}_t)^2} \tag{8}$$

The confidence of forecast day *d* starts at 95% and loses points for each source of doubt (Equation 9): the variability itself, the distance into the future (2 points per day), each missing input on that day M_d such as weather or events (12 points each), and a sales history with more than 5% missing days in the

last 28, g (10 points). The result is rounded and kept between 0 and 100. When ALG-01 had to use the similar-stations baseline, the confidence is capped at 60%, so the forecast is always flagged.

$$\text{conf}_d = \text{clamp}(\text{round}(95 - 100\,cv_t - 2(d-1) - 12\,M_d - 10 \cdot \mathbb{1}[\,g > 5\%\,]),\ 0,\ 100) \tag{9}$$

The band assumes the forecast error follows a normal distribution. Its standard deviation σ_d grows with the size of the forecast, its variability and the horizon (Equation 10). An 80% band leaves 10% of outcomes below it and 10% above, which corresponds to ±1.28 standard deviations, since $z_{0.90} = 1.28$ (Equation 11). The lower bound is never below zero.

$$\sigma_d = cv_t \cdot \hat{y}_t \cdot \big(1 + 0.05\,(d-1)\big) \tag{10}$$

$$\text{band}_d = [\max(0,\ \hat{y}_t - 1.28\,\sigma_d),\ \ \hat{y}_t + 1.28\,\sigma_d] \tag{11}$$

The week's overall confidence C is its weakest day, and the forecast is flagged uncertain when C falls below 70% (Equation 12, BR-27): 69% is flagged, 70% is not. An uncertain forecast is still used for planning; the flag only warns the manager and names the reason, such as the missing input.

$$C = \min_{d=1..7} \text{conf}_d, \qquad \text{uncertain} \Leftrightarrow C < 70\% \tag{12}$$

> **Example.** With cv_t = 0.08, day 1 scores 95 − 8 = **87%** and day 7 scores 95 − 8 − 12 = **75%**. If the weather is missing on day 5, that day drops to 95 − 8 − 8 − 12 = **67%**, so C = 67% and the week is flagged uncertain. For day 3 of ST014, ŷ = 4,021 L, so σ = 0.08 × 4,021 × 1.10 ≈ 354 L and the band is **3,568 L to 4,474 L**.

Figure 29 below shows both results for this example. On the left, confidence falls by two points per day, from 87% to 75%. The dashed line shows the same week with the weather missing on day 5: that single day drops into the shaded zone under 70%, which is enough to flag the whole forecast. On the right, the bell curve is the assumed spread of actual sales around the day-3 forecast; the shaded centre between −1.28σ and +1.28σ holds 80% of the likely outcomes, and this is the band drawn around the forecast in Figure 28.

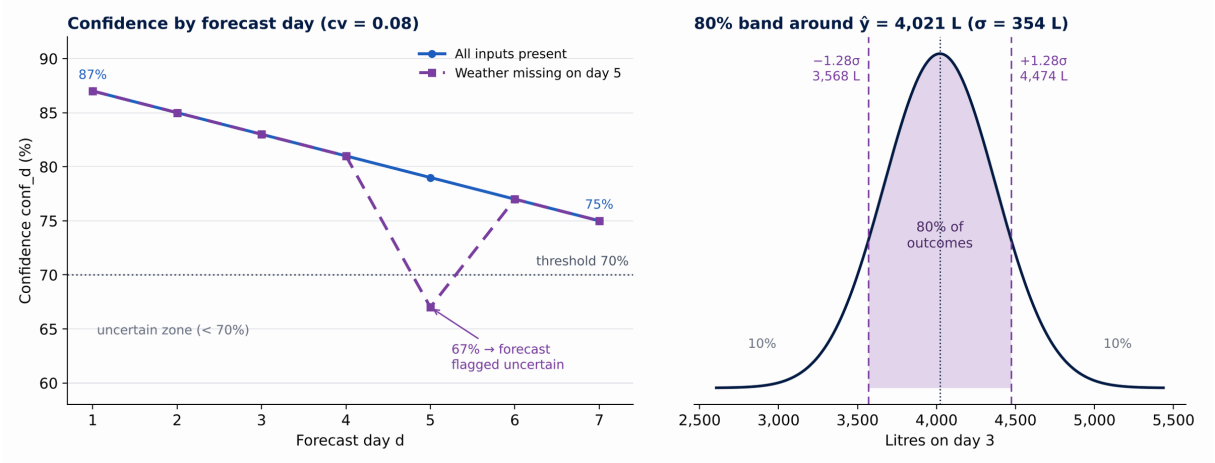


Figure 29 · ALG-02 confidence and 80% band

```
confidence(s, f, r, litres):
for d in 1..7:
    t = r + d
    cv = std(H_sameweekday(t, 8)) / mean(H_sameweekday(t, 8))   # Equation (8)
    conf[d] = clamp(round(95 − 100×cv − 2×(d−1) − 12×missing_inputs(d) − (gap_share > 5% ? 10 :
0)), 0, 100)  # (9)
    if fallback_baseline: conf[d] = min(conf[d], 60)
    sigma = cv × litres[d] × (1 + 0.05×(d−1))                    # (10)
    band[d] = [max(0, litres[d] − 1.28×sigma), litres[d] + 1.28×sigma]  # (11)
overall = min(conf[1..7]); uncertain = overall < 70              # (12), BR-27
return conf, band, overall, uncertain
```

5.9.3 ALG-03 Station risk, priority and quantity

ALG-03 turns the forecasts into decisions about each station and fuel: how soon its tank will run dry, how urgent it is compared with the others, how many litres it should receive, and which warehouse will supply them.

Time to stockout. Starting from the current level L_0 at the start of day 1, the tank is emptied day by day by the forecast demand $\hat{y}_d$, so its level after day d is L_d (Equation 13). The first day d^* on which the demand exceeds what is left is the stockout day. Inside that day, demand is assumed to be spread evenly over 24 hours, which gives the time to stockout h in hours (Equation 14). If the tank lasts the whole week, h is undefined and the station cannot run dry within the horizon.

$$L_d = L_0 - \sum_{i=1}^{d} \hat{y}_i\,, \qquad d^* = \min\{\, d : L_{d-1} \le \hat{y}_d \,\} \tag{13}$$

$$h = 24\,(d^* - 1) + 24 \cdot \frac{L_{d^*-1}}{\hat{y}_{d^*}} \tag{14}$$

Risk level. The risk is a threshold classification of h (Equation 15, BR-26): high under 24 hours, medium from 24 to 72 hours, low beyond. The boundaries are exact: 23 h is high, 24 h and 72 h are medium, 73 h is low. A station's map color is the highest risk among its fuel types (FR-47).

$$\text{risk}(h) = \begin{cases} \text{HIGH} & h < 24 \\ \text{MEDIUM} & 24 \le h \le 72 \\ \text{LOW} & h > 72 \text{ or no stockout} \end{cases} \tag{15}$$

Priority. Stations are ranked by a lexicographic sort, which compares the first key and only looks at the next one to break a tie (Equation 16): the risk level, then whether the station was left unserved the previous day (c = 0 if carried over), then the hours to stockout, then the larger quantity, the distance to its warehouse and finally the station ID. Because remoteness and volume come after risk and hours, they can never push a high-risk station below a lower-risk one (AI-16). A carried-over station's risk is recalculated from its new tank level, so it usually rises on its own; the flag only puts it first among stations with the same risk.

$$\text{key}(s) = (\,\text{risk},\ c,\ h,\ -q,\ \text{dist}(s,w),\ \text{id}\,) \tag{16}$$

Quantity. A station needs fuel when its projected level falls below its safety stock S on any of the seven days. The recommended quantity refills it to cover the week's demand plus the safety stock, but never more than the free space in the tank, $K - L_0$, where K is the capacity (BR-04). It is then rounded down to the nearest 100 L (Equation 17).

$$q = 100 \cdot \left\lfloor \frac{\min(S + \sum_{d=1}^{7} \hat{y}_d - L_0,\ \ K - L_0)}{100} \right\rfloor, \qquad q \geq 0 \tag{17}$$

Allocation. Each station is served by its nearest warehouse w^* (Equation 18). The warehouse's stock R is shared out greedily in priority order: each station receives what it asked for, or whatever is left (Equation 19). A station that receives less is *partly served*. If its nearest warehouse runs out of stock, the same rule is applied with the next nearest warehouse that still has stock, so a station stays unserved only when no warehouse can supply it.

$$w^*(s) = \text{argmin}_w \text{dist}(s, w) \tag{18}$$

$$g_i = \min(q_i,\ R_{i-1}), \qquad R_i = R_{i-1} - g_i, \qquad R_0 = \text{stock}(w, f) \tag{19}$$

> **Example.** ST014 holds L_0 = 9,500 L of Petrol 95. The forecasts are 4,047 L, then 3,915 L, so 5,453 L are left after day 1 and 1,538 L after day 2. Day 3 needs 4,021 L, so the tank runs dry on day 3, after h = 48 + 24 × 1,538 / 4,021 ≈ **57 h**: risk **medium**. The week needs 29,154 L; with a 4,000 L safety stock the raw need is 4,000 + 29,154 − 9,500 = 23,654 L, but only 20,000 − 9,500 = 10,500 L fit in the tank, so q = **10,500 L**.

Figure 30 below shows both steps. On the left, three stations empty their tanks at different speeds; the coloured zones are the risk thresholds, so ST032 runs dry after 18 hours (high), ST014 after 57 hours (medium), and ST007 lasts the whole week (low). On the right, warehouse W1 holds 30,000 L of Petrol 95 and serves its stations in priority order: ST032 and ST014 get their full quantity, ST021 gets the 7,500 L that remain out of 9,000 L, and ST007, last in line, is supplied by the next nearest warehouse, W2.

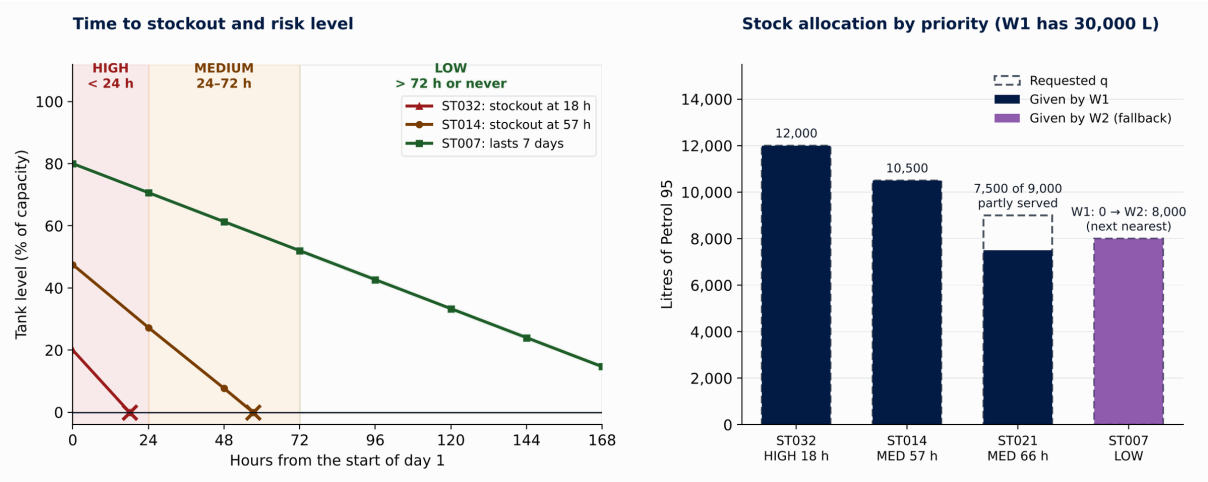


Figure 30 · ALG-03 stockout, risk and stock allocation

```
hoursToStockout(L, f[1..7]):          # L = level at the start of day 1
for d in 1..7:
    if L − f[d] ≤ 0: return 24×(d−1) + 24×L/f[d]   # Equations (13)–(14)
    L = L − f[d]
return null                                  # lasts the whole horizon
riskLevel(h): h < 24 → HIGH; h ≤ 72 → MEDIUM; else LOW   # (15), BR-26
priority key (ascending): (risk, carried_over ? 0 : 1, h (null last), −q, distance, station_id)
# (16)
quantity: q = floor(min(S + Σ f − L, K − L) / 100) × 100; q = max(q, 0)   # (17)
```

```
allocate(w, fuel): remaining = stock(w, fuel)        # (18)–(19)
    for station in priority order: give min(q, remaining); remaining −= given
    unmet stations → retry with the next nearest warehouse that has stock
    still unmet → unserved, reason recorded, carried over to the next day
```

5.9.4 ALG-04 Routing optimizer

ALG-04 builds the delivery plan. It solves a **Capacitated Vehicle Routing Problem with Time Windows (CVRPTW)**, extended with tanker compartments and second trips: given the stations to deliver, the litres for each, and the tankers and drivers available, find the set of routes that causes the fewest stockouts at the lowest cost. The road network is modelled as a **complete weighted graph** whose nodes are the warehouses and stations; each edge carries the distance d_{ij} in kilometres and the driving time τ_{ij} between two nodes. Both come from the distance matrix built by ALG-09 (Section 5.9.9). Each warehouse is solved separately, because its tankers only serve its own stations, plus the stations it takes over through the fallback of ALG-03.

Figure 31 below shows the ten steps of the algorithm, read left to right, top row then bottom row. Steps 1 to 6 prepare the problem using ALG-03; steps 7 and 8 are the optimization described in this section; step 9 completes each route; and step 10 is the fail-safe check: a plan that breaks any hard rule is withheld (ERR-SYS-02). Dashed boxes are exits. A station that still fits nowhere is listed as unserved with its reason, retried during the local search, and if still unserved, carried over to the next day's plan.

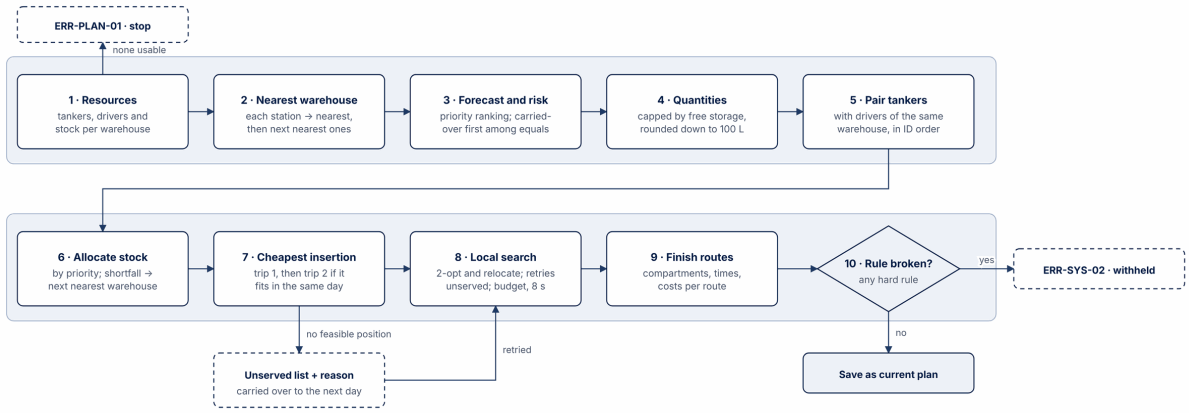


Figure 31 · ALG-04 Routing optimizer

Constraints of a trip. A trip leaves the warehouse, visits stations in order and returns. Its load per fuel must fit the tanker's compartments, each compartment holding one fuel and never more than its size (BR-31). Along the trip, the arrival time at the next station *j* is the departure from the previous station *i* plus the driving time, but never earlier than the station's opening time a_j (Equation 20), and it must not be later than its closing time b_j. Unloading takes u_i minutes, 15 minutes plus 3 minutes per 1,000 L delivered (Equation 21).

$$t_j = \max\left(a_j,\ t_i + u_i + \tau_{ij}\right), \qquad t_j \le b_j \tag{20}$$

$$u_i = 15 + 3 \cdot \frac{q_i}{1000} \ \text{min} \tag{21}$$

Second trip. A tanker may make a second trip on the same day when its first trip returns early enough. The second trip leaves after the first return plus a reload time τ_{load} of 30 minutes, and must be back before the depot closes at T_{close} = 22:00 (Equation 22). The depot opens at 05:00.

$$t_{dep}^{(2)} = t_{ret}^{(1)} + \tau_{load}, \qquad t_{ret}^{(2)} \le T_{close} \tag{22}$$

Objective. Two plans are compared lexicographically (Equation 23, FR-41): first on the number of projected stockouts, meaning stops reached after their stockout time plus high-risk stations left unserved; then on the priority left unserved, where each unserved station weighs more the higher it ranks among the *n* stations; and only then on the total route cost from ALG-05. Cost never outweighs a stockout.

$$\min \left(N_{stockout},\ \sum_{s \in U} (n + 1 - \text{rank}_s),\ \sum_{v} \text{cost}_v \right) \tag{23}$$

Construction: cheapest insertion. Stations are taken in priority order. Each one is inserted into the route and position where it adds the least distance, among the positions that respect every constraint. Inserting station *k* between *i* and *j* replaces the edge *i* → *j* by *i* → *k* → *j*, so its cost is given by Equation 24. Second trips are opened only for stations that fit nowhere on a first trip.

$$\Delta_{ins}(k;\ i, j) = d_{ik} + d_{kj} - d_{ij} \tag{24}$$

Improvement: local search. The routes are then improved with two kinds of moves. A **2-opt** move removes two edges of a route and reconnects it the other way, which removes crossings (Equation 25). A **relocate** move moves one station to another route of the same warehouse. A move is kept only if it is feasible and improves the objective. The moves are tried in a seeded random order, unserved stations are retried after each pass, and the search stops when no move improves the plan or after 200,000 evaluations, which is reached in about one second. An 8-second time limit is only a safety net.

$$\Delta_{2opt} = d_{ac} + d_{bd} - d_{ab} - d_{cd} \quad (\text{kept if} < 0) \tag{25}$$

> **Example.** Inserting station *k* between *i* and *j* adds 6.4 + 6.4 − 9.1 = **+3.8 km**, the cheapest position available, so *k* goes there. On another route, swapping the edges A→B and C→D for A→C and B→D removes a crossing and shortens the route from 47.3 km to 40.7 km, a gain of **6.6 km**. Tanker T-01 returns from its first trip at 11:40; with 30 minutes to reload it leaves again at 12:10, and its second trip ends at 17:55, before 22:00, so the second trip is allowed.

Figure 32 below illustrates the two moves and their effect. On the left, cheapest insertion places station *k* where it adds the fewest kilometres. In the middle, 2-opt removes a crossing. On the right, the curve comes from a real run of the local search on a 27-station test instance: the total distance falls in steps each time a move is accepted, from 483 km after insertion to 333 km, then stays flat when no move can improve it further. That flat end is a **local optimum**: the best plan reachable with these moves, not a guaranteed global optimum.

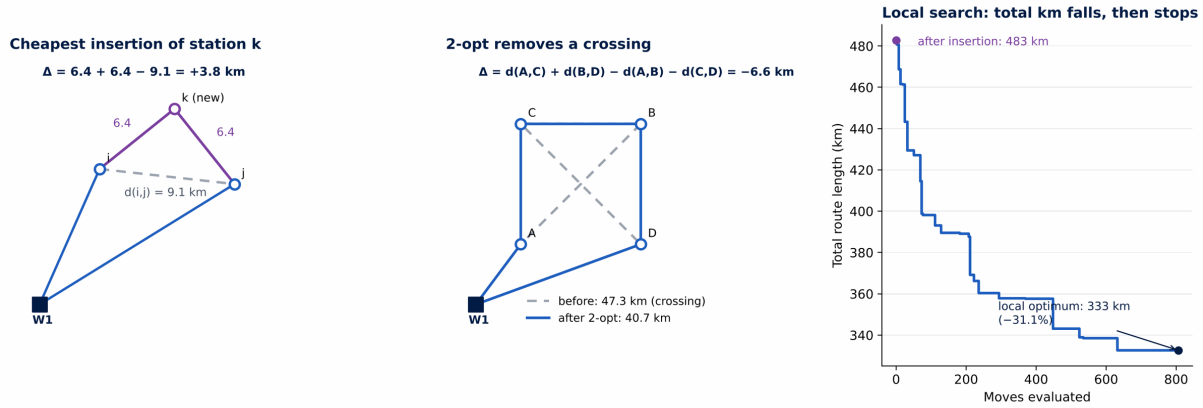


Figure 32 · ALG-04 insertion, 2-opt and local search

```
solve(problem, seed = ROUTING_SEED, maxEvals = 200 000, timeLimit = 8 s):
routes = one empty first trip per tanker–driver pair, in tanker ID order
for station in priority order:                                  # cheapest insertion, Equation (24)
    best = argmin over feasible (trip, position) of Δ_ins       # first trips; ties → lower route ID, position
    if none: best = argmin over feasible second trips (Equation 22)
    if none: unserved += (station, reason = first failing check) else insert at best
rng = PRNG(seed)
repeat until no improvement or budget used:                     # local search, first improvement
    for move in shuffle(rng, 2-opt moves ∪ relocate moves):
        if feasible(after) and objective(after) < objective(before): apply   # (23), (25)
    try to insert unserved stations again
if the time limit stopped the search: plan.time_limited = true
assign compartments; compute times (20)–(22) and costs (ALG-05)
return routes, unserved                                         # unserved → carried over to the next day
```

5.9.5 ALG-05 Cost model

ALG-05 gives every route and every plan a fuel cost in US dollars, so the manager can compare plans and see what an edit costs. It counts only the fuel the tankers burn; driver wages, tolls and wear are not part of v1. The cost of a route *v* is its total length D_v in kilometres, return leg to the warehouse included (BR-25), multiplied by the fuel consumption of its tanker c_v in litres per kilometre and by the fuel price p in dollars per litre (Equation 26, BR-16). The price is a parameter read at the moment the plan is created, so a later price change never alters the cost of an existing plan (D-20).

$$\text{cost}_v = D_v \cdot c_v \cdot p, \qquad D_v = \sum_{(i,j) \in v} d_{ij} \tag{26}$$

The cost of a plan is the sum of its route costs (Equation 27, FR-42). When the manager edits a plan, it is compared with the optimal plan of the same day in dollars and as a percentage (Equations 28 and 29, FR-43); a positive difference means the edited plan is more expensive.

$$\text{Cost}(P) = \sum_{v \in P} \text{cost}_v \tag{27}$$

$$\Delta = \text{Cost}(P_{edited}) - \text{Cost}(P_{optimal}) \tag{28}$$

$$\Delta\% = \text{round}\left(\frac{\Delta}{\text{Cost}(P_{optimal})} \times 100,\ 1\right) \tag{29}$$

Costs are computed and stored with three decimals and shown with two. Rounding is half-even (to the nearest even digit on a tie), so the totals shown always equal the sum of the route costs shown, to the cent (AC-16). If the price or a tanker's consumption is missing, the cost is left empty, the plan is flagged ERR-DATA-03, and the route is still planned on distance alone.

> **Example.** Route R-02 is 96.8 km long with tanker T-02, which uses 0.36 L/km, at a price of $1.90/L: cost = 96.8 × 0.36 × 1.90 = **$66.21**. The optimal plan for the day costs $388.10. After the manager moves two stations, routes R-02 and R-04 get longer and the edited plan costs $412.35, so Δ = **+$24.25** and Δ% = 24.25 / 388.10 × 100 = **+6.2%**.

Figure 33 below shows both sides of the model. On the left, the cost of a route is a straight line through zero: every extra kilometre adds the same amount, and the slope is the tanker's consumption times the price, so a tanker using 0.42 L/km costs 40% more per kilometre than one using 0.30 L/km. On the right, the comparison panel for the example: each route of the edited plan next to the same route in the optimal plan. Only the routes touched by the edit change, and their differences add up to the +$24.25 shown to the manager.

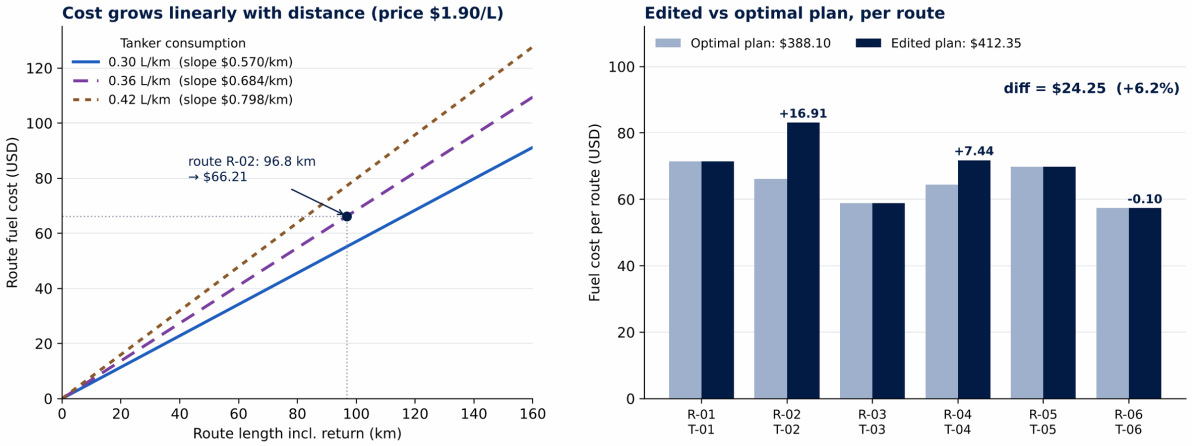


Figure 33 · ALG-05 route cost and plan comparison

```
routeCost(route) = route.km_total × tanker.consumption_l_per_km × price   # Equation (26), BR-16; km includes the return leg
price = parameter("fuel_price_usd_per_l", valid at plan creation)          # USD (D-20)
planCost = Σ routeCost                                                     # (27), FR-42
diff = planCost(edited) − planCost(optimal)                                # (28), FR-43
diff_pct = round(diff / planCost(optimal) × 100, 1)                        # (29), positive = more expensive
```

5.9.6 ALG-06 Edit recalculation and risk analysis

ALG-06 runs every time the manager reorders a route or moves a station to another tanker. It does two things: it recalculates the edited plan, and it compares it with the optimal plan to list the risks the edit

creates. A reorder or a move never re-optimizes the plan (BR-12); only the routes touched by the edit are recomputed, using the time equations of ALG-04 (20 and 21) and the cost of ALG-05 (26). This keeps every edit far under the one-second budget (NFR-01). The first edit copies the optimal plan into an edited plan, so the optimal plan stays the reference for every comparison.

Each risk compares a stop or a total in the edited plan with the same stop or total in the optimal plan. For a stop s, a_s is its arrival time in the edited plan, a_s^o its arrival time in the optimal plan, and T_s^{so} its stockout time, which is the start of day 1 plus the hours to stockout from ALG-03. Only the risks **caused** by the edit are reported: a problem that already existed in the optimal plan is not reported again.

RISK-01, capacity. A route whose load exceeds its tanker's capacity K_v, or whose fuels cannot be split into its compartments, is a high risk and blocks approval until it is fixed (Equation 30, BR-21).

$$\text{RISK-01}(v) \Leftrightarrow \text{load}_v > K_v \quad \vee \quad \text{no valid compartment assignment} \tag{30}$$

RISK-02 and RISK-03, stockout and window. A stop is a high risk when the edit makes it arrive after the station runs dry, or outside its delivery window $[a_s^{open}, b_s]$, while the optimal plan did not (Equations 31 and 32).

$$\text{RISK-02}(s) \Leftrightarrow a_s > T_s^{so} \ \wedge \ a_s^o \le T_s^{so} \tag{31}$$

$$\text{RISK-03}(s) \Leftrightarrow a_s \notin [a_s^{open},\ b_s] \ \wedge \ a_s^o \in [a_s^{open},\ b_s] \tag{32}$$

RISK-04, late arrival. The lateness ℓ_s is how many minutes later the stop is reached than in the optimal plan. Its severity is a step function: up to 60 minutes is low, up to 180 minutes medium, beyond that high (Equation 33).

$$\ell_s = a_s - a_s^o, \qquad \text{sev}(\ell_s) = \begin{cases} \text{LOW} & 0 < \ell_s \le 60 \\ \text{MEDIUM} & 60 < \ell_s \le 180 \\ \text{HIGH} & \ell_s > 180 \end{cases} \tag{33}$$

RISK-05, 06 and 07, distance, time and cost. For each measure *X* (kilometres, minutes, cost), the increase of the edited plan over the optimal plan is computed as a percentage and graded the same way: under 10% is low, from 10% to 25% medium, above 25% high (Equation 34). A decrease is not a risk.

$$\Delta X\% = \frac{X_{ed} - X_{opt}}{X_{opt}} \times 100, \qquad \text{sev} = \begin{cases} \text{LOW} & 0 < \Delta X\% < 10 \\ \text{MEDIUM} & 10 \le \Delta X\% \le 25 \\ \text{HIGH} & \Delta X\% > 25 \end{cases} \tag{34}$$

The boundaries follow the SRS exactly (FR-63): 60 minutes is low and 61 medium, 180 is medium and 181 high; 10% and 25% are both medium, 25.1% is high. Each risk lists the stops, segments and routes it affects (FR-65) and a sentence from the template catalogue (FR-64). An edit that creates no risk shows: *"No risk: this change keeps all deliveries on time and within limits."* (FR-66).

> **Example.** The manager moves station S-07 to tanker T-02, in position 2. S-07 was reached at 10:20 in the optimal plan and is now reached at 12:05, so ℓ = 105 min: RISK-04 **medium**. It is still inside its window and before its stockout time, so there is no RISK-02 or RISK-03. The plan grows from 612.4 km to 641.0 km (+4.7%), from 1,490 to 1,561 minutes (+4.8%) and from $388.10 to $412.35 (+6.2%): RISK-05, 06 and 07, all **low**. T-02 stays within its capacity, so approval is not blocked.

Figure 34 below shows the two severity scales as step functions. A filled dot means the boundary value belongs to that level, an open dot that it does not: 60 minutes is still low, while 10% is already medium. The diamonds are the example edit: a 105-minute delay falls in the medium band, and the three percentage increases all fall in the low band.

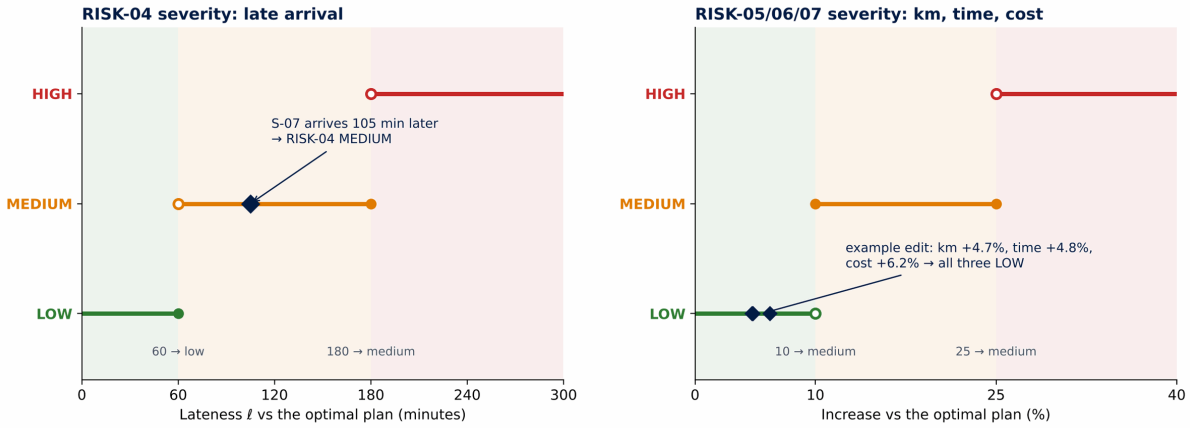


Figure 34 · ALG-06 risk severity scales

```
applyEdit(plan, edit):
edited = plan.kind == OPTIMAL ? copyAsEdited(plan) : plan
apply edit to edited.routes                    # reorder | move (same warehouse only, else
WARN-EDIT-01)
for route in affected routes: recompute times (20)–(21), km, minutes, litres, cost (26)
return edited, analyse(optimal = parentOf(edited), edited)
analyse(opt, ed):
for route in ed.routes: if load > capacity or no compartment assignment: RISK-01 HIGH, blocks
approval   # (30)
for stop in ed.stops (o = same station in opt):
    if a > T_so and not (o and a_o > T_so): RISK-02 HIGH                          # (31)
    if a outside window and not (o and a_o outside window): RISK-03 HIGH             # (32)
    late = a − a_o; if late > 0: RISK-04 with sev(late)                                # (33)
for (X, type) in [(km, RISK-05), (minutes, RISK-06), (cost, RISK-07)]:
    pct = (ed.X − opt.X) / opt.X × 100; if pct > 0: type with sev(pct)                    # (34)
no risk → TPL "No risk: this change keeps all deliveries on time and within limits."   # FR-66
```

5.9.7 ALG-07 Maximum quantity check

ALG-07 gives the largest quantity the manager may set for one fuel *f* at one stop *s* when editing a plan (BR-28). Three things limit it: the space left on the tanker trip that serves the stop, the free space in the station's tank, and the stock left at the warehouse that supplies it. The maximum is the smallest of the three, and the limit that sets it is shown to the manager. The browser asks for this value when the edit panel opens and never computes it itself; on Save, the server runs the same calculation again inside the edit transaction.

Tanker space. The fuels on a trip share the tanker's compartments, each compartment holding one fuel. The other fuels $g \neq f$ on the trip must keep enough compartments for their loads. Among all assignments *A* of compartments to fuels that do so, ALG-07 takes the one that leaves the most room for *f*, and subtracts what the other stops of the same trip already carry of *f* (Equation 35). With at most 6 compartments and 2 fuels, there are at most $2^6 = 64$ assignments, so all of them are checked.

$$F_{tanker} = \max_{A \text{ valid}} \sum_{c \in A^{-1}(f)} \text{size}(c) \; - \sum_{s' \neq s} q_{s',f} \tag{35}$$

Tank space. The station's tank can take at most its capacity K minus its current level L_0 (Equation 36, BR-04).

$$F_{tank} = K - L_0 \tag{36}$$

Warehouse stock. The warehouse w that serves the stop, which is the nearest one or the fallback warehouse chosen by ALG-03, can give at most its stock $R_{w,f}$ minus what it already delivers to every other stop (Equation 37, BR-08).

$$F_{wh} = R_{w,f} - \sum_{s' \neq s \text{ served by } w} q_{s',f} \tag{37}$$

Maximum. The maximum quantity is the smallest of the three, never below zero, and the binding limit is the one equal to it. When two limits are equal, the order tanker, tank, warehouse decides which one is named (Equation 38). A value above the maximum is refused with ERR-VAL-05a (tanker), 05b (tank) or 05c (warehouse) (FR-76).

$$q_{s,f}^{max} = \max\left(0,\ \min(F_{tanker},\ F_{tank},\ F_{wh})\right) \tag{38}$$

> **Example.** Tanker T-01 holds 36,000 L and already carries 28,000 L, so F_{tanker} = 8,000 L (FR-75). ST014's Petrol 95 tank has 10,500 L of free space and warehouse W1 has 12,400 L of Petrol 95 left. The maximum is min(8,000, 10,500, 12,400) = **8,000 L**, set by the **tanker**. For Diesel at ST021 on trip T-03, the tank is the tightest limit: min(9,500, 6,300, 14,000) = **6,300 L**, set by the **tank**.

Figure 35 below shows the three limits for both examples. Each bar is the room one limit leaves; the dark bar is the shortest one, which sets the maximum shown in the edit panel, and the dashed line marks that maximum. For ST014 the tanker is the bottleneck; for ST021 it is the station's own tank.

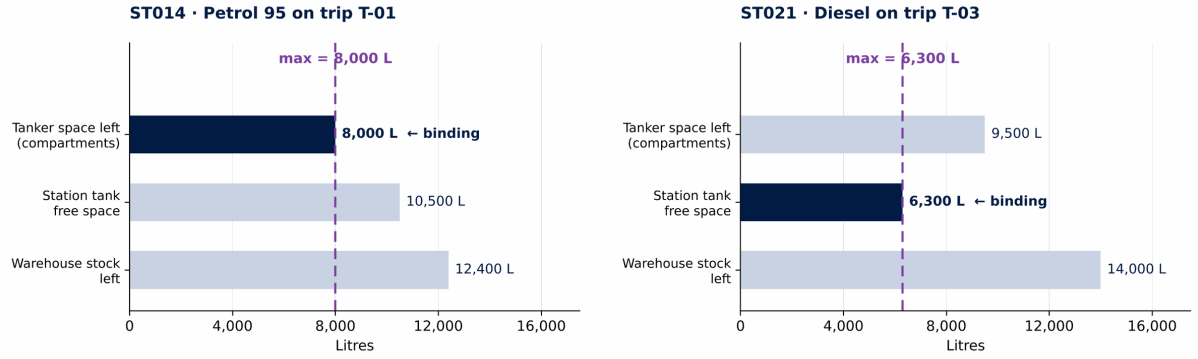


Figure 35 · ALG-07 maximum quantity and binding limit

```
maxQuantity(plan, stop, fuel f):                                  # BR-28
trip = trip of stop; w = warehouse serving stop                     # nearest or fallback (ALG-03)
tankerFree = max over valid compartment assignments of room(f) − Σ f on the other stops of trip  # (35)
```

```
tankFree = tank.capacity − tank.current_level                          # (36), BR-04
whFree = stock(w, f) − Σ final litres of f at every other stop served by w   # (37), BR-08
max = max(0, min(tankerFree, tankFree, whFree))                        # (38)
binding = first of [tanker, tank, warehouse] equal to max
return max, binding, hint (TPL-LIM-01..03)
```

5.9.8 ALG-08 Template explanations

No text is generated (D-11). ExplanationModule holds a fixed catalogue of sentences with typed blanks; a blank can only be filled from a value object passed by the caller, formatted by one shared formatter (litres "7,200 L", hours "14 hours", percent "+8%", date "05 Nov 2026", time "14:30").

```
fill(templateId, values):
  t = catalog[templateId]
  for blank in t.blanks:
    if blank.name not in values or values[blank.name] is null:
      metric template_fill_failures += 1   (OBS-M6)
      return fill(t.fallback, values restricted to known blanks)    # ERR-AI-03: shorter, never
invented
    text = text.replace(blank, format(blank.type, values[blank.name]))
  return text
```

Template	Sentence (blanks in braces)	Used by
TPL-FC-01	Recent {fuel} sales average {litres} per day on {weekday}s.	FR-22
TPL-FC-02	{month} demand is usually {pct} {higher_lower} than average.	FR-22
TPL-FC-03	{condition} weather ({temp} °C) on {date} {raises_lowers} demand by {pct}.	FR-22
TPL-FC-04	{event} on {date} raises expected demand by {pct}.	FR-22
TPL-FC-05	{holiday} on {date} changes demand by {pct}.	FR-22
TPL-FC-06	Deliveries in the last 28 days were {pct} {above_below} sales.	FR-22, D-28
TPL-FC-07	Current stock ({litres}) runs out on {date} if no delivery arrives.	FR-22
TPL-FC-08	Confidence is {pct}: {missing_input} is missing for {days} of 7 days.	FR-23, AI-20
TPL-FC-09	Little sales history for {station} {fuel}: forecast based on similar stations, confidence {pct}.	FR-23, FR-83

Template	Sentence (blanks in braces)	Used by
TPL-RISK-H/M/L	{station} is {level} risk: current {fuel} stock covers {hours} hours of forecast demand. (low: lasts more than 3 days)	FR-27
TPL-PRI-01	Priority {rank}: {level} risk, {hours} hours to stockout, needs {litres}.	FR-25
TPL-UNS-01	{station} is not served today: {reason}. It moves up in tomorrow's plan.	FR-38
TPL-FB-01	{station} gets {fuel} from {warehouse} today: {nearest} has none left.	ALG-03 fallback
TPL-TRIP-01	Plan for {date} decided: {n} trips to record.	UI-04 Trips tab
TPL-TRIP-02	Tanker {tanker} makes a second trip, leaving at {time} after reloading.	ALG-04, BR-23
TPL-TRIP-03	Trip {trip} cancelled: {reason}.	SM-03
TPL-EDIT-01	Tanker {tanker} is overloaded: {load} on a {capacity} tanker. Remove {excess} before approving.	RISK-01, FR-77
TPL-EDIT-02	{station} may run out of {fuel} {minutes} min before the tanker arrives.	RISK-02
TPL-EDIT-03	{station} would be reached at {time}, outside its {open}–{close} window.	RISK-03
TPL-EDIT-04	This change makes {station} arrive {minutes} min later than planned.	RISK-04
TPL-EDIT-05/06/07	This change adds {amount} ({pct}) of {distance_time_cost} to the plan.	RISK-05–07
TPL-WHY-01	The recommended plan was {usd} cheaper and {km} shorter.	FR-64 "why not optimal"
TPL-LIM-01..03	Max for this stop: {max}. {limit_detail}.	FR-75

All sentences are under 25 words and use no technical terms (NFR-09). The catalogue has a version (tpl-1.1.0) stored with each plan. TC-AI-04b parses every number and name in generated sentences and checks it against the data (groundedness 100%).

5.9.9 ALG-09 Distance matrix (Floyd–Warshall)

ALG-09 produces the distances and driving times that ALG-03 and ALG-04 rely on. Stations and warehouses are not linked by straight lines but by roads, so the distance between two points is the

length of the **shortest road path** between them. ALG-09 computes that path for every pair of points at once with the **Floyd–Warshall algorithm** and stores the result as the distance matrix read by the distance provider (IF-DIST). In v1 the road network itself is simulated; a later version can replace it with real road data without changing ALG-03 or ALG-04.

The road graph. The network is a weighted graph $G = (V, E)$: the nodes *V* are the 3 warehouses and 20 stations, and each edge in *E* is a road between two of them, weighted by its length w_{ij} in kilometres. Each road also has a driving time, from its length and a typical speed v_{ij} (Equation 39): 70 km/h for roads longer than 12 km, 50 km/h otherwise. Before the algorithm runs, a node is at distance 0 from itself, at distance w_{ij} from its direct neighbours, and at an infinite distance from every other node (Equation 40).

$$\tau_{ij}^{road} = 60 \cdot \frac{w_{ij}}{v_{ij}} \text{ min} \tag{39}$$

$$d_{ij}^{(0)} = \begin{cases} 0 & i = j \\ w_{ij} & (i,j) \in E \\ \infty & \text{otherwise} \end{cases} \tag{40}$$

The recurrence. Floyd–Warshall then lets every node *k*, one after another, act as a possible stopover. For each pair (*i*, *j*), if going from *i* to *k* and then from *k* to *j* is shorter than the best path known so far, that path is replaced (Equation 41). This update is called a **relaxation**. After all nodes have been tried as stopovers, d_{ij} is the shortest road distance between every pair. The algorithm also records the first step of each shortest path, so the path can be rebuilt and its driving time τ_{ij} is the sum of the driving times of its roads (Equation 42).

$$d_{ij}^{(k)} = \min\left(d_{ij}^{(k-1)},\ d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\right), \qquad k = 1, \ldots, |V| \tag{41}$$

$$\tau_{ij} = \sum_{(a,b) \in \text{path}(i,j)} \tau_{ab}^{road} \tag{42}$$

Storage. The matrix is recomputed whenever a station is added or moved, instead of at each plan. The result is 23 × 22 = 506 pairs, the content of the distance_entry table. Roads are two-way, so $d_{ij} = d_{ji}$.

> **Example.** On a road through *k*, the distance from *i* to *j* is 5.2 + 6.1 = 11.3 km, shorter than the direct road of 14.0 km, so d_{ij} becomes 11.3 km and the path goes via *k*. On the sample network, the shortest road from W1 to ST008 is W1 → ST005 → ST003 → ST001 → ST013 → ST004 → ST008: **124.6 km and 110 min**, compared with 86.3 km in a straight line.

Figure 36 below shows ALG-09 on the sample data. On the left, the simulated road network: grey lines are roads, squares are warehouses and circles are stations. The purple line is the shortest road from W1 to ST008 found by the algorithm, and the dashed line is the straight-line distance it replaces. In the middle, a single relaxation step: the path through *k* is shorter than the direct road, so it becomes the new best distance. On the right, the resulting distance matrix as a heat map: each cell is the shortest road distance between a row point and a column point, darker meaning further. The white diagonal is each point's distance to itself, and the matrix is symmetric because roads are two-way.

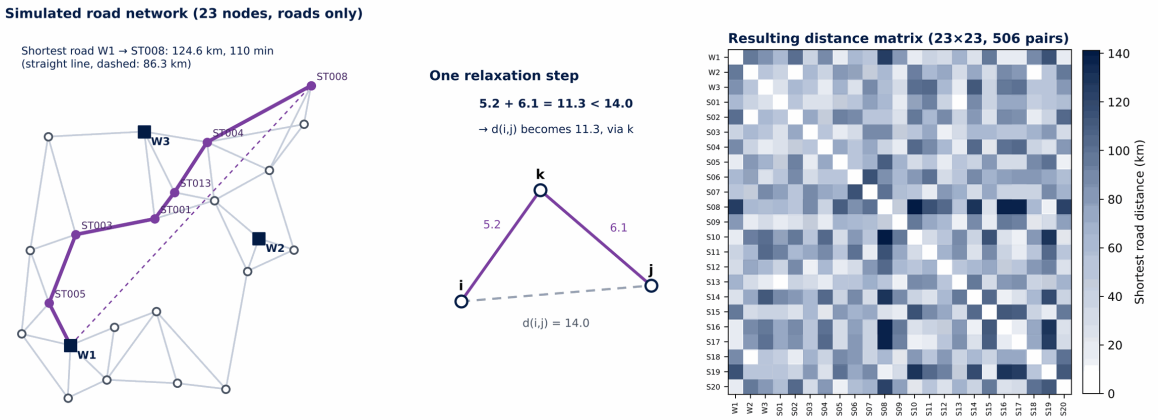


Figure 36 · ALG-09 road network and distance matrix

```
buildDistanceMatrix(V, E):                    # V = warehouses + stations, E = roads
for i, j in V × V: d[i][j] = (i == j) ? 0 : ∞;  next[i][j] = null       # Equation (40)
for (i, j, km) in E: d[i][j] = d[j][i] = km;  t[i][j] = t[j][i] = 60×km/speed(km)   # (39)
for k in V:                                                  # every node as a stopover
    for i in V:
        for j in V:
            if d[i][k] + d[k][j] < d[i][j]:                          # relaxation, Equation (41)
                d[i][j] = d[i][k] + d[k][j];  t[i][j] = t[i][k] + t[k][j];  next[i][j] =
next[i][k]
if any d[i][j] = ∞: flag the unreachable station (ERR-DATA-02)
store all pairs i ≠ j in distance_entry (km, minutes)               # 506 pairs, read by IF-DIST
```

5.9.10 Complexity analysis

Complexity describes how the work an algorithm does grows as the network grows. It is written in Big O notation, which keeps only the fastest-growing part: $O(n)$ means the work grows in step with n, while $O(n^3)$ means that doubling n multiplies the work by eight. The analysis uses the following symbols: n is the number of stations, $h = 365$ the days of sales history read, $d = 7$ the forecast days, $W = 3$ the warehouses, and $V = n + W$ the points of the road network. The number of fuel types and of tanker compartments (at most 6) are small fixed values and count as constants. Each algorithm is measured over one run as Venzini uses it: the whole network for ALG-01 to ALG-06 and ALG-09, one edit panel for ALG-07 and one sentence for ALG-08.

ALG-01 Forecasting. Each station and fuel reads its sales history once and then does a fixed amount of work for each of the 7 days, so the network forecast takes $O(n \cdot h)$ time, which is $O(n)$ because h is fixed. The same-region averages used for stations with little history are computed once per run and shared, so the fallback does not change this. Memory is $O(n \cdot h)$ for the history read.

ALG-02 Confidence and uncertainty. For each day, the confidence uses the same weekday of the last 8 weeks, a fixed number of values, so the whole network takes $O(n \cdot d) = O(n)$ time and $O(n)$ memory.

ALG-03 Station risk, priority and quantity. The time to stockout, the risk level and the quantity cost a fixed amount per station. The priority order is a sort, which takes $O(n\log n)$, and the allocation then walks that order once per warehouse, $O(n \cdot W)$. The sort dominates, so ALG-03 runs in $O(n\log n)$ time with $O(n)$ memory.

ALG-04 Routing optimizer. Cheapest insertion places each of the n stations by trying every position on every route, $O(n)$ positions, and checks each candidate by recomputing the arrival times along the route, another $O(n)$; construction therefore takes $O(n^3)$. One pass of local search tries $O(n^2)$ 2-opt and relocate moves, each checked in $O(n)$, which is again $O(n^3)$ per pass. The number of passes is not fixed in advance, which is why the search stops after 200,000 evaluations or 8 seconds (ADR-12). Memory is $O(n^2)$, mostly the distance matrix it reads. ALG-04 also runs for every re-plan, after a quantity edit or a rejection.

ALG-05 Cost model. Every stop belongs to exactly one route, so adding up all route lengths and costs touches each stop once: $O(n)$ time. Comparing two plans (Equations 28 and 29) is $O(1)$. Memory is $O(1)$ beyond the plan itself.

ALG-06 Edit recalculation and risk analysis. An edit recomputes only the routes it touches, at most $O(n)$ stops, and compares each stop with the same stop in the optimal plan, found by its station ID in $O(1)$. The compartment check of RISK-01 tries a fixed, small number of assignments. ALG-06 therefore runs in $O(n)$ time and memory, and never re-optimizes (BR-12).

ALG-07 Maximum quantity check. The tanker limit tries a fixed number of compartment assignments, and the tank limit is one subtraction. The warehouse limit adds up what that warehouse already delivers to its other stops, which can be up to $O(n)$ stops, so one check takes $O(n)$ time and $O(1)$ memory.

ALG-08 Template explanations. Filling a sentence replaces a handful of blanks from a fixed catalogue, so each sentence takes $O(1)$ time and memory, whatever the size of the network.

ALG-09 Distance matrix (Floyd–Warshall). The three nested loops over the V points of the road network make Floyd–Warshall run in $O(V^3)$ time, and the matrix takes $O(V^2)$ memory. Since $V = n + 3$, this is $O(n^3)$. With 23 points that is 23^3 = 12,167 relaxations, which takes a few milliseconds, and it runs only when a station is added or moved, not at each plan.

Venzini as a whole. A plan runs the algorithms one after the other, and a chain of steps is only as fast as its slowest step. The complexity of Venzini as a whole is therefore $O(n^3)$, set by ALG-04 and ALG-09, with $O(n^2)$ memory for the distance matrix. This cubic growth is harmless at the size of v1: with 20 stations a cubic step stays in the order of ten thousand operations, which a laptop completes in a few milliseconds. The design also keeps both expensive algorithms under control as the network grows: ALG-04 stops on its evaluation budget, which keeps a full plan within the 10 seconds required by NFR-19, and ALG-09 is computed once, cached, and rebuilt only when a station changes.

Figure 37 below shows these growth rates. Panel (a) draws one dashed curve per complexity class, ending with the algorithms that belong to it; panel (b) draws the complexity of Venzini as a whole. The dotted line marks the 20 stations of v1. The curves show theoretical growth, not measured run times.

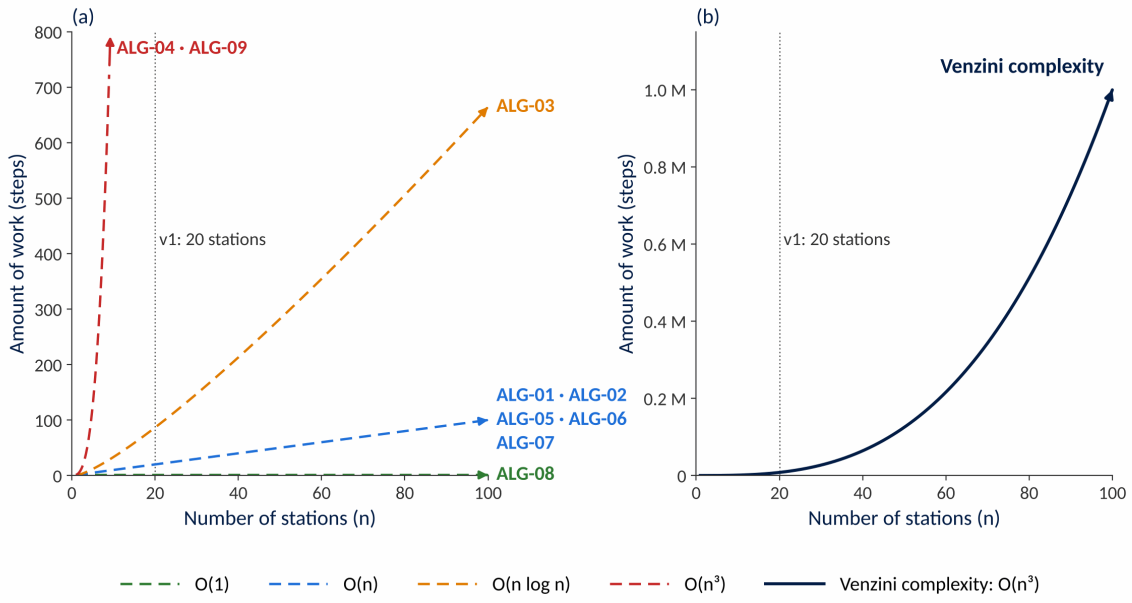


Figure 37 · Algorithm complexity

5.9.11 Asymptotic analysis

Section 5.9.10 gives each algorithm its Big O, which is an upper bound. Asymptotic analysis adds two more notations, so that the best case and the exact growth are stated as well. For a work count $f(n)$ and a growth function $g(n)$, there must be positive constants c, c_1, c_2 and a size n_0 beyond which the bound always holds (Equations 43 to 45):

$$f(n) = O\big(g(n)\big) \Leftrightarrow f(n) \leq c \cdot g(n) \quad \text{for all } n \geq n_0 \tag{43}$$

$$f(n) = \Omega\big(g(n)\big) \Leftrightarrow f(n) \geq c \cdot g(n) \quad \text{for all } n \geq n_0 \tag{44}$$

$$f(n) = \Theta\big(g(n)\big) \Leftrightarrow c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \quad \text{for all } n \geq n_0 \tag{45}$$

Big O (O) is an upper bound: the work never grows faster than g, which describes the worst case. Big Omega (Ω) is a lower bound: the work always grows at least as fast as g, which describes the best case. Big Theta (Θ) holds when both bounds use the same g: the best and worst cases grow alike, so the growth is known exactly.

Applied to the algorithms of Venzini, these notations sort them into four groups. ALG-01, ALG-02 and ALG-05 are $\Theta(n)$: they always make one pass over every station, whatever the data. ALG-08 is $\Theta(1)$: filling a sentence never depends on the size of the network. ALG-06 and ALG-07 are $\Omega(1)$ and $O(n)$: a small edit on a short route costs almost nothing, while in the worst case the edited routes or the warehouse's deliveries reach every stop. ALG-03 is $\Omega(n)$ and $O(n\log n)$: it must look at every station once, and its sort, the built-in sort of Node.js (TimSort), takes about n steps when the stations are already nearly in priority order and $n\log n$ otherwise. ALG-09 is $\Theta(n^3)$: its three loops always run in full, and with $V = n + 3$ its exact count $(n + 3)^3$ stays between n^3 and $2n^3$ from $n_0 = 12$ stations on. ALG-04 is $\Omega(n^2)$ and $O(n^3)$, and because it sets the bounds of Venzini as a whole, it is shown with the system below.

Figure 38 below shows the four groups. Each panel draws the upper bound in red and the lower bound in green. Where the growth is exact, in panels (a) and (d), the real work is a solid curve held between the two bounds from n_0 onwards; ALG-08 in panel (b) is the flat line of $\Theta(1)$. Where the best and worst cases differ, in panels (b) and (c), the shaded area is the range in which the real cost lies, depending on the data.

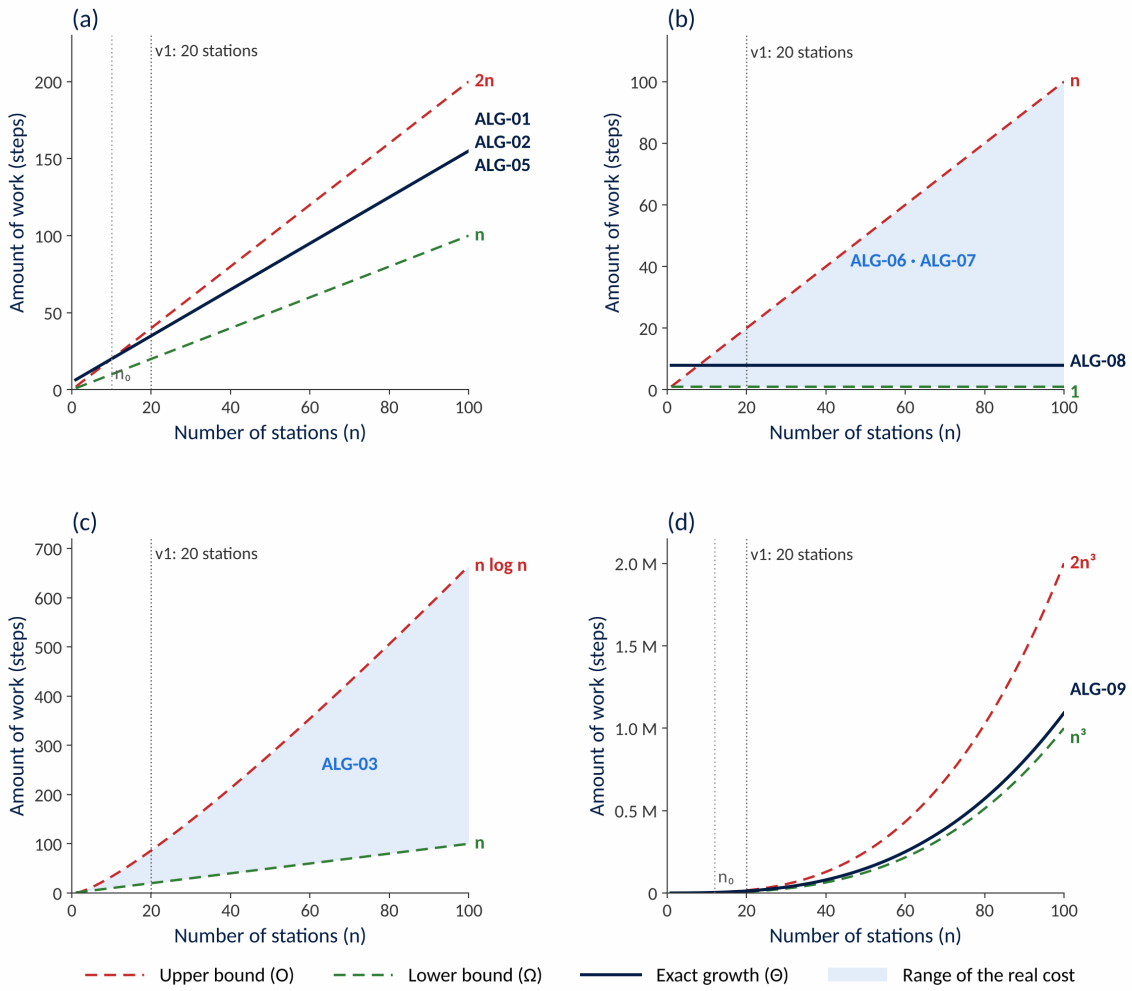


Figure 38 · Asymptotic bounds per algorithm

Venzini as a whole follows the same rules. When steps run one after the other, the overall upper bound is the largest upper bound of the steps, and the overall lower bound is the largest lower bound. For one plan, the distance matrix of ALG-09 is already cached, and ALG-04 is the most demanding step at both ends: even in its best case, cheapest insertion must try every position for every station, which takes $\Omega(n^2)$, and in the worst case every check walks the whole route, which takes $O(n^3)$. Generating a plan is therefore $\Omega(n^2)$ and $O(n^3)$, with no single Θ: its real cost depends on the stations and their constraints. When a station is added or moved, the run also rebuilds the distance matrix, which is always cubic, so that run is $\Theta(n^3)$.

Figure 39 below shows the bounds of one plan. The red curve is the upper bound, the green curve the lower bound, and the shaded area between them the range in which the cost of a plan lies. At the 20 stations of v1, both bounds stay below ten thousand steps.

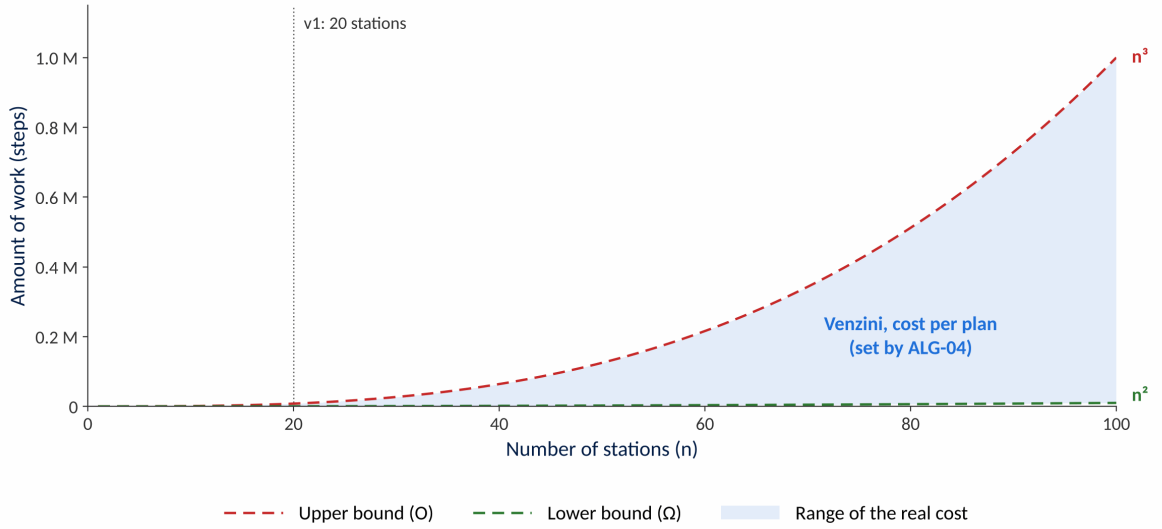


Figure 39 · Asymptotic bounds of Venzini

5.10 Resource view

The reference data set is small (20 stations, 6 tankers, 506 distance pairs), so the design meets every performance target on one laptop if two things hold: the optimizer runs off the request thread, and edits never re-optimize. All budgets are measured on the SRS reference environment (4 cores, 8 GB RAM, latest Chrome).

5.10.1 Performance budgets

NFR	Target	Budget split (design estimate, p95)	How the design meets it
NFR-01	Edit recalculated and shown ≤ 1 s (95%), ≤ 2 s (100%)	Request and lock 30 ms · recalculation of affected routes ≤ 10 ms · risk analysis ≤ 10 ms · save 40 ms · response 10 ms · map render 150 ms ≈ 250 ms	Recalculation only (BR-12); only affected routes; distance matrix in memory; no network call besides the API
NFR-02	Forecast page and plan load ≤ 3 s	API-10 ≤ 200 ms (cached forecast row) · API-P2 ≤ 300 ms · page bundle lazy-loaded per route (≤ 250 KB gzipped)	Forecast rows reused by unique key; route-level code splitting in Vite
NFR-19	Full optimal plan ≤ 10 s	Forecasts ≤ 0.3 s · risk and quantities ≤ 0.1 s · optimizer ≤ 8 s hard limit (expected about 1 s) · validate and save ≤ 0.5 s	Evaluation budget + 8 s limit; runs in a worker thread

NFR	Target	Budget split (design estimate, p95)	How the design meets it
NFR-20	Network overview and history ≤ 3 s	API-09 ≤ 400 ms · API-17 ≤ 200 ms for 50 rows	History paginated and indexed on decided_at; network computed in one query plus cached forecasts
NFR-24	100 stations, 10 warehouses, 30 tankers: plan ≤ 30 s, edits ≤ 2 s	Optimizer limit raised to 25 s by config for this profile only	Each warehouse solved alone, so the problem grows per warehouse, not for the whole network

Every budget is checked by TC-PERF-01 to TC-PERF-04 and reported through OBS-M1, OBS-M2 and OBS-M7.

5.10.2 CPU and memory

- **Worker thread:** plan generation and re-plans run the routing engine in a `node:worker_threads` pool of 1 worker. Node.js runs JavaScript on one thread, so an 8 s calculation on the main thread would freeze every other request, including other users' edits.
- **Memory:** the largest objects are the 506-pair distance matrix and one plan (about 100 rows); the api container is sized at 512 MB, the db at 512 MB.
- **Database connections:** Prisma pool of 10 connections; the longest transaction (plan save) holds one connection under 0.5 s.

5.10.3 Concurrency and locking of a day's resources

Situation	Mechanism	Result
Dispatcher saves resources while the manager generates a plan for the same day	Both take `pg_advisory_xact_lock(hash('day:' + delivery_day))`	They run one after the other; the plan uses whichever version committed first and records its ID (FR-12)
Dispatcher saves resources while the last action is being approved	Same advisory lock; the decision sets `locked = true` in its transaction	If the decision commits first, the save gets 409 ERR-VAL-04; otherwise the plan shows "resources changed" and is decided on the version it was built from
Save after the day is decided	Service check + database trigger on resource_version	409 ERR-VAL-04, nothing written (BR-30)
Two managers, or two tabs, edit the same plan	Row lock on the plan (`SELECT … FOR UPDATE`) + `expected_version`	The second write gets 409 ERR-API-03 and reloads; no edit is silently lost
Approve while another tab moves a station	Same version check	Approval refused until the page shows the latest plan, so a manager never

Situation	Mechanism	Result
		approves a plan they have not seen
Many plan generations	Rate limit 6/min per user + worker pool of 1	Requests queue instead of overloading the CPU

All multi-row writes are single transactions (NFR-26); a failure leaves the previous data untouched.

5.11 Deployment view

Venzini v1 runs only on a local machine with Docker Compose: three containers, one published port, one command to start (NFR-30).

Figure 40 below shows the local deployment with Docker Compose. Only port 8080 is published; api and db are reachable only inside the Compose network.

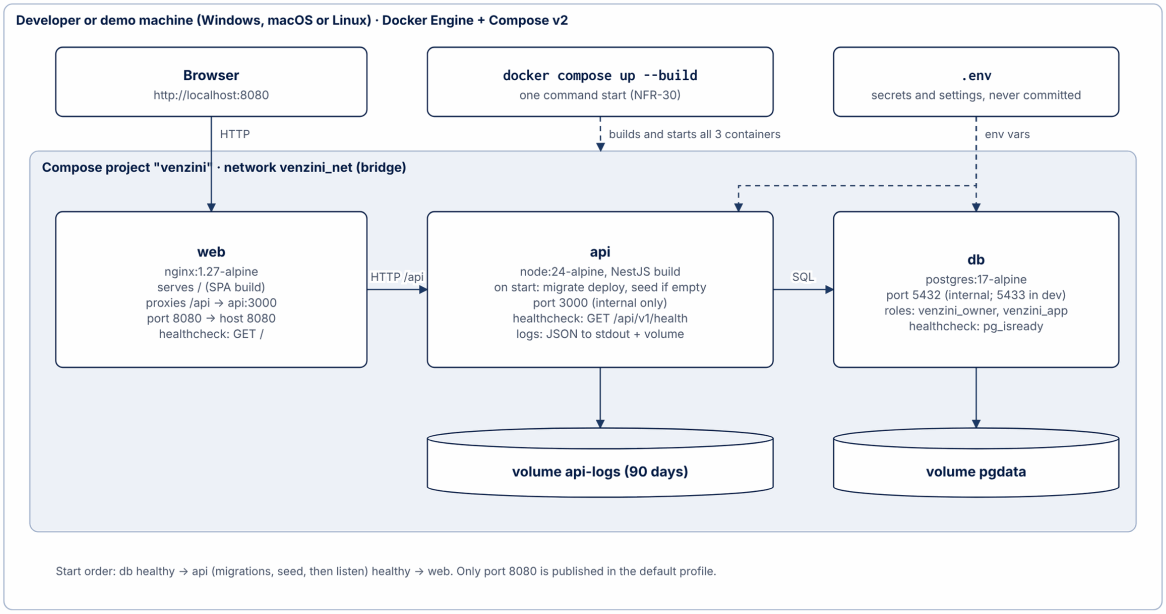


Figure 40 · Local deployment with Docker Compose

5.11.1 Ports

Service	Container port	Host port	Profile
web (nginx)	8080	8080	default
api (NestJS)	3000	– (3000 in dev profile)	default, dev
db (PostgreSQL)	5432	– (5433 in dev profile)	default, dev
Vite dev server	–	5173, proxies /api to localhost:3000	developer mode outside Docker

5.11.2 Environment variables

All settings come from a `.env` file read by Compose; `.env.example` in the repository lists every key with a safe placeholder. Secrets never go into the repository or the image.

Variable	Example	Used by	Meaning
POSTGRES_DB / POSTGRES_USER / POSTGRES_PASSWORD	venzini / venzini_owner / (secret)	db	Database and owner role (migrations only)
APP_DB_PASSWORD	(secret)	db init, api	Password of the runtime role venzini_app
DATABASE_URL	postgresql://venzini_app:…@db:5432/venzini	api	Runtime connection (no UPDATE/DELETE on the log)
MIGRATE_DATABASE_URL	postgresql://venzini_owner:…@db:5432/venzini	api start script	Used only by prisma migrate and seed
JWT_SECRET	32+ random bytes	api	Signs session tokens
SESSION_IDLE_MIN	30	api	NFR-22
LOCKOUT_MAX_FAILURES / LOCKOUT_WINDOW_MIN / LOCKOUT_DURATION_MIN	5 / 15 / 15	api	NFR-21
FORECAST_SEED / ROUTING_SEED	42 / 42	api	Determinism (NFR-13)
ROUTING_MAX_EVALS / ROUTING_TIME_LIMIT_MS	200000 / 8000	api	ALG-04 stop rules (NFR-19)
DISPLAY_TZ	Africa/Tunis	api, web	Local time for delivery days and display

Variable	Example	Used by	Meaning
PROVIDER_WEATHER / PROVIDER_EVENTS / PROVIDER_DIST	simulated	api	Adapter binding (NFR-15)
SEED_ON_START / SEED_SCENARIO	true / SC-01	api	Seed an empty database with a scenario set
DEMO_PASSWORD	(secret)	seed	Password of the three demo accounts
LOG_LEVEL	info	api	Pino level
VITE_USE_MOCKS	false	web build	true builds the MVP with MSW
VITE_MAP_TILE_URL	https://tile.openstreetmap.org/{z}/{x}/{y}.png	web build	Map background

5.11.3 Seed data

`apps/api/prisma/seed.ts` builds the SRS sample set deterministically from seed 42 (FR-74):

- **Network:** 3 warehouses and 20 stations (ST001–ST020) in the Greater Tunis area, spread over 3 regions and 3 size classes (AI-17); delivery windows between 05:00 and 20:00.
- **Fuel and tanks:** 2 fuel types (P95, DIESEL), one tank per station and fuel.
- **Fleet:** 6 tankers (T-01–T-06, two per warehouse) with 3 or 4 compartments each, and 6 drivers (D-01–D-06).
- **History:** 365 days of daily sales per station and fuel, with weekday and seasonal patterns, public holidays, about 2% missing days, plus matching weather and event rows.
- **Distances:** road km = straight-line km × 1.3, minutes = km at 45 km/h, for all 506 pairs (AS-08).
- **Accounts:** one demo account per role (manager, dispatcher, admin) using DEMO_PASSWORD.
- **Scenarios:** `pnpm seed --scenario SC-01…SC-08` loads the SRS scenario sets on top of the base data; `pnpm db:reset` restores the initial state (FR-74 alternative flow, outside the UI).

5.11.4 Startup steps

1. Copy `.env.example` to `.env` and fill the secrets.
2. Run `docker compose up --build`.
3. db starts; its init script creates the venzini_app role; the healthcheck waits for `pg_isready`.
4. api starts: `prisma migrate deploy` (owner role), then the seed if the database is empty, then the server on port 3000; the healthcheck calls `/api/v1/health`.
5. web starts when api is healthy and serves http://localhost:8080.
6. Log in with a demo account; Swagger is at http://localhost:8080/api/docs in the dev profile only.

NFR-23 (HTTPS) applies to non-local deployments only; for any future hosted demo, a TLS reverse proxy is placed in front of web and cookies keep the Secure flag (browsers already treat localhost as a secure context).

5.12 UI design view

Venzini has 8 desktop screens (UI-01 to UI-08) in one layout shell. Each role sees only its own menu items, and every screen follows the same rules: delivery-day picker and user in the header, no color-only meaning, a keyboard path for every action, minimum width 1366 px (SRS 13).

5.12.1 Layout shell

- **Left navigation (240 px, navy #021C45):** Venzini logo at the top, menu items for the role, logout at the bottom. Item labels are words with an icon, never icons alone.
- **Header (64 px, white):** page title, delivery-day picker (where relevant), the plan status chip, user name and role.
- **Content area:** 12-column grid, 24 px gutters, cards with 8 px radius and 1 px gray-300 borders, no shadows.
- **Messages:** errors and warnings appear next to the field or action they concern; page-level notices ("Resources changed", "Plan decided") are a banner under the header.

5.12.2 Screens

Screen	Layout	Main components	Visible to	Key FRs
UI-01 Login	Centered card with logo	Login, password with show/hide, Log in button, error area	Everyone	FR-01
UI-02 Network overview	KPI row; map (left 2/3) and "Stations at highest risk" table (right 1/3)	KpiCard ×4, RouteMap in network mode (markers only), RiskBadge, AlertTable, data-unavailable list	Manager	FR-14–16
UI-03 Station forecast	Selectors row; chart card; factor list and uncertainty panel side by side; Totals tab	StationSelect, FuelSelect, ConfidenceBadge, ForecastChart (line + band, actuals dashed), FactorList, UncertainPanel, TotalsTable + chart, "Show as table" toggle	Manager (both tabs)	FR-17–23, FR-83, FR-85
UI-04 Resources and trips	Day picker; tabs Resources, Trips, Warehouse forecast	TankerTable (availability checkboxes), DriverTable, StockTable, LastUpdatedLine, Save; TripTable with time inputs, gap and "Cancel trip" (with a reason, before departure only); Trips tab badge with the number of trips to record and the banner TPL-TRIP-01 once the plan is decided; WarehouseTotals	Dispatcher (edit); manager (read-only Resources and Trips)	FR-07–13, FR-79, FR-84, FR-85
UI-05 Delivery	Left: route cards and stop lists	RouteMap (routes, arrows, numbers, filter), StopList (drag, Move	Manager	FR-28–

Screen	Layout	Main components	Visible to	Key FRs
planner	(1/3); right: map (2/3); bottom bar: comparison panel, risk list, Reset, Go to approvals	up/down/to tanker), StationDetailsBox, RouteSummaryCard, ComparisonPanel, RiskList, UnservedList, QuantityEditor with limit hint		61, FR-75–77
UI-06 Review and approve	Info banner; actions table; legend; Approve all / Reject all	ActionsTable (Approve, Reject, Adjust per row), ConfirmDialog, OverloadWarning, RiskLegend	Manager	FR-67–71, FR-77, FR-80–82
UI-07 Decision history	Filters row; table; detail side panel	Search, DateRange, ActionFilter, HistoryTable, EntryDetail (original vs final, edits, risks, note), "View only" label	All roles	FR-72, FR-73
UI-08 Administration	Tabs Users and Stations	UserTable + UserForm (role select), StationTable + StationForm (tanks per fuel, window)	Admin (both tabs); manager (Stations tab)	FR-05, FR-06

5.12.3 Navigation

Figure 41 below shows the screen navigation map per role, from login to each role's home page and the main shortcuts between screens. The tag on each screen gives the role's rights on it: C create, R read, U update, D delete, a dash meaning not allowed.

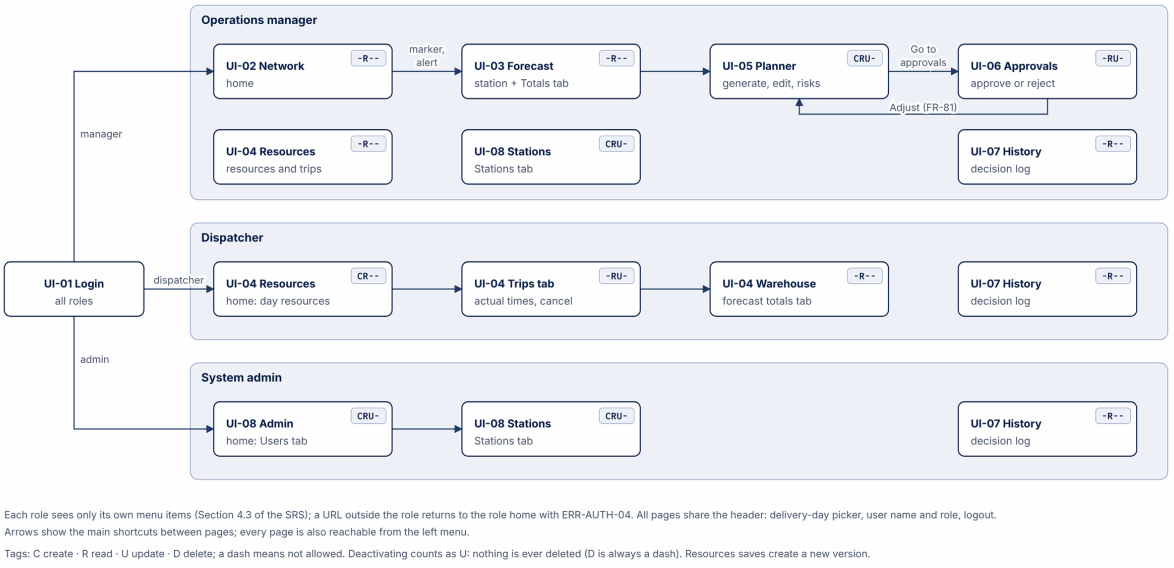


Figure 41 · Screen navigation map per role

Route (React Router)	Screen	Roles allowed	Home of
/login	UI-01	Public	–
/network	UI-02	manager	manager
/forecast, /forecast/totals	UI-03	manager	–
/resources, /resources/trips, /resources/warehouse-forecast	UI-04	dispatcher (all tabs); manager (resources, trips: read-only)	dispatcher
/planner?day&station	UI-05	manager	–
/approvals?day	UI-06	manager	–
/history, /history/{entryId}	UI-07	manager, dispatcher, admin	–
/admin/users, /admin/stations	UI-08	admin (both); manager (stations)	admin

A route guard reads `GET /me`; an unauthenticated user goes to /login and returns to the requested URL after login, if the role allows it (FR-01 alternative flow).

5.12.4 Design system

Figure 42 below shows the Venzini design system v1: brand and neutral colors, risk levels, uncertainty and status chips, the route line palette, the type scale and the controls. Contrast ratios are measured against white.

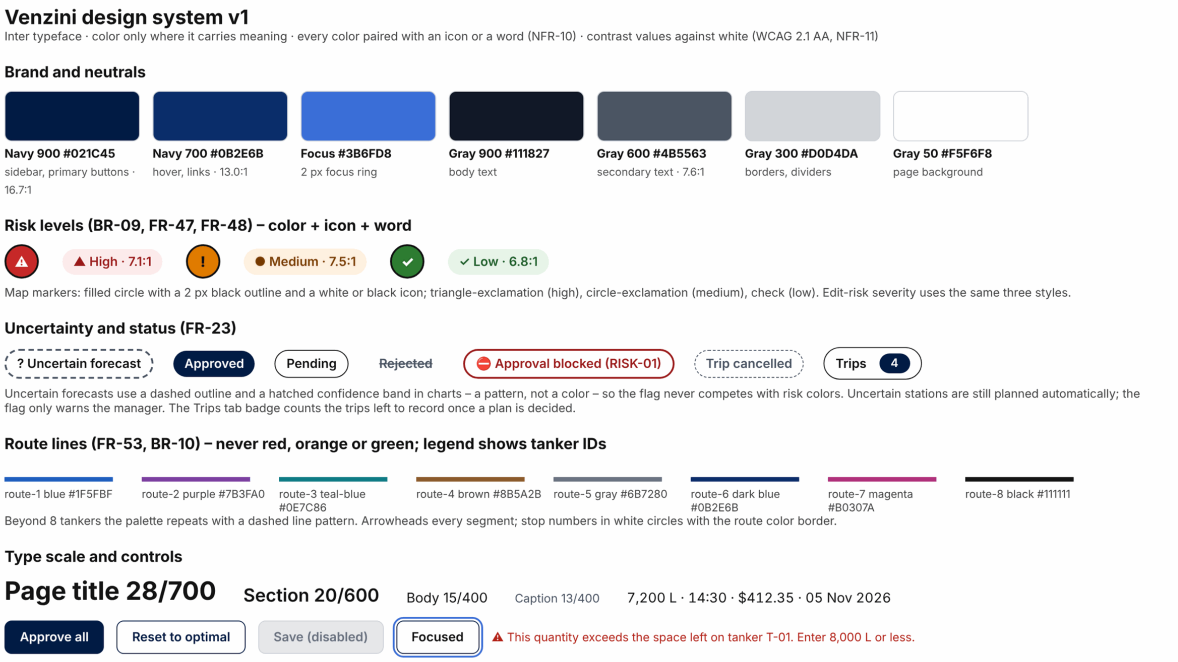


Figure 42 · Venzini design system

- **Tokens:** every color, radius, spacing and font size is a CSS variable in `tailwind.config.ts` (for example `--vz-risk-high`, `--vz-route-3`); components never use raw hex values.
- **Color rules:** navy #021C45 is the only brand color. The risk colors, high #C62828, medium #E07B00 and low #2E7D32, are reserved for risk (BR-09) and never used for routes or buttons (BR-10); every colored element also has an icon or a word (NFR-10).
- **Numbers:** tabular digits; litres with thousands separators and "L"; times 24 h; dates DD MMM YYYY; costs "$412.35" (NFR-31).
- **Uncertainty:** dashed outline, "?" icon and the words "Uncertain forecast" wherever the forecast is used: station page, planner stop, approval row (FR-23, AI-11). Uncertain stations are still planned automatically; the flag only warns the manager.

5.12.5 Screen states

State	Rule	Example
Loading	Skeleton after 300 ms; map keeps its last view while a plan reloads	Planner after an edit: numbers greyed until the answer
Empty	Plain sentence plus the next action	"No resources uploaded for this day" · "No decisions recorded yet."
Error	Message from SRS 14.2 next to the field or action; never a code alone	ERR-VAL-05a under the quantity input
Locked	Fields disabled, banner says why	"The plan for 03 Nov 2026 is already decided. This data can no longer be

State	Rule	Example
		changed."
Blocked approval	Approve buttons disabled with the overload warning beside them	ERR-PLAN-04 banner on UI-05 and UI-06
Trips to record	Badge with the count on the Trips tab; banner under the header once the day's plan is decided	"Plan for 03 Nov 2026 decided: 4 trips to record."
Map unavailable	Plain background, routes still drawn, notice	ERR-SYS-03

5.12.6 Accessibility

- **Keyboard (NFR-27):** every drag has a button path: in the stop list, Move up, Move down and "Move to tanker…" (a menu limited to tankers of the same warehouse); Enter opens a station's details; Esc closes boxes and dialogs; focus is always visible (2 px focus ring).
- **Map alternative:** the risk table on UI-02 and the stop list on UI-05 contain everything the map shows, so the map is never the only way to read or change data.
- **Charts:** every chart has a "Show as table" toggle; forecast and actuals differ by line style.
- **Screen readers:** labels tied to inputs, errors announced with `aria-live`, dialogs trap focus and return it on close (shadcn/ui primitives).
- **Checks:** axe-core in Playwright runs on every screen; grayscale screenshot review for TC-ACC-01 (AC-18).

5.12.7 Preliminary designs

Figure 43 below shows the preliminary screen designs made with Google Stitch, used as visual inspiration only.

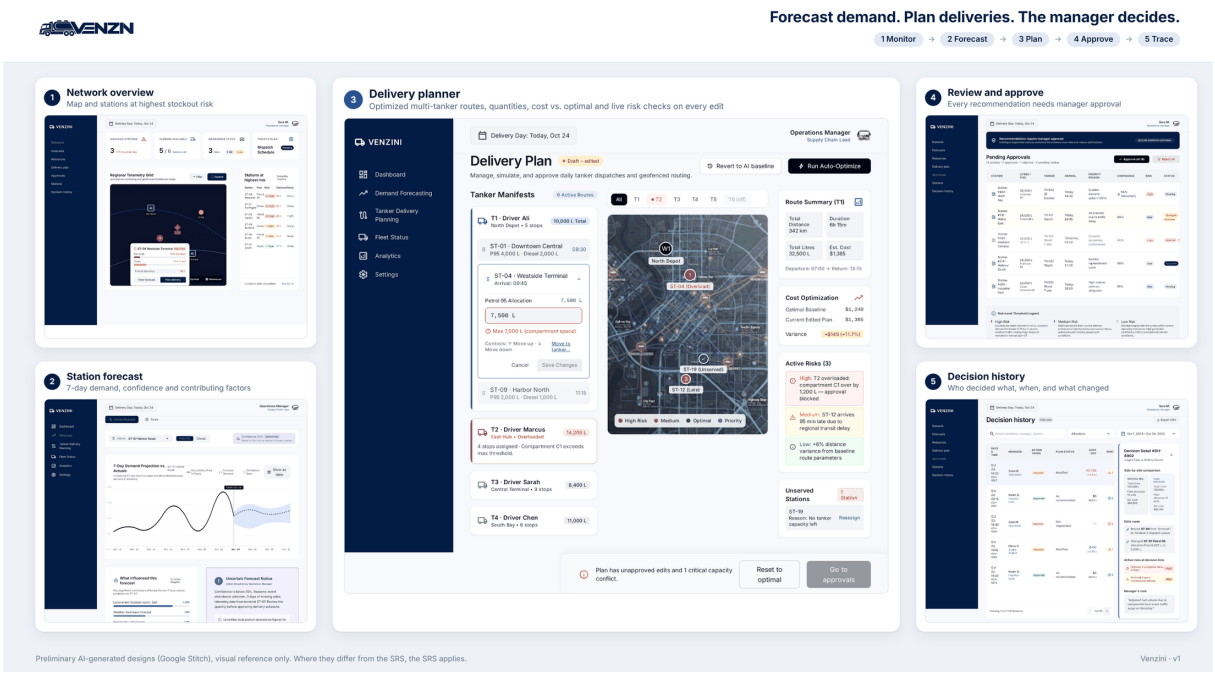


Figure 43 · Preliminary designs, visual reference only

Note: Where they differ from the SRS, the SRS applies.

Known differences that this design does not follow: extra fuel types, extra menu items (Fleet Status, Analytics, Settings), "neural/telemetry" wording, and a confidence-based risk legend. Risk in Venzini is set by time to stockout only (BR-26), and confidence is shown separately (BR-27).

6. Cross-cutting concerns

Security, errors, logs, configuration and time are handled once, in CoreModule and AuthModule, and every other module inherits them. No module writes its own error format, log format or role check.

6.1 Security

6.1.1 RBAC matrix (endpoint level)

The SRS permission matrix (Section 4.3) becomes `@Roles(...)` on each route; a global guard refuses everything not listed (default deny). The same matrix drives `GET /me`, so menus and server always agree.

Endpoint	Manager	Dispatcher	Admin
API-01 login, API-02 logout, API-03 me	✓	✓	✓
API-04 users	–	–	✓
API-05 stations (read and write)	✓	–	✓
API-06 GET resources	✓(read)	✓	–

Endpoint	Manager	Dispatcher	Admin
API-07 PUT resources	–	✓	–
API-08 PATCH trips, API-P3 GET trips	read (P3)	✓	–
API-09 network, API-10 forecasts	✓	–	–
API-19 totals	✓(network + warehouse)	✓(warehouse only)	–
API-11 to API-14, API-P2 plans and edits	✓	–	–
API-15, API-16 decisions	✓	–	–
API-17, API-18 history	✓	✓	✓
API-P1 reference	✓	✓	✓
Update or delete a log entry, set prices, control equipment	no endpoint	no endpoint	no endpoint

TC-SEC-01 calls every endpoint with every role and checks 200/201 or 403 against this table (AC-01).

6.1.2 Sessions and authentication

- **Login:** Argon2id verify; generic error for unknown login or wrong password; lockout after 5 failures in 15 minutes, for 15 minutes (NFR-21, Figure 16).
- **Token:** JWT signed with HS256 and JWT_SECRET, containing only `sub` (user ID), `role` and `sid` (session ID); lifetime 12 hours as an upper bound.
- **Cookie:** `vz_session`, HttpOnly, Secure, SameSite=Strict, Path=/api; never readable by page scripts.
- **Each request:** SessionGuard verifies the signature, loads the session row, refuses it if revoked or idle for more than 30 minutes (ERR-AUTH-05), then updates `last_seen_at` at most once a minute (NFR-22).
- **Logout and deactivation:** revoke the session row, so the token stops working at once even though it is not expired.
- **Role change:** revokes the user's sessions, so the new role applies at the next login.
- **Session warning:** the web app warns 2 minutes before idle expiry and offers "Stay signed in" (SRS 14.3).

6.1.3 Input validation and output safety

- **One schema, two checks:** the Zod schemas in `packages/contracts` validate forms in the browser and every request on the server (ZodValidationPipe); unknown fields are rejected, strings have maximum lengths, numbers have ranges.
- **Business limits** (BR-04 to BR-08, BR-28) are checked in services inside the write transaction, never trusted from the browser.
- **Database access** only through Prisma's parameterized queries; the few raw SQL statements (advisory locks) use tagged templates with bound parameters.

- **Output:** React escapes text by default; `dangerouslySetInnerHTML` is banned by lint rule; the rejection note is always shown as plain text.
- **Payload size:** JSON body limit 256 KB; larger requests get 413.

6.1.4 OWASP Top 10 baseline

Risk (OWASP 2021)	Control in Venzini
A01 Broken access control	Default-deny guard, roles on every route, ownership checks (a dispatcher cannot read station forecasts), TC-SEC-01 matrix test
A02 Cryptographic failures	Argon2id hashes; secrets only in .env; HTTPS required outside localhost (NFR-23)
A03 Injection	Prisma parameterized queries; Zod validation; React escaping
A04 Insecure design	Approval rules enforced server-side (BR-21, FR-71); append-only log at database level
A05 Security misconfiguration	Helmet headers (CSP default-src 'self' plus the tile server for images), no stack traces in errors, Swagger only in the dev profile, only port 8080 published
A06 Vulnerable components	pnpm audit and Dependabot alerts on GitHub; locked versions in pnpm-lock.yaml
A07 Identification and authentication failures	Lockout, idle expiry, server-side session revocation, no default passwords (DEMO_PASSWORD from .env)
A08 Software and data integrity failures	Lockfile, CI build from a clean checkout, migrations versioned in Git
A09 Security logging and monitoring failures	OBS-L3 security log for logins, lockouts, 403s and role changes
A10 Server-side request forgery	The API makes no outgoing HTTP calls in v1 (providers are simulated)

CSRF is covered by SameSite=Strict cookies plus the `X-Requested-With` header check (ERR-AUTH-06). CORS is disabled, because web and API share one origin.

6.2 Error handling

- **Domain errors:** services throw a typed `VenziniError(code, params, field?)`; a global exception filter maps the code to the HTTP status and message of SRS 14.2 and the error body of Section 5.6.1.

- **Unexpected errors:** logged at Error or Critical with the stack trace (in logs only) and returned as ERR-SYS-01 or ERR-SYS-04 without internal details.
- **Warnings** (WARN-RES-01, WARN-EDIT-01) are returned in a `warnings` array or as 422, never as silent success.
- **Recovery:** failed edits are undone (the transaction rolls back and the UI restores the previous view); a failed decision changes nothing; a plan that fails validation is never shown (SRS 14.3).
- **Frontend:** one error mapper turns error codes into the message under the right field or in a toast; a React error boundary per page shows "Something went wrong" with a Reload button and the correlation ID.

6.3 Logging and metrics

nestjs-pino writes one JSON object per line to stdout and to the api-logs volume. A correlation ID is created by the web app per request (`X-Correlation-Id`), accepted or generated by the API, and passed to every module call and worker (SRS 15.3).

Log	What is written	Where in code	Retention
OBS-L1 Request	time, correlation ID, user ID, role, method, route, status, duration ms	pino-http middleware	90 days
OBS-L2 Error	code, level, route, user ID, correlation ID, safe context	Global exception filter	90 days
OBS-L3 Security	logins, failures, lockouts, 403s, session expiry	AuthService, guards	90 days
OBS-L4 Admin change	user and station create, edit, (de)activate, with before and after (no passwords)	UserService, StationService	90 days
OBS-L5 Planning	plan ID, day, resource version, data and model versions, runtime, routes, unserved count, validator result, time_limited	PlanService	90 days

- **Redaction:** pino `redact` removes `password`, `password_hash`, cookies, `authorization` and JWTs from every log line (NFR-25); TC-SEC-06 scans the logs.
- **Metrics:** OBS-M1 to OBS-M8 are computed from these logs by a small script (`pnpm report:metrics`) after each test or demo session, so v1 needs no metrics server. Browser page-load times (OBS-M7) are sent with each request as a header from the Navigation Timing API and logged by OBS-L1.
- **Alerts:** ERR-SYS-02 and ERR-SYS-04 are logged at level fatal and printed highlighted on the api console; the metrics report flags slow edits and repeated lockouts (SRS 15.8).

6.4 Configuration

- All runtime settings are environment variables (Section 5.11.2), read once at start by a Zod-validated config service; a missing or invalid value stops the API at start with a clear message, not at the first request.

- Business values that may change over time (fuel price, unloading times) live in the `parameter` table with history, not in environment variables, so every plan can say which value it used.
- Rules from the SRS (thresholds 24 h / 72 h, 70% confidence, risk severity bands) are constants in `packages/contracts/src/rules.ts`, shared by API and web, each with a comment naming its SRS ID. They are not editable at runtime, because the SRS fixes them.

6.5 Time zones and units

Topic	Rule
Storage	All instants are `timestamptz` in UTC; delivery days and sales dates are `date` values meaning the local day in Africa/Tunis
Display	Converted to Africa/Tunis (DISPLAY_TZ) in the browser with `Intl.DateTimeFormat`; times 24 h "14:30"; dates "05 Nov 2026" (NFR-31)
Windows	Station delivery windows are local wall-clock times; the TimeEstimator converts them to instants for each day of a trip
Trip day	Every trip, first or second, starts and ends on the delivery day, between depot opening (05:00) and closing (22:00)
Daylight saving	Tunisia does not observe daylight saving time; the code still uses time-zone-aware conversion, so a future change needs no code change
Litres	Integers, thousands separator, "L" suffix: 34,000 L
Distance and time	km with one decimal; durations as "8 h 15 min"
Money	USD, stored with 3 decimals, shown with 2: $412.35 (D-20)
Percent	One decimal for differences (+6.2%), whole numbers for confidence (66%)

7. MVP design

The MVP is the real v1 frontend running against a mock API in the browser. It uses the same pages, components, hooks and contracts as v1; only the layer that answers `/api/v1` requests is fake (AC-21, D-26). Moving to v1 means turning one build flag off.

Figure 44 below compares the MVP and v1. Mock Service Worker (MSW) answers the frontend's real API calls from fixtures in the MVP; in v1 the same calls reach the NestJS API.

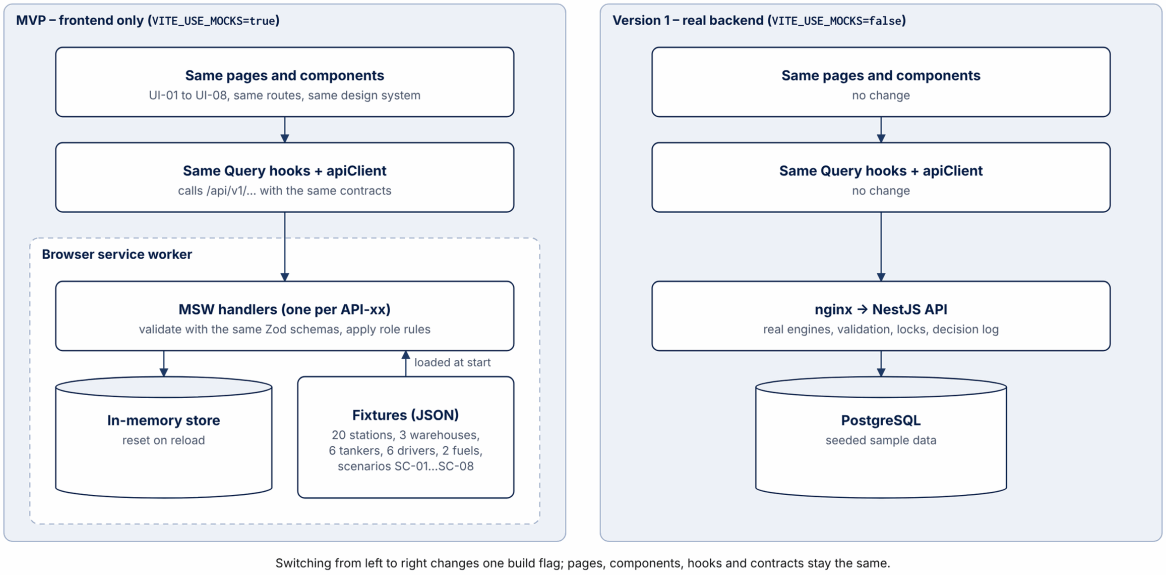


Figure 44 · MVP versus v1

7.1 Mock data layer

- **MSW handlers:** one handler per endpoint (API-01 to API-19, API-P1 to API-P3), in `apps/web/src/mocks/handlers/`. Each handler parses the request with the contract's Zod schema and returns a response that the same schema accepts, so a mock can never drift from the contract without a failing test.
- **Roles:** the login handler sets a mock session for the chosen demo account; handlers apply the RBAC matrix of Section 6.1.1, so a dispatcher in the MVP gets the same 403 as in v1.
- **State:** an in-memory store, filled from the fixtures at start, keeps what the user does during the session (approvals, saved resources, edits), so flows can be clicked through end to end. Reloading the page resets it; no data is saved (AC-21).
- **No real calculations:** handlers do not forecast, route or analyse risk. Edits return scripted outcomes (below); anything not scripted gets a simple generic answer (stops renumbered, values unchanged, "No risk" message).
- **Errors on demand:** a hidden developer toolbar (only in MVP builds) can switch scenario sets and force error responses (ERR-PLAN-01, ERR-VAL-04, ERR-SYS-01) to show every error state.

7.2 Fixtures

Fixtures come from `packages/sample-data`: one deterministic generator, seed 42, that later also feeds the Prisma seed. The MVP and v1 therefore show the same stations, tankers and history.

Fixture	Content	Used by
reference.json	2 fuel types, 3 warehouses, 3 regions	All screens
users.json	3 demo accounts (manager, dispatcher, admin) + 4 others for the admin list	UI-01, UI-08

Fixture	Content	Used by
network.json	20 stations with tanks, levels and risk (3 high, 5 medium, 12 low), 1 station with data unavailable	UI-02
forecasts/*.json	7-day forecasts for every station and fuel, including one uncertain (66%) and one short-history pair (similar-stations baseline)	UI-03
totals.json	Network and per-warehouse totals	UI-03, UI-04
resources.json	Tankers with compartments, drivers, stock for the demo day; one tanker with an open trip	UI-04
plans/optimal.json	Optimal plan for 03 Nov 2026: 6 routes, 17 stops, 2 unserved, 1 tanker on a second trip	UI-05, UI-06
plans/scripted-edits.json	Scripted outcomes: move ST032 to T-02 → RISK-01 overload; reorder on T-03 → RISK-04 medium; reorder on T-01 → no risk; quantity above limit → ERR-VAL-05a; reject ST014 → re-plan with ST019 served	UI-05, UI-06
trips.json	Trips of a decided past day: planned, departed and returned	UI-04 Trips
decisions.json	30 past decision log entries (as recommended, modified, not dispatched)	UI-07

7.3 Screens and flows included

Screen	In the MVP	Flows that can be clicked through
UI-01 Login	Yes	UC-01 for each role, wrong password, locked account (scripted)
UI-02 Network	Yes	UC-03: markers, alerts, details, go to forecast or planner
UI-03 Forecast and Totals	Yes	UC-04: change station and fuel, uncertain and short-history states, totals
UI-04 Resources, Trips, Warehouse forecast	Yes	UC-02 save with warning; UC-10 record departure and return; locked day
UI-05 Planner	Yes	UC-05 generate; UC-06 reorder, move, overload block, quantity limit, reset
UI-06 Approvals	Yes	UC-07 approve, reject with re-plan, adjust, approve all, blocked by overload

Screen	In the MVP	Flows that can be clicked through
UI-07 History	Yes	UC-08 list, filters, detail, read-only for all roles
UI-08 Administration	Yes	UC-09 users, UC-11 stations (admin and manager)

All 8 screens and all 11 use cases are covered, per role, which is what AC-21 asks for. The Should items (quantity editing, estimated vs. actual gap) are included as screens because AC-21 covers all eight screens; their logic is still built after the MVP (D-24).

7.4 Path to the real backend

1. **Contracts first:** the MVP freezes `packages/contracts`. Any change to a schema after that is a change to both the mock and the API, reviewed as one change.
2. **Backend by vertical slice,** each one replacing its MSW handlers: (a) Auth and Admin, (b) Data and seed, (c) Resources and Trips, (d) Forecast and Risk with Network, (e) Routing and Cost with Plans, (f) Edit, (g) Decisions and History.
3. **Per slice:** set `VITE_USE_MOCKS=false` for that slice only (MSW `passthrough` for its paths), run the Playwright flows of that slice against the real API, then delete nothing: the handlers stay for frontend unit tests.
4. **Contract tests** run the same request and response examples against MSW and against the API (Supertest); both must pass the same Zod schemas.
5. **Done** when the MVP Playwright suite passes unchanged with `VITE_USE_MOCKS=false` and no handler is active at runtime.

The MVP is accepted when AC-21 passes: every screen and use-case flow can be clicked through for each role with fake data (Section 8.4).

8. Testing strategy

Every SRS test (TC-FR-01 to TC-FR-85, TC-AI, TC-SEC, TC-PERF and the others) is mapped to a test level, a tool and a folder below, and every acceptance criterion AC-01 to AC-21 has a named check. Tests run on the SRS reference data set and the eight scenario sets SC-01 to SC-08.

8.1 Levels and tools

Level	Tool	Scope	Location	Gate
Unit – engines	Jest	ALG-01 to ALG-08 as pure functions: boundaries, determinism, constraint checks	apps/api/src/**/engine/*.spec.ts	≥ 80% lines on FC, RISK, ROUTE, COST, EDIT (NFR-28)
Unit – services	Jest + test database	Services with a real PostgreSQL in a container; one test per BR	apps/api/src/**/*.service.spec.ts	Every BR-01 to BR-31 has at least one test
Unit – frontend	Vitest + Testing	Components, hooks, formatters, quantity editor,	apps/web/src/**/*.test.tsx	≥ 70% lines on apps/web

Level	Tool	Scope	Location	Gate
	Library	role-based menus		
API / integration	Supertest	Every endpoint: success, validation error, 401, 403; contract shape against the Zod schema	apps/api/test/api/*.e2e-spec.ts	100% of endpoints covered
Contract	Vitest + Supertest	Same examples through MSW and the real API, validated by the same schema	packages/contracts/test	MSW and API agree
End to end	Playwright	UC-01 to UC-11 per role in Chrome, Firefox and Edge at 1366 × 768 (NFR-16)	e2e/*.spec.ts	All flows pass on SC-01; exceptions on SC-02 to SC-06
Accessibility	Playwright + axe-core	Every screen; keyboard-only run of UC-03 to UC-07	e2e/a11y/*.spec.ts	0 serious violations; NFR-27
Performance	Playwright timing + k6-style Node script (autocannon)	TC-PERF-01 to TC-PERF-04	perf/	NFR-01, -02, -19, -20, -24
Security	Supertest + scripts	TC-SEC-01 to TC-SEC-08, adversarial requests, log scan	apps/api/test/security	0 failures
Reliability	Jest	TC-REL-01 determinism (byte-equal JSON of two runs); TC-REL-02 fault injection on saves	apps/api/test/reliability	0 differences, 0 partial writes
AI evaluation	Jest + report script	TC-AI-01 to TC-AI-06 on held-back data and scenario sets	apps/api/test/ai	SRS 16.3 thresholds
Regression	GitHub Actions	All of the above on every push; plan snapshots for SC-01 to SC-08	.github/workflows/ci.yml	Snapshot changes must be explained in the commit
Usability	Observed	TC-USE-01 with 5 non-technical users; TC-USE-02	docs/usability	≥ 4 of 5 complete UC-

Level	Tool	Scope	Location	Gate
	sessions	template review		05 to UC-07 in ≤ 10 min

Coverage target: 80% lines on the core engine modules is the hard gate (NFR-28); 80% overall is a goal reported by CI.

8.2 Test data

- **Base set:** the seed of Section 5.11.3 (20 stations, 3 warehouses, 6 tankers, 6 drivers, 2 fuels, 365 days), generated by `packages/sample-data` with seed 42.
- **Scenario sets:** SC-01 to SC-08 as defined in SRS 16.1, each a small overlay script (for example SC-02 marks 2 tankers unavailable; SC-07 sets tank levels to give exactly 23, 24, 72 and 73 hours to stockout and confidence of 69% and 70%).
- **Held-back data:** the last 28 days of history are hidden from the forecast for TC-AI-01 and used as "actuals".
- **Small instances:** 10 generated problems with at most 8 stations, solved by exhaustive search, for the optimality gap test TC-AI-03b.
- **Isolation:** each API test file runs in its own database schema created from migrations, so tests never share state.

8.3 Key test cases by design element

Design element	Tests	Examples
ALG-02 thresholds	TC-FR-23, TC-AI-02	69% → uncertain, 70% → not; 23/24/72/73 h → high/medium/medium/low
ALG-03 priority	TC-FR-25, TC-FR-26	Not enough stock for everyone: high-risk stations are served first; on a tie the station unserved yesterday goes first, then the fewest hours to stockout
ALG-04 constraints	TC-AI-03	100 plans across SC-01 to SC-04 and random seeds: 0 violations of BR-04 to BR-08, BR-24, BR-25, BR-31
ALG-06 severity	TC-FR-63, TC-AI-04	60/61/180/181 min; 9.9/10/25/25.1%
ALG-07 limits	TC-FR-75, TC-FR-76	Each binding limit; max and max + 1 L, in the UI and by direct API call
Decision transaction	TC-FR-71, TC-REL-02	Make the log insert fail: no status change remains
Log immutability	TC-SEC-07	UPDATE and DELETE on decision_log_entry as venzini_app and through every role's API: all refused
Locking	TC-FR-11 + concurrency test	Save resources and approve the last action in parallel 50 times: never both succeed after the lock
Optimistic versions	API test	Two edits with the same expected_version: second gets ERR-API-03
Re-plan after rejection	TC-FR-82	Unchanged approvals kept, changed ones pending, freed capacity serves ST019 in SC-02

8.4 How AC-01 to AC-21 are verified

AC	Verified by	Level
AC-01 Access control	TC-SEC-01 matrix (3 roles × every endpoint, UI and API) + redirect test	API, E2E
AC-02 Resources and master data	TC-FR-06 to TC-FR-13; locked-day test	API, E2E
AC-03 Network overview	TC-FR-14 to TC-FR-16 (marker count, levels, alerts)	API, E2E
AC-04 Forecast	TC-FR-17 to TC-FR-20, TC-FR-85 (totals equal sums)	Unit, API, E2E

AC	Verified by	Level
AC-05 Confidence and uncertainty	TC-FR-21, TC-FR-23, TC-FR-83 on SC-06 and SC-07	Unit, E2E
AC-06 Risk and priority	TC-AI-02 boundaries; TC-FR-25 ranking; TC-FR-26 allocation order	Unit
AC-07 Explanations	TC-AI-04b groundedness, TC-AI-04c coverage	Unit
AC-08 Plan constraints	TC-AI-03 (100 plans, 0 violations); TC-FR-34 closest warehouse	Unit, API
AC-09 Multi-tanker routes and shortages	TC-FR-35 to TC-FR-39 on SC-01 to SC-03	API, E2E
AC-10 Route map and editing	TC-FR-45 to TC-FR-57, TC-FR-60; keyboard path; TC-PERF-01	E2E, Perf
AC-11 Quantity editing with limits	TC-FR-58, TC-FR-59, TC-FR-75, TC-FR-76	Unit, API, E2E
AC-12 Risk analysis	TC-AI-04 (one scenario per risk type + no-risk), TC-FR-44, TC-FR-65, TC-FR-66	Unit, E2E
AC-13 Overload blocks approval	TC-FR-77: 1 L overload, UI buttons disabled, API returns ERR-PLAN-04	API, E2E
AC-14 Approval	TC-FR-67 to TC-FR-71, TC-FR-80 to TC-FR-82 (5 actions → 6 log entries)	API, E2E
AC-15 Decision history	TC-FR-72, TC-FR-73, TC-SEC-07	API, E2E
AC-16 Cost	TC-FR-40 to TC-FR-43, TC-FR-52 (to $0.01)	Unit
AC-17 Performance	TC-PERF-01 to TC-PERF-03 on the reference environment	Perf
AC-18 Accessibility	TC-ACC-01 grayscale, TC-ACC-02 contrast (axe-core), TC-ACC-03 keyboard-only	E2E
AC-19 Trips	TC-FR-78 (to the minute), TC-FR-79, TC-FR-84 (+45 min, +9.4%)	Unit, API, E2E
AC-20 Sample data and determinism	TC-FR-74, TC-REL-01	API, Reliability
AC-21 MVP prototype	MVP Playwright suite: every screen and UC-01 to UC-11 per role with VITE_USE_MOCKS=true; checklist signed by the author with a recorded demo	E2E (MVP build)

9. Design rationale

The decisions below shape the whole design. Each one records the choice, the options considered and the trade-off accepted, in Architecture Decision Record (ADR) style. ADR-01 to ADR-05 restate the decided stack; the others are design choices of this SDD.

ADR	Decision	Alternatives considered	Why this one	Trade-off accepted	Refs
ADR-01	Modular monolith in NestJS, one module per SRS component	Microservices; plain Express app	One developer, one deployable; NestJS modules give enforced boundaries and dependency injection for replaceable engines	All modules scale together; acceptable for one distributor	D-12, NFR-12, CMP-*
ADR-02	Forecasting and routing as TypeScript engines inside the API	Python service (scikit-learn, OR-Tools); external solver	One language and one container fewer; engines are pure functions behind interfaces, so a Python or solver engine can replace them later	Fewer ready-made libraries; heuristics coded by hand	AIC-01, AIC-03, NFR-12
ADR-03	PostgreSQL 17 with Prisma	MongoDB; SQLite; TypeORM	Relational integrity for plans and the log; triggers and roles protect the log; Prisma gives typed queries and migrations	Advisory locks and triggers need small raw SQL migrations	NFR-14, NFR-26
ADR-04	REST/JSON with OpenAPI generated from Zod contracts	GraphQL; tRPC	Matches the SRS API list; contracts shared by web, API and mocks; Swagger for reviewers	More endpoints to maintain than one GraphQL schema	SRS 9
ADR-05	Local Docker Compose, three containers	Cloud hosting; single container	Local only (decided); one command on any OS	No public demo URL in v1	NFR-30
ADR-06	JWT in an HttpOnly cookie, checked against a session row	Pure stateless JWT; server sessions without JWT	Meets immediate logout and 30-minute idle expiry, which a pure JWT cannot; keeps the decided JWT	One database read per request (indexed, under 1 ms)	NFR-22, approved 26 Oct

ADR	Decision	Alternatives considered	Why this one	Trade-off accepted	Refs
ADR-07	Argon2id for passwords	bcrypt (suggested by the SRS)	Current OWASP recommendation, resistant to GPU guessing; same outcome as NFR-06	Slightly more memory per login (19 MiB)	NFR-06, approved 26 Oct
ADR-08	Decision log protected in the database (trigger + role without UPDATE/DELETE)	Application check only	A bug or a manual SQL session still cannot change history	Corrections are impossible by design; a wrong decision is fixed by a new decision	BR-13, NFR-14
ADR-09	Plans versioned as rows: optimal, edited copy, re-plans linked by parent	Edit the optimal plan in place; event sourcing	Reset restores the optimal plan exactly; the log can snapshot any version; simple to query	More rows per day (a few dozen)	FR-60, FR-72, FR-82
ADR-10	Resources saved as full versions per day	Edit rows in place	A plan always points to unchanging resources (AI-18); history of who changed what	Small duplication (about 15 rows per save)	FR-12, FR-13
ADR-11	Advisory lock per delivery day + optimistic plan versions	Table locks; no concurrency control	Serializes only the writes that conflict (resources vs decisions); tells a second editor to reload instead of losing work	Users may see ERR-API-03 and reload	BR-30, NFR-26
ADR-12	Optimizer stops on an evaluation budget, with the 8 s limit as a safety net	Time limit only	A time limit alone makes results depend on machine speed, which breaks NFR-13	A slow machine could hit the 8 s net first; the plan is then flagged	NFR-13, NFR-19
ADR-13	Optimizer runs in a worker thread	Main thread	Keeps edits of other users under 1 s while a plan is generated	Worker messaging code	NFR-01, NFR-19

ADR	Decision	Alternatives considered	Why this one	Trade-off accepted	Refs
ADR-14	Edits only recalculate; quantity edits and rejections re-plan	Re-optimize after every edit	Matches BR-12 and FR-82; keeps edits fast and predictable	The edited plan may stay worse than a re-optimized one; risks show it	BR-12, RK-01
ADR-15	Fixed sentence templates for all explanations	LLM-generated text	Required by D-11; every number traceable (AI-12)	Less varied wording	D-11, AIC-04
ADR-16	MVP = real frontend + MSW mock API	Separate clickable prototype (Figma, throw-away app)	Nothing is thrown away; contracts are tested before the backend exists	MSW handlers to maintain until v1 is complete	AC-21, D-26
ADR-17	One shared sample-data generator for MVP fixtures and Prisma seed	Hand-written fixtures	MVP and v1 show the same network; tests reuse it	Generator must stay deterministic (seeded)	FR-74, AI-17
ADR-18	Color only where it carries meaning; risk colors reserved	Brand colors on every state	Clear reading on maps; WCAG and grayscale checks pass	Plainer look than the preliminary designs	BR-09, BR-10, NFR-10

Appendix A. Traceability matrix

Every SRS requirement maps forward to the design elements that realize it and to the test that verifies it. Requirements sharing exactly the same design elements share a row; every ID appears.

A.1 Functional requirements

Requirement	Design elements	Tests
FR-01 Login required	MOD-AUTH, SessionGuard, API-01, API-02, TBL-01/02/03, SEQ-01, SM-04, UI-01	TC-FR-01, TC-SEC-03, TC-SEC-04
FR-02 Three roles, matrix	RolesGuard, Section 6.1.1, API-03, route guard (5.12.3)	TC-FR-02, TC-SEC-01
FR-03 Only managers decide	@Roles on API-15, API-16; DecisionService	TC-FR-03

Requirement	Design elements	Tests
FR-04 Only dispatchers change resources	@Roles on API-07, API-08	TC-FR-04
FR-05 Manage users	MOD-ADMIN UserService, API-04, TBL-01, SM-04, UI-08 Users	TC-FR-05
FR-06 Manage stations	StationService, API-05, TBL-06/07, UI-08 Stations	TC-FR-06
FR-07, FR-08, FR-09 Upload tankers, drivers, stock	ResourceService, API-06, API-07, TBL-11 to TBL-14, SEQ-07, UI-04	TC-FR-07, TC-FR-08, TC-FR-09
FR-10 Other resources	Not built in v1	TC-FR-10 (deferred)
FR-11 Update until decided	resource_version.locked, trigger, advisory lock (5.10.3)	TC-FR-11
FR-12 Latest resources used	plan.resource_version_id, PlanService	TC-FR-12
FR-13 Last update shown	resource_version.saved_by/saved_at, API-06	TC-FR-13
FR-14, FR-15, FR-16 Map, alerts, levels	MOD-RISK RiskService.network, API-09, UI-02, ALG-03	TC-FR-14, TC-FR-15, TC-FR-16
FR-17, FR-18, FR-19, FR-20 Forecast	MOD-FC, ALG-01, API-10, TBL-17, IF-WEATHER, IF-EVENTS, UI-03	TC-FR-17 to TC-FR-20, TC-AI-01, TC-AI-01b
FR-21, FR-23 Confidence, uncertain flag	ALG-02, forecast.uncertain (generated), UncertainFlag component	TC-FR-21, TC-FR-23
FR-22 Factors	ALG-01 factors, ALG-08 TPL-FC-01..07	TC-FR-22, TC-AI-04b
FR-83 Short sales history	forecast.baseline, ALG-01 similar-stations baseline, TPL-FC-09, PlanView.fallback_forecast	TC-FR-83
FR-85 Totals	ForecastService.totals, API-19, UI-03 Totals, UI-04 Warehouse tab	TC-FR-85
FR-24, FR-25 Risk, priority	ALG-03, RiskScorer	TC-FR-24, TC-FR-25, TC-AI-02
FR-26 Yield allocation	ALG-03 priority rule (Equation 16)	TC-FR-26
FR-27 Urgency reasons	ALG-08 TPL-RISK-H/M/L, stop.reason	TC-FR-27, TC-AI-04c
FR-28 to FR-34 Quantities	ALG-03 quantity, ALG-04, PlanValidator, TBL-18	TC-FR-28 to TC-FR-

Requirement	Design elements	Tests
and hard limits	to TBL-21	34, TC-AI-03
FR-35, FR-36 Routes per tanker, home warehouse	ALG-04, route unique (plan_id, tanker_id)	TC-FR-35, TC-FR-36
FR-37, FR-38, FR-39 Plan always, shortages, missing resource	ALG-04 steps 1 and 7, plan.unserved, ERR-PLAN-01/02, SEQ-02	TC-FR-37 to TC-FR-39, TC-AI-06
FR-40 to FR-43 Cost	MOD-COST ALG-05, TBL-26 parameter, PlanView.comparison	TC-FR-40 to TC-FR-43
FR-44 Saving with risks	ComparisonPanel + RiskList together (UI-05)	TC-FR-44
FR-45 to FR-54 Route map	RouteMap (Leaflet, polylinedecorator), route palette (Figure 42), StationDetailsBox, tanker filter	TC-FR-45 to TC-FR-54
FR-55, FR-56, FR-57 Reorder, move, recalculation	MOD-EDIT ALG-06, API-13, SEQ-03, dnd-kit StopList	TC-FR-55 to TC-FR-57, TC-PERF-01
FR-58, FR-59 Quantity edit, re-plan	API-13 quantity, PlanService.replanWithQuantities, SEQ-04	TC-FR-58, TC-FR-59
FR-60 Reset	API-13 reset, plan versioning (ADR-09)	TC-FR-60
FR-61 to FR-66 Risk analysis	ALG-06, TBL-22, TPL-EDIT-01..07, highlight on map	TC-FR-61 to TC-FR-66, TC-AI-04
FR-67, FR-77 Approval with warnings, overload block	risk.blocks_approval, DecisionService guard, ERR-PLAN-04	TC-FR-67, TC-FR-77
FR-68 to FR-71 Actions and approval	API-P2, API-15, API-16, SM-02, transaction with log	TC-FR-68 to TC-FR-71
FR-72, FR-73 Decision log, history	TBL-24, DecisionLog, API-17, API-18, UI-07	TC-FR-72, TC-FR-73, TC-SEC-07
FR-74 Sample data	packages/sample-data, prisma seed, SC-01..08	TC-FR-74
FR-75, FR-76 Limit hint, block save	ALG-07, API-14, QuantityEditor	TC-FR-75, TC-FR-76
FR-78 Estimated trip times	TimeEstimator (ALG-04 time part)	TC-FR-78
FR-79, FR-84 Actual times, gap	TripService, API-08, API-P3, SM-03, SEQ-08	TC-FR-79, TC-FR-84
FR-80 Approve or reject all	API-16, SEQ-06	TC-FR-80

Requirement	Design elements	Tests
FR-81 Adjust	Route /planner?station= (5.12.3)	TC-FR-81
FR-82 Rejection re-plan	DecisionService + PlanService.replanWithout, SEQ-05	TC-FR-82

A.2 Non-functional requirements and AI capabilities

Requirement	Design elements	Tests
NFR-01, NFR-02, NFR-19, NFR-20, NFR-24 Performance	5.10.1 budgets, worker thread (ADR-13), evaluation budget (ADR-12), indexes	TC-PERF-01 to TC-PERF-04
NFR-03 Availability	Compose healthchecks and restart policy	Session log
NFR-04 Scalability by data	Everything keyed by data, no hard-coded stations or fuels	TC-SCAL-01
NFR-05 RBAC	6.1.1	TC-SEC-01
NFR-06 Password hashing	Argon2id (ADR-07)	TC-SEC-02
NFR-07, NFR-25 Privacy	6.1, log redaction, driver table without contacts	TC-SEC-06, model review
NFR-08, NFR-09 Usability, plain language	5.12, ALG-08 catalogue	TC-USE-01, TC-USE-02
NFR-10, NFR-11, NFR-27 Accessibility	5.12.4, 5.12.6, Figure 42	TC-ACC-01 to TC-ACC-03
NFR-12 Replaceable modules	Engine interfaces (5.3), DI tokens	TC-MAINT-01
NFR-13 Determinism	Seeds, fixed order, evaluation budget	TC-REL-01
NFR-14 Log immutability	ADR-08, TBL-24 trigger and grants	TC-SEC-07
NFR-15 Interfaces for external data	IF-WEATHER, IF-EVENTS, IF-DIST (5.6.6)	Code review
NFR-16 Browsers	Playwright on Chrome, Firefox, Edge	TC-COMP-01
NFR-17, NFR-18 Compliance	No price or equipment endpoint; PlanValidator	TC-SEC-08, TC-AI-03
NFR-21, NFR-22, NFR-23 Lockout, expiry, HTTPS	LockoutPolicy, SessionGuard, TLS note (5.11.4)	TC-SEC-03 to TC-SEC-05
NFR-26 Atomic saves	Transactions everywhere (5.10.3)	TC-REL-02

Requirement	Design elements	Tests
NFR-28 Tested core	8.1 coverage gate	CI report
NFR-29 Observability	6.3 logs and metrics	TC-OBS-01
NFR-30 Portability	Docker Compose one command	TC-PORT-01
NFR-31 Units and formats	6.5	UI review
AIC-01 Forecasting	ALG-01, ALG-02	TC-AI-01, TC-AI-01b
AIC-02 Risk and priority	ALG-03	TC-AI-02, TC-AI-06
AIC-03 Routing	ALG-04, ALG-05	TC-AI-03, TC-AI-03b
AIC-04 Risk analysis and explanations	ALG-06, ALG-08	TC-AI-04, TC-AI-04b, TC-AI-04c

Appendix B. Repository and folder structure

One pnpm monorepo on GitHub. Each backend module folder matches one SRS component; engines sit in an `engine/` folder with no NestJS or Prisma import. The `README.md` at the root is the entry point for any new reader: it explains what Venzini is and how to install, run and test it.

```
venzini/
├─ apps/
│  ├─ web/                          # React 19 SPA (CMP-UI)
│  │  ├─ src/
│  │  │  ├─ app/                    # shell, router, route guards, providers
│  │  │  ├─ pages/                  # login, network, forecast, resources, planner,
│  │  │  │                          #   approvals, history, admin
│  │  │  ├─ features/               # route-map, stop-list, forecast-chart, risk,
│  │  │  │                          #   comparison, quantity-editor
│  │  │  ├─ components/ui/          # shadcn/ui components
│  │  │  ├─ api/                    # apiClient + TanStack Query hooks
│  │  │  ├─ stores/                 # Zustand stores
│  │  │  ├─ mocks/                  # MSW handlers + fixtures (MVP)
│  │  │  └─ lib/                    # formatters, error mapper
│  │  ├─ index.html · vite.config.ts · tailwind.config.ts
│  │  └─ Dockerfile                 # build, then nginx image
│  └─ api/                          # NestJS modular monolith
│     ├─ src/
│     │  ├─ core/                   # MOD-CORE: config, prisma, logging, errors, purge job,
│     │  │                          #   health
│     │  ├─ auth/                   # MOD-AUTH
│     │  ├─ admin/                  # MOD-ADMIN
│     │  ├─ resources/              # MOD-RES (resources, trips)
│     │  ├─ forecast/  └─ engine/   # MOD-FC + ALG-01, ALG-02
│     │  ├─ risk/      └─ engine/   # MOD-RISK + ALG-03
│     │  ├─ routing/   └─ engine/   # MOD-ROUTE + ALG-04, worker, validator
│     │  ├─ cost/      └─ engine/   # MOD-COST + ALG-05
│     │  ├─ edit/      └─ engine/   # MOD-EDIT + ALG-06, ALG-07
│     │  ├─ explanation/            # MOD-EXPL + template catalogue (ALG-08)
│     │  ├─ decision/               # MOD-DEC
│     │  └─ data/                   # MOD-DATA: repositories, simulated adapters IF-*
│     ├─ prisma/
│     │  ├─ schema.prisma
│     │  ├─ migrations/             # incl. raw SQL: triggers, roles, partial indexes
│     │  └─ seed.ts                 # uses packages/sample-data
│     ├─ test/                      # api, security, reliability, ai evaluation
│     └─ Dockerfile
├─ packages/
│  ├─ contracts/                    # Zod schemas API-01–19, provider interfaces, rules.ts
│  ├─ sample-data/                  # deterministic generator (seed 42), scenarios
│  │                                #   SC-01–SC-08
│  └─ config/                       # shared ESLint, Prettier, tsconfig
├─ e2e/                             # Playwright: flows per role, a11y, MVP suite
├─ perf/                            # performance scripts
├─ docs/                            # SRS, SDD, ADRs, usability notes
├─ docker-compose.yml · .env.example
├─ .gitignore                       # never committed: .env, node_modules/, builds,
│                                   #   reports, private data
├─ .dockerignore                    # kept out of Docker images: .env, .git/,
│                                   #   node_modules/, tests, docs
├─ README.md                        # what Venzini is; how to install, run and test it
├─ pnpm-workspace.yaml · package.json
└─ .github/workflows/ci.yml         # lint, type-check, tests, coverage, build
```

Main scripts: `pnpm dev` (Vite + API in watch mode, db in Docker) · `pnpm test` · `pnpm e2e` · `pnpm seed --scenario SC-0x` · `pnpm db:reset` · `pnpm report:metrics` · `docker compose up --build`.

Git workflow: main branch always green; one branch per vertical slice (Section 7.4); conventional commit messages; CI must pass before merge. The `.gitignore` keeps secrets (`.env`), downloaded libraries (`node_modules/`), build outputs and test reports out of the repository; only `.env.example` is shared. The sample-data generator and the MVP fixtures are committed, so every machine produces the same data. Real distributor data is never stored in the repository: v1 runs on sample data only, and any real data kept on a developer's machine goes in a local `private-data/` folder, which the `.gitignore` excludes together with database dumps (`*.dump`). The `.dockerignore` does the same for Docker, so secrets and local files never end up inside a container image.